Кейс Альфа-Банка · Хакатон «Альфа Будущее»

Умный кредитный кабинет предпринимателя

Документация проекта

Версия	Дата сборки	Репозиторий
v1.0.0	21 октября 2026 г.	github.com/BreezeAreaSay/alfa-hakaton

Содержание

1 Обзор проекта 5

Назначение 5
Быстрый старт 5
Требования к окружению 6
Запуск 6
Демо-версия без сервера 6
Архитектура 7
Конфигурация и переменные окружения 7
Используемые компоненты 8
Сценарий проверки 9
Тестовые данные 9
Тесты 10
ИИ-помощник 10
Документация 10
Структура репозитория 11

2 Продуктовое обоснование 12

Для кого 12
Задачи пользователей (Jobs to be Done) 12
Гипотезы и как мы их проверяем 12
Пути пользователей 13
Что вошло в MVP и почему 13
Приоритеты (MoSCoW) 15
Ключевые продуктовые решения 16
Как измерять успех 16
Развитие 16

3 Соответствие требованиям ТЗ 18

Постановка задачи (стр. 6) 18
Технические требования и формат сдачи (стр. 7) 18
Минимальный сквозной кредитный путь (стр. 13–15) 19
Критерии успеха (стр. 9) 20
Ограничения (стр. 10–11) 20
Уточнения организаторов 21

4 Архитектура 22

Схема 22
Компоненты 22
Основной сценарий в динамике 24
Как устроен конвейер 25
Обновление интерфейса без перезагрузки 25
Безопасность 25
Надёжность и сопровождение 26
Масштабирование и путь к боевому контуру 26
Почему так 27

5 Кредитный процесс 28

Границы процесса 28
Участники 28
Этапы 28
Статусы заявки 30
Правила переходов 31

Шаги конвейера 32
Таймеры 33
Уведомления 33
Упрощения 34

6 Кредитная политика и скоринг 35

Почему балльная карта 35
Каталог продуктов 35
Подбор продукта по задаче 36
Признаки для оценки 36
Скоринговая карта 37
Стоп-факторы 37
Флаги для ручного рассмотрения 38
Расчёт лимита 38
Матрица решения 39
Полномочия аналитика 39
Объяснение решения клиенту 40
Версионирование 40

7 Модель данных 41

ER-диаграмма 41
Таблицы 48
Статус заявки и история 51
Прогоны конвейера и очередь заданий 52
Оптимистическая блокировка 53
События: уведомления и SSE 53
Журнал действий 53
Идемпотентность 54
Документы и файлы 54
Деньги и даты 54
Номер заявки 55
Персональные данные 55
Тестовые данные 56

8 API 58

Где смотреть 58
Аутентификация и роли 58
Соглашения 59
Методы 62
Ключевой сценарий клиента 65
Запрос информации 69
Рабочее место аналитика 71
Управление демо-стендом 75
Поток событий (SSE) 77
Внутренний API симулятора 78

9 Тестовые сценарии 81

Тестовые учётные записи 81
Подготовка 82
Сценарии 82
Автоматические тесты 85

10 ИИ-помощник 86

Что делает модель 86
Модели 86
Как включить 87

Оценка качества 88
Ограничения 89
11 Интерфейс 90
Дизайн-система 90
Шрифт 90
Компоновка 90
Принципы 91
Экраны 91
12 Публикация на внешнем сервере 92
Что нужно 92
Установка 92
Перед показом 93
Обновление и остановка 93
Безопасность стенда 93
13 Демо-версия без сервера (GitHub Pages) 94
Что это и чем не является 94
Запуск на своём компьютере 95
Публикация на GitHub Pages 95
Как устроена имитация 95
Снимки ответов API 96
Как расширить имитацию при новых методах API 97
Проверки 98
Ограничения 98
14 Сценарий интервью с предпринимателями 99
Кого ищем 99
Правила 99
Вопросы 99
Как записать результат 100
Что делаем с результатами 100

В этот файл не вошли — они есть в репозитории:

- docs/plan.md — рабочий черновик плана, устарел;
- THIRD_PARTY_NOTICES — сторонние компоненты, модели, данные и смоделированные интеграции; длинные сгенерированные таблицы лицензий.

Документ 1 из 14 · README.md

Обзор проекта

Прототип цифрового кредитного конвейера для малого и микробизнеса — решение кейса Альфа-Банка на хакатоне «Альфа Будущее». Кабинет клиента и рабочее место кредитного аналитика связаны одной заявкой и одним процессом: от потребности бизнеса в деньгах до выдачи кредита.

Версия для проверки — тег `v1.0.0`.

Назначение

Предприниматель без банковской экспертизы описывает задачу своими словами и сразу видит подходящие продукты с платежом и переплатой. Заявка заполняется почти сама: данные компании подтягиваются из ЕГРЮЛ, финансы — из отчётности, БКИ и оборотов по счёту. Дальше заявка автоматически проходит проверку данных, внешние источники, скоринг и кредитную политику. Статус меняется на экране в реальном времени. Итог — не только «да» или «нет»: банк может предложить другие условия или запросить информацию. Подписание и выдача проходят прямо в кабинете.

Банк получает конвейер, который решает типовые заявки автоматически по прозрачным правилам, и рабочее место аналитика для остальных: очередь со сроками, досье со всеми источниками и объяснением скоринга, решения в пределах полномочий, журнал действий, контроль сбоев внешних систем.

Для клиента	Для банка
Подбор продукта по задаче, в том числе по описанию своими словами	Автоматический конвейер: проверка данных, ЕГРЮЛ/ЕГРИП, ФНС, ГИР БО, БКИ, обороты по счёту
Предварительный расчёт платежа и переплаты до заявки	Балльная скоринговая карта и кредитная политика с объяснением
Заявка с автозаполнением по ИНН	Четыре исхода: одобрение, контрпредложение, запрос информации, отказ
Статус и шаги проверки в реальном времени, уведомления в кабинете	Очередь, досье, решения аналитика с пересчётом условий
Понятные причины решения и альтернативы	Устойчивость к сбоям источников: повторы, ожидание, перезапуск
Чат с банком, подписание, выдача и график платежей	Журнал действий, сброс демо-данных, управление сбоями

Почему решение устроено так и что вошло в MVP — в docs/product.md.

Быстрый старт

```bash
git clone <адрес репозитория> alfa-hakaton
cd alfa-hakaton
git checkout v1.0.0
docker compose up -d --build --wait
```

Откройте http://localhost:8080 и войдите кнопкой быстрого входа, например под `veter` (пароль у всех — `demo`). Первая сборка занимает несколько минут.

Требования к окружению

Что	Версия	Зачем
Docker с Compose v2	Docker 24+ (Docker Desktop на Windows и macOS)	Запуск всех компонентов
Память	от 4 ГБ свободной	Сборка образов и работа стенда
Порт	8080 свободен (меняется через `APP_PORT`)	Веб-интерфейс
Браузер	Chrome, Edge, Firefox, Safari последних версий	Интерфейс рассчитан на экраны от 1280 px
Интернет	Только для первой сборки образов	Пакеты и базовые образы

Для разработки и запуска тестов без Docker: Python 3.12 и uv 0.11, Node.js 22.12 или новее.

Запуск

```bash
docker compose up -d --build --wait   # или make up
```

Команда собирает образы, применяет миграции, загружает справочники, тестовые компании и две демо-заявки на ручное рассмотрение и ждёт, пока все сервисы будут готовы.

Адрес	Что там
http://localhost:8080	Веб-интерфейс: кабинет клиента и рабочее место банка
http://localhost:8080/api/docs	Swagger UI — интерактивное описание API
http://localhost:8080/api/health	Проверка, что стенд работает

Полезные команды (Makefile, `make help`):

Команда	Что делает
`make reset`	Сбросить демо-данные: удалить заявки, снять сбои внешних систем. Без make: `docker compose exec api python -m app.seed --reset`
`make logs`	Логи всех сервисов
`make down` / `make clean`	Остановить стенд с сохранением данных / с удалением данных
`make test`, `make test-api`, `make e2e`	Юнит-тесты, сценарии через API, сценарии в браузере

Публикация на внешнем сервере с HTTPS — в docs/deploy.md.

Демо-версия без сервера

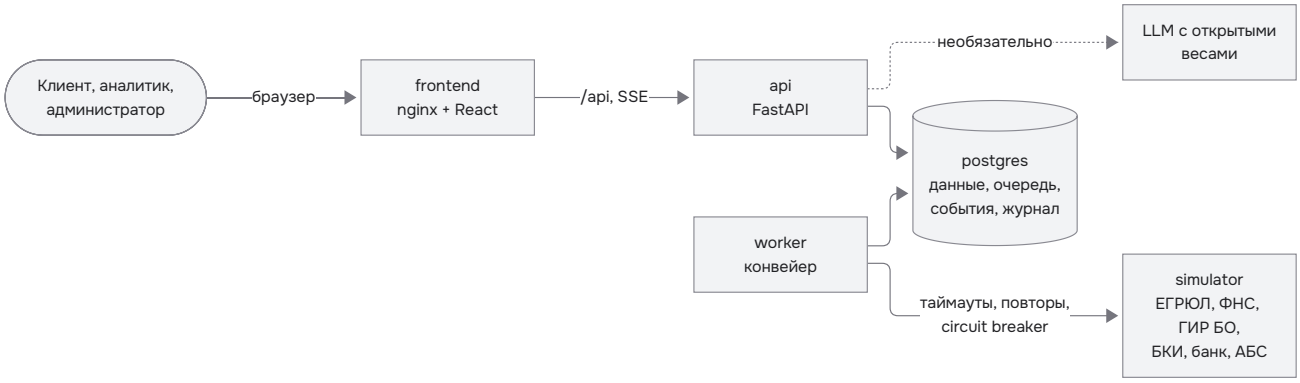
Для работы над интерфейсом есть сборка без бэкенда: запросы к API обслуживает имитация сервера прямо в браузере, данные хранятся в `localStorage`. Те же экраны, тестовые учётные записи и исходы заявок, что и на стенде, но без Docker. Проверяемая версия решения — по-прежнему стенд в Docker:

настоящая логика конвейера, скоринга и политики живёт в бэкенде.

Где	Как
На своём компьютере	`cd frontend && npm ci && npm run dev:demo` → http://localhost:5173
GitHub Pages	Workflow pages.yml публикует демо-версию при каждом изменении фронтенда в `main`. Один раз включите Settings → Pages → Source: GitHub Actions; адрес — `https://<владелец>.github.io/<репозиторий>/`

Чем имитация отличается от настоящего сервера, как её расширять при новых методах API и как обновлять снимки ответов — в docs/demo-mode.md.

Архитектура



Компонент	Что делает
`frontend`	Кабинет клиента и рабочее место банка на React и дизайн-системе Альфа-Банка; nginx раздаёт статику и проксирует API и поток событий
`api`	REST API на FastAPI: роли и вход, заявки, документы, предложения, подписание, рабочее место банка, поток событий (SSE), договоры в PDF
`worker`	Обработчик конвейера: проверка данных, внешние источники, скоринг, кредитная политика, формирование документов, выдача, таймеры сроков
`postgres`	Данные, очередь заданий (`SELECT … FOR UPDATE SKIP LOCKED`), события для интерфейса, журнал действий
`simulator`	Синтетические внешние системы для 10 тестовых компаний и управляемые сбои

Смена статуса, событие и задание обработчику сохраняются в одной транзакции, поэтому заявка не теряется и не обрабатывается дважды. Подробнее — схема, последовательность обработки, безопасность, надёжность и масштабирование — в docs/architecture.md.

Конфигурация и переменные окружения

Без настройки стенд работает на значениях по умолчанию. Чтобы изменить их, скопируйте .env.example в `.env` (файл не попадает в репозиторий).

Переменная	По умолчанию	Назначение
`APP_PORT`	`8080`	Порт веб-интерфейса на компьютере. На сервере — `127.0.0.1:8080`
`POSTGRES_PASSWORD`	`credit`	Пароль базы данных
`JWT_SECRET`	`change-me-in-production`	Секрет подписи токенов входа. На внешнем сервере обязательно заменить
`AI_ENABLED`	`false`	Включить ИИ-помощника
`LLM_BASE_URL`	`http://host.docker.internal:11434/v1`	Адрес модели с API в формате OpenAI: модель организаторов или локальная Ollama
`LLM_API_KEY`	пусто	Ключ доступа к модели организаторов
`LLM_MODEL`	`qwen3:4b`	Имя модели
`LLM_TIMEOUT_SECONDS`	`40`	Сколько ждать ответа модели, потом — разбор на правилах
`DOMAIN`	—	Домен для HTTPS при публикации на сервере (docs/deploy.md)

Внутренние адреса сервисов (`DATABASE_URL`, `SIMULATOR_URL`, `FILES_DIR`) задаются в docker-compose.yml. Таймеры процесса, повторы и пороги circuit breaker описаны в backend/app/config.py. Их можно переопределить переменными окружения с тем же именем в верхнем регистре, например `OFFER_TTL_HOURS`; для Docker такую переменную нужно добавить в блок `x-backend-env` файла `docker-compose.yml`.

Используемые компоненты

Слой	Компоненты
Бэкенд	Python 3.12, FastAPI, SQLAlchemy 2, Alembic, Pydantic 2, psycopg 3, PyJWT, bcrypt, httpx, ReportLab
Фронтенд	React 19, TypeScript, Vite 8, дизайн-система Альфа-Банка `@alfalab/core-components` (MIT), TanStack Query, React Router, шрифт Onest (OFL)
Инфраструктура	Docker Compose, PostgreSQL 16, nginx 1.27; для сервера — Caddy
Качество	pytest, ruff, Prettier, Playwright, GitHub Actions
ИИ (необязательно)	Модели с открытыми весами через API в формате OpenAI: Qwen организаторов или Qwen3 4B в Ollama

Версии зафиксированы lock-файлами: backend/uv.lock, simulator/uv.lock, frontend/package-lock.json, e2e/package-lock.json; базовые образы — точными тегами. Полный перечень сторонних компонентов, лицензий, моделей и смоделированных интеграций — в THIRD_PARTY_NOTICES. Платные сервисы и закрытые лицензии не используются.

Сценарий проверки

Тестовые учётные записи (пароль у всех — `demo`):

Логин	Кто	Что показывает
`veter`	ООО «Северный ветер»	Автоодобрение, подписание и выдача
`smirnova`	ИП Смирнова	Контрпредложение по овердрафту
`tehnomaster`	ООО «ТехноМастер»	Два варианта условий на выбор
`orlov`	ИП Орлов	Банк запрашивает выписку, затем одобряет
`vektor`	ООО «Вектор Логистик»	Ручное рассмотрение с флагами
`grand`	ООО «Гранд Девелопмент»	Кредит под залог, решает аналитик
`start`	ООО «Новый старт»	Молодой бизнес
`altair`	ООО «Альтаир»	Сбой внешнего источника
`romashka`, `meridian`	ООО «Ромашка-Строй», ООО «Меридиан»	Отказ по стоп-фактору
`analyst`, `analyst2`	Кредитные аналитики	Рабочее место банка
`admin`	Администратор демо	Сброс данных и сбои внешних систем

Основной сценарий (5 минут):

1. Войдите как `veter` → «Подобрать кредит».
2. В поле «Опишите задачу своими словами» напишите: «Нужно 3 млн на закупку товара к сезону, верну за 2 года» → «Подобрать по описанию». Цель, сумма и срок заполнятся, справа — подходящие продукты с платежом.
3. «Кредит на развитие бизнеса» → «Оформить заявку» → «Далее» → отметьте согласия → «Отправить заявку».
4. Смотрите, как заявка проходит шаги проверки без перезагрузки страницы. Через несколько секунд — «Кредит одобрен» и объяснение, что повлияло на условия.
5. «Принять предложение» → договор в PDF → «Подписать договор» (пароль `demo`) → «Кредит выдан» и график платежей.
6. Войдите как `analyst`: в очереди заявка ООО «Вектор Логистик» на ручном рассмотрении. Откройте досье — источники, скоринг по факторам, расчёт лимита, флаги — и примите решение: например, «Изменить условия» с обоснованием превышения лимита.
7. Войдите как `admin` → «Конвейер и сбои» и включите сбой бюро кредитных историй. Отправьте заявку от `altair`: она остановится на шаге «Кредитная история» с пометкой «Ждём ответа бюро кредитных историй». Верните источник в рабочее состояние — проверка продолжится с того же шага, заявка не потеряется.

Все сценарии по шагам с ожидаемыми результатами — в docs/test-scenarios.md. Перед повторной проверкой сбросьте данные: `make reset` или кнопка «Сбросить демо-данные» под `admin`.

Тестовые данные

Все компании, люди, документы, заявки и финансовые показатели синтетические и созданы командой:

- справочники, пользователи и демо-заявки — backend/app/seed/demo.py;

- ответы внешних систем для 10 компаний — simulator/sim/data/companies.json. Даты заданы относительно текущего дня, поэтому сценарии не устаревают;
- тестовые выписки по счёту в форматах 1С и CSV — frontend/public/demo-files.

Исход каждой заявки определяется данными компании, а не подменой в коде. Описание таблиц — в docs/data-model.md.

Тесты

Набор	Что проверяет	Команда
Юнит-тесты	Расчёты, скоринг, политика, разбор потребности, качество на эталонном наборе	`make test`
Сценарии через API	14 сквозных сценариев на поднятом стенде, включая проверки ошибок и доступа	`make test-api`
Сценарии в браузере	6 сценариев Playwright: путь клиента, решение аналитика, доступ по ролям	`make e2e`
Те же сценарии на демо-версии	6 сценариев Playwright на сборке без сервера и сверка имитации со снимками ответов API	`cd e2e && npm run test:demo`, `cd frontend && npm run demo:check`

Все наборы запускаются в GitHub Actions: линтеры и юнит-тесты, затем сборка образов, `docker compose up` и сценарии через API и в браузере (.github/workflows/ci.yml).

ИИ-помощник

Необязательное дополнение, по умолчанию выключено — основной сценарий работает без модели. С моделью помощник разбирает описание задачи клиента и пишет для аналитика сводку «сильные стороны / риски». Кредитных решений модель не принимает. Используются только модели с открытыми весами: Qwen от организаторов по ключу или локальная Ollama вне Docker. Установка, подключение и оценка качества — в docs/ai.md.

Документация

Документ	Содержание
docs/product.md	Пользователи, задачи, гипотезы, границы MVP и приоритеты
docs/architecture.md	**Архитектурная схема**, компоненты, безопасность, надёжность, масштабирование
docs/process.md	Кредитный процесс, **диаграмма состояний заявки**, правила переходов, таймеры
docs/credit-policy.md	Продукты, скоринговая карта, стоп-факторы, расчёт лимита, решение
docs/api.md	**Описание API**: роли, соглашения, методы, примеры, поток событий
docs/api/openapi.json	**Спецификация OpenAPI** (то же — http://localhost:8080/api/docs)
docs/api/postman_collection.json, docs/api/requests.http	**Коллекция запросов**: Postman и HTTP-клиент IDE

Документ	Содержание
docs/data-model.md	**Модель данных**: ER-диаграмма и описание таблиц
docs/test-scenarios.md	**Тестовые сценарии** и автотесты
docs/ai.md	ИИ-помощник: модели, подключение, оценка качества
docs/design.md	Интерфейс: дизайн-система, шрифт, компоновка, экраны
docs/deploy.md	Публикация на внешнем сервере с HTTPS
docs/demo-mode.md	Демо-версия без сервера: запуск, GitHub Pages, устройство имитации
docs/Документация-проекта.pdf	Вся документация одним PDF (пересборка — tools/docs-pdf)
docs/requirements.md	Соответствие требованиям ТЗ
THIRD_PARTY_NOTICES	Сторонние компоненты, модели, данные, сервисы и смоделированные интеграции

Структура репозитория

```
backend/      API и обработчик конвейера (FastAPI, SQLAlchemy, Alembic)
  app/domain/      скоринг, кредитная политика, финансовые расчёты
  app/pipeline/    очередь заданий и шаги конвейера
  app/api/         маршруты для клиента, банка, администратора
  tests/           юнит-тесты и сценарии через API
simulator/    симулятор внешних систем с синтетическими данными
frontend/     веб-интерфейс (React, дизайн-система Альфа-Банка), nginx
  src/demo/        имитация сервера для демо-версии без бэкенда
e2e/          сценарии в браузере (Playwright)
deploy/       публикация на сервере с HTTPS (Caddy)
docs/         документация
tools/        сборка документации в PDF
```

Документ 2 из 14 · docs/product.md

Продуктовое обоснование

Почему решение устроено именно так: для кого оно, какие задачи решает, что вошло в MVP и что сознательно оставлено за его границами.

Для кого

Предприниматель малого или микробизнеса — собственник, директор, финансовый директор или бухгалтер ООО либо ИП, который ищет финансирование или оформляет заявку от имени компании (ТЗ, стр. 5). Чаще всего у него нет банковской экспертизы и мало времени: кредит нужен под конкретную задачу бизнеса, а не «вообще».

С другой стороны процесса — банк: кредитный аналитик, который разбирает нестандартные заявки, и администратор процесса. Банку нужно обрабатывать много заявок с разными параметрами, решать типовые автоматически и не терять контроль над риском.

Задачи пользователей (Jobs to be Done)

№	Кто	Ситуация	Что хочет сделать	Чтобы
J1	Предприниматель	Нужны деньги под задачу: закупка к сезону, кассовый разрыв, оборудование, новая точка	Понять, какой продукт подходит, сколько дадут и сколько это стоит	Решить, брать ли кредит, до того как тратить время на заявку
J2	Предприниматель	Решил оформить кредит	Подать заявку быстро, не собирая справки и не вводя то, что банк может узнать сам	Не отвлекаться от бизнеса
J3	Предприниматель	Заявка отправлена	Видеть, что с ней происходит и что нужно от него	Не звонить в банк и не гадать
J4	Предприниматель	Банк не готов дать запрошенное	Понять причину и получить альтернативу	Всё равно решить задачу бизнеса
J5	Предприниматель	Условия устраивают	Подписать документы и получить деньги без визита в офис	Получить деньги вовремя
J6	Кредитный аналитик	Заявка не решилась автоматически	Видеть в одном месте данные всех источников, оценку риска и причину передачи	Принять обоснованное решение быстро и в пределах полномочий
J7	Банк	Поток типовых заявок	Решать их автоматически по прозрачным правилам и сохранять след каждого действия	Масштабировать выдачи без роста риска и ручного труда

Гипотезы и как мы их проверяем

Гипотезы сформулированы по ТЗ и публичным материалам банков о кредитовании бизнеса. Для проверки подготовлен сценарий интервью с предпринимателями: вопросы о последнем реальном случае, когда понадобились деньги, без подсказок решения.

Гипотеза	Как отражена в решении	Как проверить
Предприниматель думает задачей («закупить товар к сезону»), а не названием продукта	Подбор начинается с цели или описания своими словами; продукты предлагаются под задачу	Интервью: как человек формулировал потребность в последний раз
Главный страх — неизвестность после подачи заявки	Статус и шаги проверки на экране обновляются сами, уведомления в кабинете, понятный следующий шаг	Доля обращений «что с моей заявкой»
Повторный ввод данных о компании раздражает и приводит к ошибкам	Реквизиты, статус, руководитель, виды деятельности подтягиваются из ЕГРЮЛ/ЕГРИП; финансы — из отчётности, БКИ и оборотов	Время заполнения заявки, доля брошенных черновиков
Отказ без объяснения теряет клиента	Небинарный исход: контрпредложение с вариантами, запрос информации; причины простыми словами	Конверсия «контрпредложение → принято»
Аналитик тратит время на сбор данных из разных систем	Досье с данными источников, скорингом по факторам, расчётом лимита и следом решения	Время рассмотрения заявки

Пути пользователей

Клиент. Задача → подходящие продукты с платежом и переплатой → заявка с минимумом полей → автоматическая проверка в реальном времени → одно из четырёх: предложение, контрпредложение, запрос информации, отказ с причиной → подписание в кабинете → выдача и график платежей.

Аналитик. Очередь со сроками и фильтрами → досье: источники, скоринг, лимит, флаги, документы, история → решение в пределах полномочий: одобрить, изменить условия с пересчётом, запросить информацию, отказать → клиент сразу видит результат.

Сценарии по шагам — в test-scenarios.md.

Что вошло в MVP и почему

ТЗ рекомендует не воспроизводить весь банковский процесс, а выбрать реалистичный MVP и объяснить границы (стр. 8). Мы взяли весь путь от потребности до выдачи, но каждый этап — в упрощённом виде.

Этапы

Этап	В MVP	Почему
Подбор продукта и предварительный расчёт	Да	J1: решение «брать или нет» принимается до заявки. Без этого кабинет — просто анкета
Заявка с автозаполнением	Да	J2: минимум ввода — цель, сумма, срок, при необходимости залог и выручка
Проверка полноты и корректности	Да	Обязательный этап по ТЗ; ошибки ловятся до запросов во внешние системы
Сбор сведений из внешних источников	Да, через симулятор	ЕГРЮЛ/ЕГРИП, ФНС, ГИР БО, БКИ и обороты по счёту прямо названы в ТЗ как источники

Этап	В MVP	Почему
Скоринг и кредитная политика	Да	Ядро конвейера. Скоринг отделён от решения: одинаковый риск даёт разные условия для разных продуктов (ТЗ, стр. 14)
Ручное рассмотрение	Да	Нестандартные случаи требуют эксперта; без этого нет банковской стороны процесса
Запрос информации	Да	Альтернатива отказу, когда данных не хватает
Предложение и контрпредложение	Да	Небинарный исход по ТЗ (стр. 15)
Подписание и выдача	Да, смоделированы	Путь должен заканчиваться результатом; ТЗ допускает моделирование (стр. 11)
Обслуживание и погашение кредита	Нет	За пределами кредитного конвейера
Оценка залога	Нет	Отдельный процесс со своей экспертизой; клиент заявляет тип и стоимость, банк применяет дисконт

Роли

Роль	В MVP	Почему
Представитель клиента с правом подписи	Да	Основной пользователь по ТЗ
Кредитный аналитик	Да	Принимает решения по нестандартным заявкам
Администратор демо	Да	Сброс данных и управление сбоями для показа устойчивости
Несколько пользователей компании с разными правами	Нет	Важно для бизнеса («готовит бухгалтер, подписывает директор»), но не меняет логику конвейера — в развитии
Кредитный комитет, риск-менеджер, залоговая служба	Нет	Полномочия аналитика смоделированы лимитом (+20 % к расчётному лимиту с обоснованием); коллегиальные решения — в развитии

Данные

Принцип: не спрашивать у клиента то, что банк может узнать сам.

Данные	Откуда	Зачем
Цель, сумма, срок	Клиент	Параметры заявки
Выручка по оценке клиента	Клиент, необязательно	Сверка с отчётностью и оборотами
Тип и стоимость залога	Клиент, только для залогового продукта	Лимит по залогу
Реквизиты, статус, дата регистрации, ОКВЭД, руководитель	ЕГРЮЛ / ЕГРИП	Существование и возраст бизнеса, отраслевой риск
Налоговая задолженность, массовый адрес, дисквалификация	ФНС «Прозрачный бизнес»	Стоп-факторы и флаги
Выручка и прибыль за два года	ГИР БО (только ООО)	Финансовое положение и лимит

Данные	Откуда	Зачем
Кредиты, платежи, просрочки	БКИ	Долговая нагрузка и платёжная дисциплина
Поступления на счёт	Банк — если клиент обслуживается	Подтверждение выручки, лимит овердрафта
Выписка по счёту другого банка	Клиент — по запросу	Когда выручку нечем подтвердить (ИП без счёта в банке)
Согласия на запрос в БКИ и обработку персональных данных	Клиент	Обязательны перед запросом в БКИ

Проверки

Стоп-факторы (отказ без скоринга), флаги (на ручное рассмотрение), скоринговая карта из 9 факторов, классы риска A–E и ограничения лимита: максимум продукта, доля выручки, 80 % стоимости залога для залогового кредита, платёжеспособность (DSCR ≥ 1,3); для овердрафта — 40 % поступлений на счёт. Подробно — в credit-policy.md.

Приоритеты (MoSCoW)

Приоритет	Функция	Статус
Must	Вход для клиента и сотрудника банка	Готово
Must	Подбор продукта по задаче и калькулятор	Готово
Must	Заявка с автозаполнением из ЕГРЮЛ/ЕГРИП	Готово
Must	Автоматическая обработка: проверка данных, 5 источников, скоринг, политика	Готово
Must	Четыре исхода: одобрение, контрпредложение, запрос информации, отказ	Готово
Must	Ручное рассмотрение	Готово
Must	Предложение, подписание, выдача (смоделированы)	Готово
Must	Статус на лету и уведомления в продукте	Готово
Must	Docker, комплект документации по ТЗ	Готово
Should	Чат клиента с банком	Готово
Should	Контрпредложение с пересчётом условий	Готово
Should	Демонстрация сбоя внешнего источника	Готово
Should	Журнал действий в интерфейсе	Готово
Should	Объяснение факторов решения	Готово
Should	График платежей	Готово
Should	Сброс демо-данных	Готово
Could	ИИ: подбор по описанию и сводка досье	Готово, включается по желанию
Could	Принцип «четырёх глаз» для крупных сумм	В развитии

Приоритет	Функция	Статус
Could	Настройка продуктов и политики в интерфейсе	В развитии
Could	Несколько пользователей одной компании	В развитии
Won't	Реальные ЕСИА, ЭЦП, платежи и интеграции; мобильное приложение; оценка залога; обслуживание кредита	Вне MVP по ТЗ

Ключевые продуктовые решения

1. **Задача вместо анкеты.** Первый экран — «На что нужны деньги» и поле «Опишите задачу своими словами». Продукты подбираются под задачу с объяснением, почему какой-то продукт недоступен («Доступен бизнесу от 1 года»).
2. **Цена до заявки.** Ставка, платёж и переплата видны для каждого подходящего продукта ещё до отправки.
3. **Минимум ввода.** Всё, что можно узнать по ИНН и из источников, подтягивается автоматически.
4. **Небинарный исход.** Вместо отказа — контрпредложение с одним или двумя вариантами (больше срок или меньше сумма) либо запрос информации.
5. **Объяснимость.** Клиент видит, что повлияло на условия; аналитик — вклад каждого фактора, ограничения лимита и путь решения по правилам политики.
6. **Прозрачный статус.** Шаги проверки и статус меняются на экране без перезагрузки, уведомления — только в продукте: ТЗ исключает email, SMS и звонки из сценария.
7. **Устойчивость.** Сбой внешней системы не теряет заявку: клиент видит, что ждём ответа, аналитик — список сбоев и кнопку повтора.
8. **Полномочия аналитика.** Превышение расчётного лимита — до 20 % и только с обоснованием, всё фиксируется в журнале.
9. **ИИ там, где есть ценность.** Модель разбирает описание задачи и пишет сводку для аналитика, но не принимает кредитных решений (ai.md).

Как измерять успех

Метрики, которые стоит отслеживать в пилоте:

Метрика	Что показывает
Время от отправки до решения (медиана)	Скорость конвейера
Доля автоматических решений	Эффективность правил и данных
Доля заявок с запросом информации и время ответа клиента	Полнота данных на входе
Конверсия «предложение → подписание», в том числе для контрпредложений	Ценность небинарного исхода
Доля брошенных черновиков и время заполнения	Удобство подачи
Время аналитика на заявку и доля просрочек SLA	Эффективность рабочего места
Обращения «что с моей заявкой»	Прозрачность статуса

Развитие

- Несколько пользователей компании с ролями «готовит» и «подписывает».

- Принцип «четырёх глаз» и кредитный комитет для крупных сумм.
- Редактор продуктов и параметров политики с версионированием.
- Реальные интеграции через адаптеры с теми же контрактами, вход через Альфа-Бизнес или ЕСИА, УКЭП.
- Распознавание загруженных документов и сверка с данными источников.
- Статистическая модель риска поверх балльной карты после накопления истории решений.

Документ 3 из 14 · docs/requirements.md

Соответствие требованиям ТЗ

Где в решении выполнено каждое требование кейса «Умный кредитный кабинет предпринимателя». Номера страниц — по документу ТЗ.

Постановка задачи (стр. 6)

Требование	Как выполнено	Где посмотреть
Исследовать предметную область и потребности пользователей	Задачи пользователей (JTBD), гипотезы и способы их проверки, сценарий интервью с предпринимателями	product.md, research/interview-guide.md
Спроектировать модель кредитного процесса: границы, этапы, действия участников, данные, результаты	Границы, участники, 11 этапов, 14 статусов, правила переходов, таймеры, упрощения; данные и проверки на каждом этапе	process.md, credit-policy.md
Личный кабинет клиента: выбор продукта и все этапы от подачи заявки до результата	Подбор по задаче, расчёт, мастер заявки, статус в реальном времени, ответ на запросы, предложение, подписание, выдача и график	Сценарий C1 в test-scenarios.md
Этапы обработки со стороны банка и модель принятия решения	Автоматический конвейер из 8 шагов, скоринговая карта, кредитная политика, ручное рассмотрение в рабочем месте аналитика	process.md, credit-policy.md, сценарии C5–C7
Единая бизнес-логика: данные сохраняются, система реагирует на действия, состояние заявки отображается динамически	Одна база и одна машина состояний для клиента и банка; события по каждому изменению доставляются в браузер потоком SSE	architecture.md

Технические требования и формат сдачи (стр. 7)

Требование	Как выполнено	Где посмотреть
Код в GitHub с историей коммитов, однозначная версия	История коммитов по этапам работы, тег `v1.0.0`	Репозиторий
Cloud-native, запуск в Docker, единый воспроизводимый способ запуска на обычном ПК	Все компоненты в `docker compose`, запуск одной командой, проверки готовности сервисов	README, docker-compose.yml
Публикация в облаке для демонстрации (желательно)	Готовая конфигурация HTTPS-публикации на внешнем сервере	deploy.md, deploy/
Версии языка и библиотек зафиксированы	`uv.lock` для Python, `package-lock.json` для npm, точные теги базовых образов, версия uv и Node.js в образах	README

Требование	Как выполнено	Где посмотреть
README: назначение, архитектура, требования к окружению, запуск, конфигурация и переменные окружения, компоненты, пошаговый сценарий проверки	Все разделы	README
Синтетические тестовые данные и учётные записи для сквозного сценария	10 тестовых компаний со своими сценариями, аналитики и администратор, тестовые выписки	README, test-scenarios.md
THIRD_PARTY_NOTICES: компоненты, модели, датасеты, сервисы, смоделированные интеграции	Файл в корне репозитория	THIRD_PARTY_NOTICES
Архитектурная схема	Схема компонентов, последовательность обработки заявки	architecture.md
Описание API	Роли, соглашения, методы, примеры, поток событий	api.md
OpenAPI / Swagger	Спецификация в репозитории и Swagger UI на стенде	api/openapi.json, http://localhost:8080/api/docs
Коллекция запросов	Postman-коллекция с проверками и HTTP-файл для IDE	api/postman_collection.json, api/requests.http
Диаграмма состояний заявки	Mermaid-диаграмма и таблица переходов	process.md
Модель данных	ER-диаграмма и описание таблиц	data-model.md
Тестовые сценарии	13 сценариев с шагами и ожидаемым результатом, автотесты через API и в браузере	test-scenarios.md
Взаимодействие с работающим прототипом, а не просмотр экранов	Живой стенд: все действия меняют данные, исход определяется данными компаний	README

Минимальный сквозной кредитный путь (стр. 13–15)

Элемент пути	Как выполнено
Потребность → выбор продукта → заявка с суммой, сроком, целью и сведениями о клиенте	Подбор по цели или описанию, расчёт для подходящих продуктов, заявка с данными компании из ЕГРЮЛ/ЕГРИП
Приём, проверка полноты и корректности, приведение к единой форме, уточнение	Шаг «Проверка данных», признаки для скоринга из всех источников, статус «Нужна информация» с перечнем и сроком
Обогащение из внешних источников: БКИ, ЕГРЮЛ/ЕГРИП, ФНС «Прозрачный бизнес»	Шаги конвейера к симулятору этих систем, а также ГИР БО и данным банка; повторы, ожидание и circuit breaker при сбоях

Элемент пути	Как выполнено
Скоринг отдельно от кредитного решения	Балльная карта из 9 факторов → класс риска; решение учитывает продукт, сумму, срок, стоп-факторы, флаги и лимит. Один класс риска даёт разные условия для разных продуктов
Небинарный итог: изменить сумму или срок, предложить другой продукт, запросить информацию	Автоматические контрпредложения (больше срок или меньше сумма), решение аналитика с выбором другого продукта и пересчётом условий, запрос информации
Оформление документов и предоставление финансирования	Договор и график в PDF, простая электронная подпись в кабинете, выдача через симулятор АБС, кредит и график платежей у клиента

Критерии успеха (стр. 9)

Критерий	Чем подтверждается
Целостность решения	Сквозной путь от потребности до выдачи; 14 сценариев через API, 6 в браузере и Postman-коллекция проходят автоматически
Готовность к проверке	Запуск одной командой, тестовые учётные записи с быстрым входом, сброс данных, пошаговые сценарии
Качество пользовательского опыта	Подбор по задаче, расчёт до заявки, автозаполнение, статус в реальном времени, объяснение решений, интерфейс на дизайн-системе Альфа-Банка (design.md)
Согласованность модулей	Одна машина состояний, данные заявки используются на всех этапах, переходы проверяются централизованно
Enterprise-ориентированность	Транзакционная очередь заданий, идемпотентность, оптимистическая блокировка, повторы и circuit breaker, ролевой доступ, журнал действий, JSON-логи, миграции, CI (architecture.md)
Потенциал развития	Путь к боевому контуру и план развития (architecture.md, product.md)

Ограничения (стр. 10–11)

Ограничение	Как соблюдено
Комплексность	Клиентский путь, кредитная логика и инженерные практики в одном решении
Независимость от платных ресурсов	Только свободные компоненты, без платных API, подписок и покупных данных (THIRD_PARTY_NOTICES)
Синтетические данные	Все данные созданы командой
Автономность	Доступ к системам банка не нужен; внешние проверки — в симуляторе
Сценарий внутри продукта	Уведомления, чат, документы, подписание и выдача — только в кабинете; email, SMS, мессенджеры и звонки не используются
Допустимые упрощения описаны	Раздел «Упрощения» в process.md
Веб-интерфейс, не мобильное приложение	Веб-приложение для браузера
LLM только с открытыми весами	Qwen организаторов или Qwen3 4B в Ollama; ИИ по умолчанию выключен (ai.md)

Уточнения организаторов

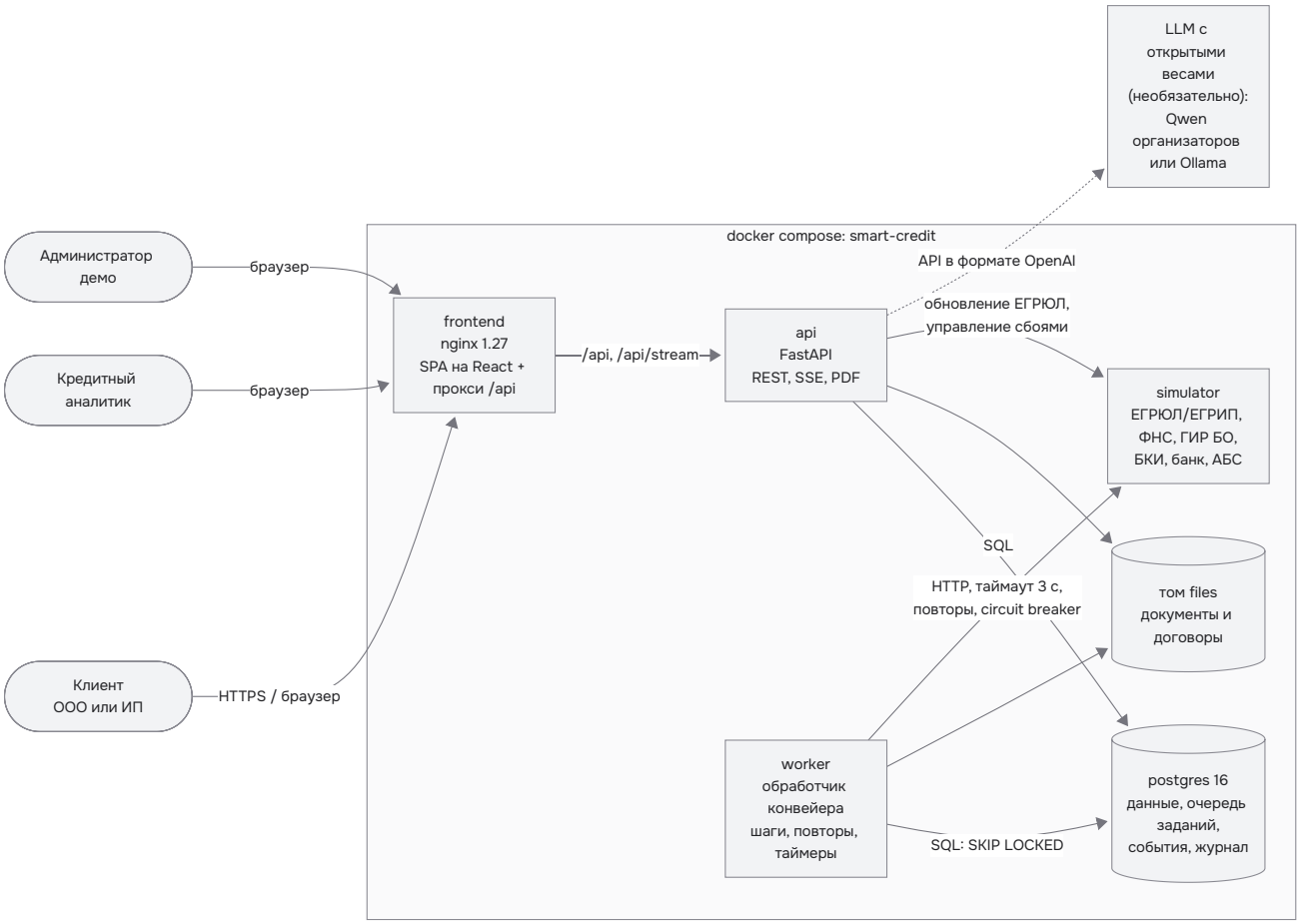
Уточнение	Как учтено
Демонстрация на бесплатном внешнем сервере на случай проблем с запуском	deploy.md
Ollama можно запускать вне Docker, описав установку, модель и подключение	ai.md
Логика скоринга и тестовые данные — на усмотрение команды, допускается упрощённая логика	credit-policy.md
Адаптивная вёрстка не обязательна	Интерфейс рассчитан на экраны от 1280 px (design.md)

Документ 4 из 14 · docs/architecture.md

Архитектура

Решение — модульный монолит на Python с отдельным обработчиком конвейера, веб-клиентом на React и симулятором внешних систем. Всё запускается одной командой `docker compose up` на обычном ПК, без доступа к системам банка и без платных сервисов.

Схема



Наружу открыт только порт `frontend` (по умолчанию 8080). База, API, обработчик и симулятор доступны лишь внутри сети Docker.

Компоненты

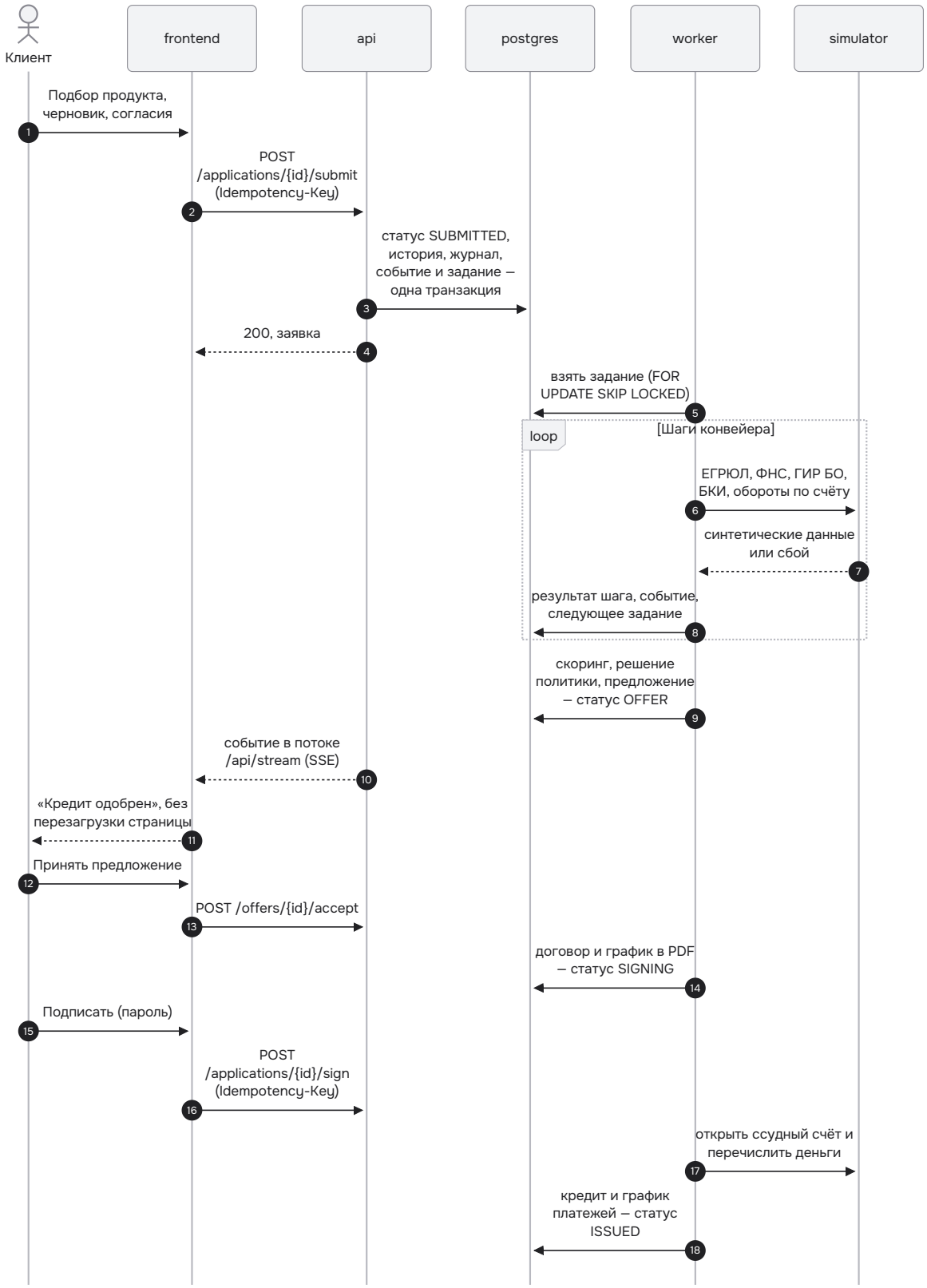
Сервис	Технологии	Отвечает за	Масштабирование
`frontend`	React 19, TypeScript, Vite 8, дизайн-система Альфа-Банка `@alfalab/core-components`, TanStack Query; nginx	Кабинет клиента и рабочее место аналитика; раздаёт статику, проксирует `/api` и поток событий без буферизации	Статика, любое число копий за балансировщиком

Сервис	Технологии	Отвечает за	Масштабирование
`api`	Python 3.12, FastAPI, SQLAlchemy 2, Alembic, Pydantic 2, PyJWT, bcrypt, ReportLab	Вход и роли, заявки, документы, предложения, подписание, рабочее место банка, справочники, поток событий (SSE), ИИ-функции	Без состояния в памяти (JWT), горизонтально
`worker`	Тот же образ, `python -m app.pipeline.worker`	Шаги конвейера: проверка данных, внешние источники, скоринг, политика; формирование договора; выдача; таймеры сроков	Горизонтально: задания разбираются через `SELECT … FOR UPDATE SKIP LOCKED`
`postgres`	PostgreSQL 16	Данные заявок и компаний, очередь заданий, события для уведомлений, журнал действий, ключи идемпотентности	Один экземпляр в демо; в боевом контуре — кластер
`simulator`	FastAPI	Синтетические ответы внешних систем для 10 тестовых компаний и управляемые сбои (ошибка, таймаут, медленный ответ)	Не требуется

Код разделён на слои (backend/app):

Слой	Каталог	Содержимое
Домен	`domain/`	Каталог продуктов, признаки, скоринговая карта, кредитная политика, финансовые расчёты. Чистые функции без базы и сети, покрыты юнит-тестами
Сервисы	`services/`	Машина состояний заявки, события, журнал, документы и разбор выписок, идемпотентность, ИИ-помощник
Конвейер	`pipeline/`	Очередь заданий, шаги, обработчик
Интеграции	`integrations/`	Клиент внешних систем с таймаутами, повторами и circuit breaker
API	`api/`	Маршруты по ролям: клиент, банк, администрирование, уведомления, ИИ

Основной сценарий в динамике



Статусы, переходы и таймеры описаны в process.md, правила скоринга и решения — в credit-policy.md, таблицы — в data-model.md, API — в api.md.

Как устроен конвейер

- **Единая точка смены статуса.** Все переходы проходят через `services/workflow.py` и проверяются по таблице допустимых переходов с учётом роли. Смена статуса, запись в историю, журнал действий, событие для интерфейса и задание обработчику сохраняются в одной транзакции (transactional outbox): если транзакция откатилась, задание не появится, а если зафиксировалась — не потеряется.
- **Очередь заданий в PostgreSQL.** Обработчики забирают задания через `SELECT … FOR UPDATE SKIP LOCKED`, поэтому несколько копий `worker` не мешают друг другу. Задание, зависшее у упавшего обработчика дольше 120 секунд, возвращается в очередь.
- **Повторы.** Ошибка шага приводит к повтору с растущей паузой: от 2 до 30 секунд, не более 8 попыток. После последней попытки аналитики получают событие «Ошибка конвейера», а заявка видна на странице «Конвейер и сбои».
- **Circuit breaker.** После 5 ошибок подряд вызовы источника приостанавливаются на 20 секунд, чтобы не нагружать упавшую систему.
- **Ожидание источника.** Пока источник недоступен, заявка остаётся в автоматической проверке, клиент видит «Ждём ответа бюро кредитных историй». Если ожидание дольше лимита (15 минут в демо), заявка уходит аналитику с флагом F05. После восстановления источника аналитик перезапускает проверку.
- **Прогоны.** Каждый запуск конвейера по заявке — отдельный прогон (`run_no`). Результаты шагов хранятся по прогонам, поэтому повторная проверка после ответа клиента не затирает историю.
- **Таймеры.** Раз в 10 секунд обработчик закрывает просроченные предложения и запросы информации и возвращает зависшие задания.

Обновление интерфейса без перезагрузки

Каждое изменение по заявке пишет событие в таблицу `events` с адресатом (клиент компании или сотрудники банка). Браузер держит соединение `GET /api/stream` (Server-Sent Events): API раз в секунду отдаёт новые события, при переподключении продолжает с `Last-Event-ID`. Получив событие, интерфейс перезапрашивает только затронутые данные. nginx проксирует поток без буферизации с таймаутом в час.

SSE выбраны вместо WebSocket: поток односторонний, работает через обычный HTTP-прокси и переподключается сам. Для боевой нагрузки опрос таблицы заменяется на `LISTEN/NOTIFY` или брокер сообщений без изменения контракта.

Безопасность

Мера	Реализация
Аутентификация	Логин и пароль, пароли хранятся как bcrypt-хеши. JWT (HS256) живёт 12 часов, секрет задаётся переменной `JWT_SECRET`
Авторизация	Роли `CLIENT`, `UNDERWRITER`, `ADMIN` проверяются на каждом маршруте. Клиент видит только заявки своей компании, чужие отдают 404. Переходы статусов дополнительно проверяются по роли
Конкурентные действия	На время действия строка заявки блокируется (`SELECT … FOR UPDATE`), при записи проверяется версия заявки (оптимистическая блокировка). Если аналитик и клиент действуют одновременно, проходит первое действие, второй получает 409: действие недоступно в новом статусе или «Заявка изменилась»

Мера	Реализация
Повторные запросы	Отправка заявки и подписание идемпотентны по заголовку `Idempotency-Key`
Файлы	Белый список расширений, лимит 10 МБ (в nginx — 12 МБ). Файлы лежат вне веб-корня в каталоге заявки, к имени добавляется случайный префикс; выдача только через API с проверкой прав. Для каждого файла хранится SHA-256
Подписание	Повторный ввод пароля, в журнал пишутся время, пользователь и SHA-256 договора
Журнал	Все действия пользователей и системы — в `audit_log` с идентификатором запроса
Веб	В стенде интерфейс и API отдаются с одного адреса через nginx; CORS разрешён только для локального дев-сервера Vite. Заголовки `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`
Контейнеры	Сервисы на Python работают от непривилегированного пользователя, наружу открыт только порт веб-интерфейса
Данные	Только синтетические. Секреты (`JWT_SECRET`, ключ LLM) передаются через `.env`, который не попадает в репозиторий. В модель уходит только то, что нужно для задачи (см. ai.md)

Надёжность и сопровождение

- **Запуск по готовности.** У `postgres`, `simulator` и `api` есть проверки здоровья. `api` стартует после базы и симулятора, применяет миграции и загружает справочники, остальные сервисы ждут готовности `api`. Все сервисы перезапускаются при сбое.
- **Пробы.** `GET /api/health` — процесс жив, `GET /api/ready` — доступна база.
- **Логи.** JSON-логи с идентификатором запроса, временем ответа и ключевыми полями задания: удобно собирать в ELK или Loki.
- **Миграции.** Схема базы ведётся в Alembic и применяется при старте.
- **Воспроизводимость.** Версии зафиксированы: `uv.lock` (Python), `package-lock.json` (npm), точные теги базовых образов, шрифты лежат в репозитории.
- **Проверки в CI.** Линтеры, юнит-тесты, сборка образов, сценарии через API и в браузере на поднятом стеке (.github/workflows/ci.yml).

Масштабирование и путь к боевому контуру

Сейчас (демо)	В боевом контуре
Один `api`, один `worker`	Несколько копий за балансировщиком; обработчики масштабируются без изменений кода (`docker compose up --scale worker=3` работает уже сейчас)
PostgreSQL в контейнере	Управляемый кластер с репликами и резервным копированием
Очередь заданий в PostgreSQL	Можно оставить (тот же паттерн outbox) или вынести в Kafka/RabbitMQ при росте нагрузки
Документы в томе Docker	Объектное хранилище S3
Симулятор внешних систем	Адаптеры к ЕГРЮЛ/ЕГРИП, ФНС, ГИР БО, БКИ и АБС через интеграционную шину; контракты клиента `integrations/` сохраняются
Логин и пароль	Корпоративный IdP (Keycloak, OIDC), для клиентов — вход Альфа-Бизнес или ЕСИА
Простая электронная подпись паролем	УКЭП или подпись в мобильном приложении банка

Сейчас (демо)	В боевом контуре
Секреты в `.env`	Хранилище секретов (Vault)
JSON-логи	Сбор логов и метрик, трассировка, алерты на ошибки конвейера и превышение SLA

Почему так

Решение	Альтернатива	Почему выбрали
Модульный монолит + отдельный обработчик	Микросервисы	Один репозиторий, одна схема данных и одна транзакция на смену статуса — меньше точек отказа при той же изоляции модулей. Обработчик масштабируется отдельно
Очередь в PostgreSQL	Kafka, RabbitMQ	Атомарность «статус + задание» без распределённых транзакций, на один сервис меньше в Docker. Паттерн переносится на брокер
Балльная скоринговая карта и явные правила	ML-модель	Прозрачно для аналитика и клиента, объясняется по факторам, не требует обучающих данных. Организаторы допускают упрощённую логику
Симулятор как отдельный сервис	Заглушки внутри кода	Сетевые вызовы, таймауты и сбои настоящие, поэтому повторы и circuit breaker проверяются в работе
SSE	WebSocket, опрос из браузера	Достаточно потока «сервер → клиент», работает через прокси, переподключается сам
Дизайн-система Альфа-Банка	Свои компоненты	Привычный клиентам банка интерфейс и доступность из коробки; фирменные шрифты заменены свободным Onest (см. design.md)
ИИ только как помощник	ИИ в решении	Решение должно быть объяснимым и воспроизводимым; модель заполняет форму и пишет сводку для аналитика, основной сценарий от неё не зависит

Документ 5 из 14 · docs/process.md

Кредитный процесс

Модель процесса MVP: участники, этапы, статусы заявки, правила переходов, обработка сбоев и принятые упрощения. Логика скоринга и решения описана в credit-policy.md.

Границы процесса

- **Начало:** у предпринимателя появилась потребность в финансировании, и он открыл кабинет.
- **Конец:** кредит выдан (деньги перечислены, график платежей сформирован) или заявка завершена отказом, отзывом либо истечением срока.
- **Вне MVP:** обслуживание и погашение кредита, оценка залога, реальные электронная подпись, ЕСИА и платежи. Подробнее — в разделе «Упрощения».

Участники

Роль	Кто	Что делает
Клиент	Собственник, директор, бухгалтер или другой уполномоченный сотрудник ООО либо ИП	Подбирает продукт, заполняет и отправляет заявку, отвечает на запросы банка, принимает предложение, подписывает документы
Конвейер	Автоматическая обработка	Проверяет данные, запрашивает внешние источники, считает скоринг, применяет кредитную политику, формирует документы, проводит выдачу
Кредитный аналитик	Сотрудник банка	Рассматривает заявки, которые не решены автоматически: изучает досье, запрашивает информацию, принимает решение в пределах полномочий
Администратор	Сотрудник банка	Сбрасывает демо-данные, управляет сбоями симулятора; в развитии — каталог продуктов и параметры политики

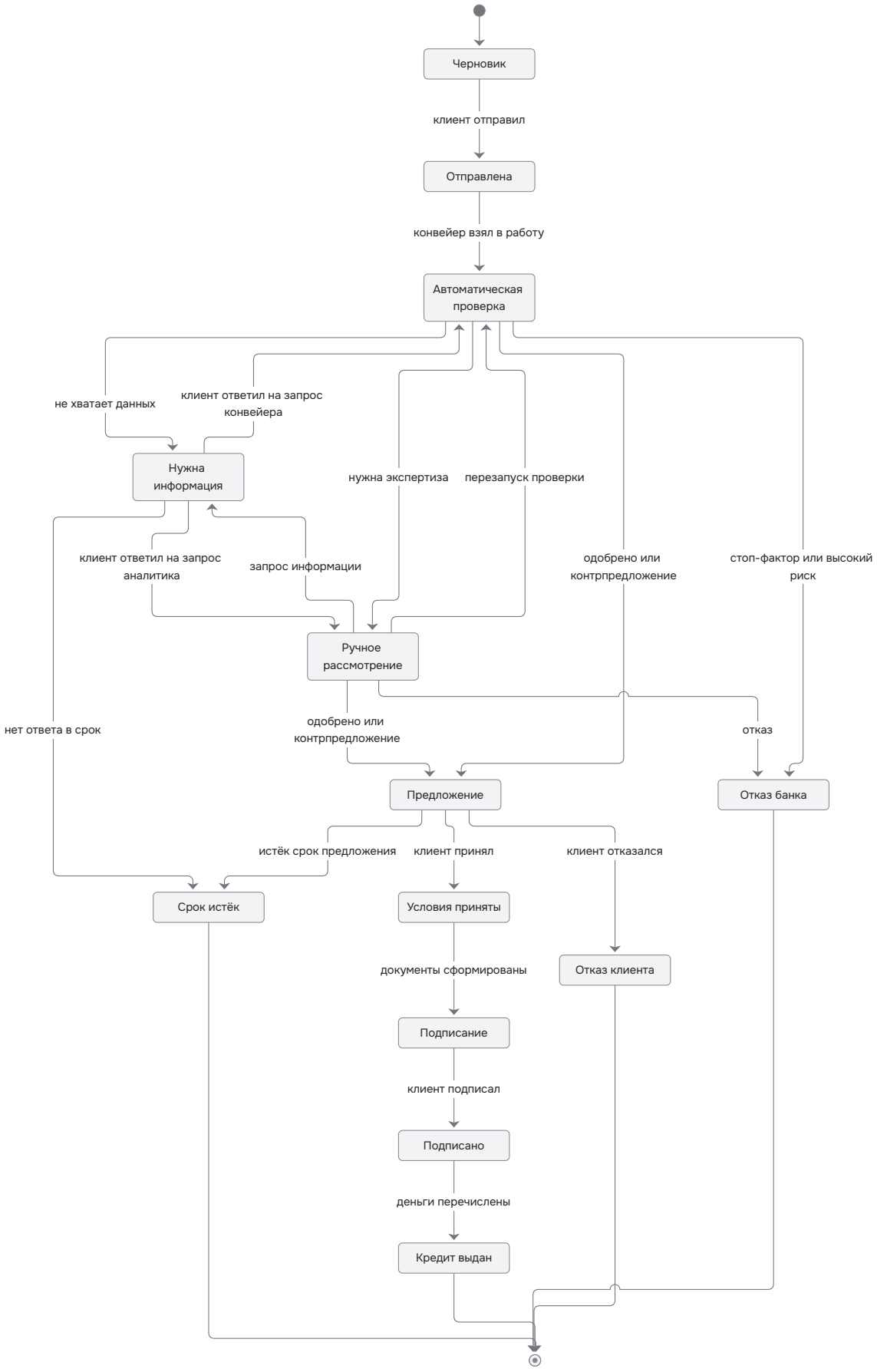
В MVP у каждой тестовой компании один пользователь с правом подписи. Разделение ролей внутри компании (готовит бухгалтер, подписывает директор) — в развитии.

Этапы

№	Этап	Исполнитель	Результат
1	Подбор продукта	Клиент	Выбран продукт, есть предварительный расчёт платежа
2	Заполнение заявки	Клиент	Черновик: компания (по ИНН), параметры, финансы и документы, согласия
3	Отправка	Клиент	Заявка принята в обработку
4	Проверка данных	Конвейер	Данные полные, корректные и приведены к единому виду, либо клиенту отправлен запрос информации

№	Этап	Исполнитель	Результат
5	Сбор сведений	Конвейер	Получены сведения из ЕГРЮЛ/ЕГРИП, ФНС, отчётности, БКИ и внутренних данных банка
6	Скоринг	Конвейер	Балл, класс риска и вклад каждого фактора
7	Кредитное решение	Конвейер	Автоматическое решение или передача аналитику
8	Ручное рассмотрение	Аналитик	Одобрение, контрпредложение, запрос информации или отказ
9	Предложение	Клиент	Условия приняты или отклонены
10	Документы и подписание	Конвейер, клиент	Договор и график платежей сформированы и подписаны
11	Выдача	Конвейер	Кредит выдан, график платежей в кабинете

Статусы заявки



Отдельно от схемы: до статуса «Подписано» клиент может в любой момент отозвать заявку. Тогда она переходит в статус «Отозвана» (`CANCELLED`). Черновик не отзывают, а удаляют.

Код	Для клиента	Для банка	Что дальше
`DRAFT`	Черновик	—	Заполните и отправьте заявку
`SUBMITTED`	Заявка отправлена	Новая	Начинаем проверку
`IN_PROCESSING`	Проверяем заявку	Автоматическая обработка	Обычно это занимает несколько минут
`NEED_INFO`	Нужна информация	Ожидает клиента	Ответьте на запрос банка
`MANUAL_REVIEW`	Заявку рассматривает специалист	В очереди или у аналитика	Специалист изучает заявку
`OFFER`	Предложение готово	Предложение выдано	Примите условия или откажитесь
`ACCEPTED`	Готовим документы	Условия приняты	Документы появятся в кабинете
`SIGNING`	Подпишите документы	Ожидает подписи	Подпишите договор в кабинете
`SIGNED`	Документы подписаны	Подписано	Перечисляем деньги
`ISSUED`	Кредит выдан	Выдан	Смотрите график платежей
`REJECTED`	Банк отказал	Отказ	Причина и что можно сделать
`DECLINED`	Вы отказались от предложения	Отказ клиента	Можно подать новую заявку
`CANCELLED`	Заявка отозвана	Отозвана	Можно подать новую заявку
`EXPIRED`	Срок истёк	Истёк срок	Можно подать новую заявку

Правила переходов

Из	В	Кто	Условие	Что происходит
`DRAFT`	`SUBMITTED`	Клиент	Обязательные поля заполнены, даны согласия на запрос в БКИ и обработку персональных данных	Фиксируется версия данных заявки, ставится задание конвейеру
`SUBMITTED`	`IN_PROCESSING`	Конвейер	Задание взято в работу	Запускаются шаги конвейера
`IN_PROCESSING`	`NEED_INFO`	Конвейер	Не хватает данных или документов	Клиенту приходит запрос с перечнем и сроком
`IN_PROCESSING`	`MANUAL_REVIEW`	Конвейер	Флаги политики, класс риска C/D, сумма выше автолимита, недоступный источник	Заявка попадает в очередь аналитиков
`IN_PROCESSING`	`OFFER`	Конвейер	Класс A/B без флагов	Формируется предложение: как запрошено или контрпредложение
`IN_PROCESSING`	`REJECTED`	Конвейер	Стоп-фактор или класс риска E	Клиент видит причину простыми словами
`NEED_INFO`	`IN_PROCESSING`	Клиент	Ответ на запрос конвейера	Конвейер повторяет проверку с новыми данными
`NEED_INFO`	`MANUAL_REVIEW`	Клиент	Ответ на запрос аналитика	Заявка возвращается тому же аналитику
`NEED_INFO`	`EXPIRED`	Таймер	Срок ответа истёк	Заявка закрывается
`MANUAL_REVIEW`	`OFFER`	Аналитик	Решение в пределах полномочий	Формируется предложение

Из	В	Кто	Условие	Что происходит
`MANUAL_REVIEW`	`NEED_INFO`	Аналитик	Нужны документы или пояснения	Клиенту приходит запрос
`MANUAL_REVIEW`	`REJECTED`	Аналитик	Указаны код причины и комментарий	Клиент видит причину
`MANUAL_REVIEW`	`IN_PROCESSING`	Аналитик	Заявка ушла к аналитику из-за недоступного источника, источник восстановился	Конвейер заново запрашивает источники и пересчитывает решение (новый прогон с увеличенным номером)
`OFFER`	`ACCEPTED`	Клиент	Выбран вариант, предложение действует	Запускается формирование документов
`OFFER`	`DECLINED`	Клиент	—	Заявка закрывается
`OFFER`	`EXPIRED`	Таймер	Срок предложения истёк	Заявка закрывается
`ACCEPTED`	`SIGNING`	Конвейер	Договор и график сформированы	Документы появляются в кабинете
`SIGNING`	`SIGNED`	Клиент	Подтверждение подписи паролем	Банк подписывает автоматически, ставится задание на выдачу
`SIGNED`	`ISSUED`	Конвейер	Симулятор подтвердил перечисление	Создаются кредит и график платежей
Любой активный до `SIGNED`	`CANCELLED`	Клиент	—	Заявка закрывается, текущие задания снимаются

Общие правила:

- **Единая точка смены статуса.** Все переходы проходят через один сервис, недопустимый переход отклоняется. Смена статуса, запись в историю, журнал действий и событие для обработчика сохраняются в одной транзакции.
- **Версия заявки (оптимистическая блокировка).** Если аналитик принимает решение, а клиент в это же время отзывает заявку, пройдёт только первое действие. Второй участник увидит, что заявка изменилась.
- **Идемпотентность.** Отправка и подписание идемпотентны (`Idempotency-Key`): повторное нажатие или повтор запроса не создают дубликатов.

Шаги конвейера

Пока идёт статус «Автоматическая проверка», конвейер последовательно выполняет шаги. Клиент видит их прогресс, банк — подробности по каждому шагу.

Шаг	Источник	Что получаем или проверяем	Если не получилось
Проверка данных	Заявка	Обязательные поля, контрольные суммы ИНН и ОГРН, сумма и срок в пределах продукта, согласия, обязательные документы	«Нужна информация» с перечнем недостающего
ЕГРЮЛ / ЕГРИП	Симулятор	Существование и статус компании, дата регистрации, ОКВЭД, руководитель и дата назначения, адрес	Повторы, затем ожидание источника
ФНС «Прозрачный бизнес»	Симулятор	Налоговая задолженность, массовый адрес, дисквалификация руководителя, численность, налоговый режим	То же

Шаг	Источник	Что получаем или проверяем	Если не получилось
Отчётность (ГИР БО)	Симулятор	Выручка, прибыль от продаж и чистая прибыль за два последних года. Только для ООО	Для ИП шаг пропускается, финансы берутся из оборотов по счёту или выписки
БКИ	Симулятор	Действующие кредиты и платежи по ним, просрочки за 24 месяца, текущая просрочка	Повторы, затем ожидание источника
Внутренние данные банка	Симулятор	Поступления на расчётный счёт за 12 месяцев, срок обслуживания	Не клиент банка — шаг пропускается
Скоринг	Модуль `scoring`	Балл, класс риска, вклад факторов	Ручное рассмотрение
Кредитная политика	Модуль `policy`	Стоп-факторы, флаги, лимит, решение	Ручное рассмотрение

Обработка сбоев внешних источников:

- Каждый вызов ограничен таймаутом.
- При ошибке шаг повторяется с растущей паузой.
- Если источник падает массово, вызовы к нему на время приостанавливаются (circuit breaker), чтобы не перегружать его.
- Пока источник недоступен, заявка остаётся в статусе «Автоматическая проверка». Клиент видит «Ждём ответа от бюро кредитных историй». Банк видит заявку в списке сбоев конвейера и может повторить шаг вручную.
- Когда источник восстанавливается, обработка продолжается с того же шага.
- Если ожидание превысило лимит, заявка уходит на ручное рассмотрение с пометкой «источник недоступен» (флаг F05). Когда источник восстановится, аналитик перезапускает автоматическую проверку из досье, и конвейер проходит шаги заново.

Таймеры

Значения задаются в настройках.

Таймер	Значение в демо	Ориентир для боевой работы	Что происходит
Ответ на запрос информации	3 дня	5 рабочих дней	Статус «Срок истёк»
Срок действия предложения	7 дней	30 дней	Статус «Срок истёк»
Срок ручного рассмотрения	1 день	1 рабочий день	Подсветка в очереди
Ожидание внешнего источника	15 минут	24 часа	Ручное рассмотрение с пометкой

Уведомления

Все уведомления приходят только в продукте: лента событий, счётчик и обновление страницы без перезагрузки. Email, SMS, мессенджеры и звонки в сценарии не используются.

Событие	Кому
Заявка отправлена	Клиенту

Событие	Кому
Запрос информации	Клиенту
Ответ клиента на запрос	Аналитику
Предложение готово / отказ	Клиенту
Документы готовы к подписанию	Клиенту
Кредит выдан	Клиенту
Новое сообщение в чате	Клиенту или аналитику
Новая заявка на ручное рассмотрение	Аналитикам

Упрощения

Допущения сохраняют смысл и связность процесса (ТЗ, стр. 11).

- **Подписание.** Простая электронная подпись смоделирована в продукте: клиент подтверждает подписание повторным вводом пароля. Сохраняются хэш документа, время и пользователь. Реальной ЭЦП и SMS-кодов нет.
- **Выдача.** Симулятор банковской системы «открывает» ссудный счёт и «перечисляет» деньги без реальных операций.
- **Внешние источники.** ЕГРЮЛ/ЕГРИП, ФНС, ГИР БО, БКИ и внутренние данные банка заменены симулятором с синтетическими данными. Контракты упрощены, смысл интеграций сохранён.
- **Залог.** Клиент заявляет тип и оценочную стоимость, оценка и проверка залога не моделируются.
- **Пользователи.** Один пользователь на компанию, вход по логину и паролю вместо ЕСИА.
- **Расчёты.** Только рубли и аннуитетные платежи. Для овердрафта и кредитной линии расчёт упрощён до лимита и ставки.
- **Скоринг.** Балльная карта на явных правилах. Организаторы допускают упрощённую логику решения.

Документ 6 из 14 · docs/credit-policy.md

Кредитная политика и скоринг

Как конвейер оценивает заявку и принимает решение. Скоринг даёт оценку риска заёмщика, а кредитная политика превращает её в решение с учётом продукта, суммы, срока, результатов проверок и правил банка. Поэтому при одинаковом риске разные продукты дают разные условия.

Все продукты, ставки и пороги синтетические и не являются офертой. Организаторы разрешили командам самим определять логику скоринга и допустили её упрощение.

Почему балльная карта

- **Объяснимость.** Каждый фактор даёт понятный вклад в баллах. Аналитик видит расшифровку, клиент — главные причины простыми словами.
- **Воспроизводимость.** Решение хранит версию политики и входные признаки, поэтому его можно повторить и проверить задним числом.
- **Развитие.** Карту можно откалибровать на исторических данных или заменить статистической моделью, не меняя остальной конвейер.

Каталог продуктов

Код	Продукт	Сумма, ₽	Срок, мес	Базовая ставка	Залог	Срок ведения бизнеса	Особые условия	Автолимит, ₽
`EXPRESS`	Экспресс-кредит без залога	300 тыс – 5 млн	6–36	21%	нет	от 9 мес	—	5 млн
`GROWTH`	Кредит на развитие бизнеса	1–30 млн	12–60	19%	нет	от 12 мес	—	10 млн
`SECURED`	Кредит под залог	3–150 млн	12–120	15,5%	обязателен	от 12 мес	Сумма не больше 80% заявленной стоимости залога	— (всегда аналитик)
`OVERDRAFT`	Овердрафт	100 тыс – 20 млн	12	13%	нет	от 6 мес	Только клиентам банка с поступлениями на счёт от 6 мес; лимит не больше 40% среднемесячных поступлений	5 млн
`CREDIT_LINE`	Возобновляемая кредитная линия	1–50 млн	12–24	14,5%	нет	от 12 мес	Транши на 1–6 мес	10 млн
`STARTUP`	Кредит для начинающего бизнеса	100 тыс – 3 млн	6–36	23%	нет	от 3 до 12 мес	—	1 млн

Все продукты доступны ООО и ИП. Рефинансирование, автокредит и кредитная карта — в развитии.

Автолимит — максимальная сумма, которую конвейер может одобрить без аналитика.

Подбор продукта по задаче

Задача клиента	Продукты в порядке показа
Пополнить оборотные средства, закупить товар	`GROWTH`, `EXPRESS`, `CREDIT_LINE`
Закрыть кассовый разрыв, рассчитаться с поставщиками	`OVERDRAFT`, `CREDIT_LINE`, `EXPRESS`
Купить оборудование или транспорт	`SECURED`, `GROWTH`
Расширить бизнес, открыть новую точку	`GROWTH`, `SECURED`

Компаниям младше 12 месяцев при любой задаче первым показывается `STARTUP`.

Как подбор отбирает продукты:

- Показываются только продукты, условия которых компания выполняет: срок ведения бизнеса, наличие счёта в банке для овердрафта, сумма и срок в пределах продукта.
- У остальных продуктов указана причина, например «Доступен бизнесу от 1 года».
- Форма бизнеса и срок его ведения берутся из ЕГРЮЛ/ЕГРИП по ИНН, клиент их не вводит.

Признаки для оценки

Признак	Как считается	Источник
Срок ведения бизнеса	Месяцы с даты регистрации	ЕГРЮЛ / ЕГРИП
Годовая выручка	ООО — отчётность за последний год. Если её нет или клиент ИП — поступления на счёт за 12 месяцев (внутренние данные или загруженная выписка), за неполный период — в пересчёте на год	ГИР БО, внутренние данные, выписка
Операционная прибыль в месяц	ООО — прибыль от продаж / 12. Без отчётности — выручка × отраслевая маржа / 12	ГИР БО или расчёт
Текущие платежи по кредитам в месяц	Сумма платежей по действующим кредитам	БКИ
Текущая долговая нагрузка (DSCR)	Операционная прибыль в месяц / текущие платежи; без кредитов — максимум	Расчёт
Динамика выручки	Выручка за последний год / за предыдущий - 1	ГИР БО
Рентабельность продаж	Прибыль от продаж / выручка	ГИР БО
Кредитная история	Просрочки за 24 месяца, текущая просрочка	БКИ
Поступления на счёт	Число месяцев с поступлениями за последние 12 и их стабильность (коэффициент вариации)	Внутренние данные
Налоговая задолженность	Сумма	ФНС
Отраслевой риск	По ОКВЭД	ЕГРЮЛ / ЕГРИП
Доля суммы в выручке	Запрошенная сумма / годовая выручка	Заявка + расчёт

Отраслевая маржа (если нет отчётности): торговля 8%, производство 12%, транспорт 12%, строительство 10%, общепит 15%, услуги 20%.

Отраслевой риск:

- Низкий: ИТ (ОКВЭД 62–63), профессиональные услуги (69–70), медицина (86).
- Высокий: строительство (41–43), гостиницы и общепит (55–56), операции с недвижимостью (68).
- Остальные отрасли — средний.

Источник финансовых данных выбирается по приоритету:

1. отчётность за полный год;
2. поступления на счёт в банке, если клиент обслуживается от 3 месяцев;
3. выписка из другого банка, загруженная клиентом.

Если нет ни одного источника, клиенту отправляется запрос информации.

Скоринговая карта

Стартовый балл — 600. К нему прибавляются баллы по каждому фактору.

Фактор	Значение → баллы
Срок ведения бизнеса	меньше 12 мес: −60 · 1–3 года: 0 · 3–5 лет: +30 · больше 5 лет: +50
Кредитная история	без просрочек: +60 · нет истории: 0 · просрочки до 30 дней: −40 · просрочки 30+ дней, сейчас погашены: −120
Текущая долговая нагрузка (DSCR)	от 2,0 или нет кредитов: +60 · 1,5–2,0: +30 · 1,3–1,5: 0 · 1,0–1,3: −60 · меньше 1,0: −150
Динамика выручки	рост больше 10%: +30 · от −10% до +10%: +10 · падение на 10–30%: −40 · падение больше 30%: −100 · нет данных за два года: 0
Рентабельность продаж	больше 15%: +30 · 5–15%: +10 · 0–5%: −10 · убыток: −80 · нет данных: 0
Поступления на счёт в банке	все 12 мес и стабильно: +40 · 6–11 мес: +20 · нерегулярно: −20 · меньше 6 мес или не клиент: 0
Налоговая задолженность	нет: 0 · до 100 тыс ₽: −30 · больше 100 тыс ₽: −100
Отраслевой риск	низкий: +10 · средний: 0 · высокий: −40
Доля суммы в выручке	до 25%: +20 · 25–50%: 0 · больше 50%: −60

Классы риска:

Класс	Балл	Надбавка к ставке	Условная вероятность дефолта
A	от 760	0 п.п.	~1%
B	680–759	+1,5 п.п.	~3%
C	600–679	+3 п.п.	~6%
D	520–599	+5 п.п.	~12%
E	меньше 520	не выдаётся	больше 20%

Вероятность дефолта условная и нужна только для пояснений.

Стоп-факторы

При стоп-факторе — автоматический отказ независимо от балла.

Код	Условие	Источник	Что видит клиент
`S01`	Компания ликвидируется, реорганизуется или проходит банкротство	ЕГРЮЛ	Мы не выдаём кредиты компаниям в процессе ликвидации, реорганизации или банкротства
`S02`	Руководитель дисквалифицирован	ФНС	По данным ФНС руководитель компании дисквалифицирован
`S03`	Текущая просрочка больше 30 дней	БКИ	У компании есть просроченный долг по кредитам. Погасите его и подайте заявку снова
`S04`	Бизнес младше 3 месяцев	ЕГРЮЛ / ЕГРИП	Кредиты доступны бизнесу от 3 месяцев с даты регистрации
`S05`	Класс риска E	Скоринг	Сейчас мы не можем предложить кредит. Ниже — что можно улучшить

Флаги для ручного рассмотрения

Код	Условие
`F01`	Адрес массовой регистрации
`F02`	Есть налоговая задолженность
`F03`	Руководитель сменился меньше 6 месяцев назад
`F04`	Выручка, указанная в заявке, отличается от данных источников больше чем на 30%
`F05`	Внешний источник недоступен после всех повторов
`F06`	Текущая просрочка до 30 дней
`F07`	Класс риска C или D

Расчёт лимита

Лимит — минимум из применимых ограничений:

1. **Продукт:** максимальная сумма продукта.
2. **Выручка:** без залога — не больше 50% годовой выручки. Под залог — не больше 100% выручки и 80% стоимости залога.
3. **Платёжеспособность:** с учётом нового платежа DSCR должен быть не ниже 1,3.
   - Максимальный платёж = операционная прибыль в месяц / 1,3 - текущие платежи.
   - Сумма считается из максимального платежа по ставке и сроку.
4. **Овердрафт:** не больше 40% среднемесячных поступлений на счёт. Ограничения 2 и 3 к овердрафту не применяются.

Лимит округляется вниз до 10 тыс ₽.

Ставка = базовая ставка продукта + надбавка за класс риска.

Платёж (аннуитет): `P = S × i / (1 - (1 + i)^(-n))`, где `i` — годовая ставка / 12, `n` — срок в месяцах.

Переплата = `P × n - S`.

Матрица решения

Правила проверяются по порядку, срабатывает первое подходящее.

№	Условие	Решение
1	Есть стоп-фактор	Отказ
2	Нет источника финансовых данных	Нужна информация: выписка по счёту или отчётность
3	Лимит меньше минимальной суммы продукта	Отказ (`R02`). Если другой подходящий продукт проходит, клиенту предлагается он
4	Есть флаги `F01` – `F07` или продукт всегда требует рассмотрения	Ручное рассмотрение
5	Класс A/B, сумма к выдаче = минимум из запрошенной суммы и лимита, она не больше автолимита	Одобрено как запрошено или контрпредложение
6	Иначе (сумма к выдаче больше автолимита)	Ручное рассмотрение

Контрпредложение содержит до двух вариантов, клиент выбирает один:

- **та же сумма на более длинный срок**, если на максимальном сроке продукта она вписывается в лимит;
- **сумма, равная лимиту, на запрошенный срок**, если она не меньше минимальной суммы продукта.

Если по той же задаче подходит другой продукт с большим лимитом (например, под залог), клиенту показывается подсказка.

Полномочия аналитика

- **Решения:** одобрить как запрошено, сделать контрпредложение (сумма, срок, продукт), запросить информацию, отказать.
- **Сумма:** не больше расчётного лимита. Превышение до 20% — только с обязательным комментарием.
- **Ставка и срок:** ставка не ниже расчётной, срок в пределах продукта.
- **Комментарий** обязателен при отказе и при превышении лимита.
- **Сумма больше 30 млн ₽:** решение подтверждает второй сотрудник, принцип «четырёх глаз» (Could).

Коды причин отказа:

Код	Причина	Что видит клиент
`R01`	Высокий кредитный риск	Сейчас мы не можем предложить кредит
`R02`	Недостаточная платёжеспособность	Текущая выручка и долговая нагрузка не позволяют выдать кредит на эту сумму
`R03`	Негативная кредитная история	Отказ связан с вашей кредитной историей
`R04`	Недостоверные данные	Данные в заявке не подтвердились
`R05`	Не предоставлены документы	Не получили документы, необходимые для решения
`R06`	Не соответствует политике банка	Заявка не соответствует условиям кредитования

Код	Причина	Что видит клиент
`R07`	Иное	Текст из комментария аналитика

Объяснение решения клиенту

- **Одобрение и контрпредложение.** Блок «Что повлияло на условия»: до трёх факторов с наибольшим положительным вкладом и до двух ограничивающих.
- **Отказ.** Причина по коду и что можно сделать.
- **Что не показываем.** Баллы и класс риска клиенту не показываются, их видит только аналитик.

Фактор	Положительная формулировка	Ограничивающая формулировка
Срок ведения бизнеса	Бизнес работает больше 5 лет	Бизнесу меньше года
Кредитная история	Кредиты погашаются без просрочек	Были просрочки по кредитам
Долговая нагрузка	Невысокая текущая долговая нагрузка	Текущие платежи по кредитам ограничивают сумму
Динамика выручки	Выручка растёт	Выручка снизилась
Рентабельность	Бизнес прибыльный	Бизнес работает в убыток
Поступления на счёт	Стабильные поступления на счёт в банке	Нерегулярные поступления на счёт
Налоговая задолженность	—	Есть налоговая задолженность
Отраслевой риск	—	Отрасль с повышенным риском
Доля суммы в выручке	—	Сумма большая относительно выручки

Версионирование

- **Параметры в конфигурации.** Параметры продуктов, скоринга и политики хранятся в конфигурации с номерами версий (`scoring_version`, `policy_version`), а не в коде.
- **Решение хранит всё для проверки.** С каждым решением сохраняются версии, входные признаки и результат каждого правила. Поэтому решение можно воспроизвести и объяснить задним числом.

Документ 7 из 14 · docs/data-model.md

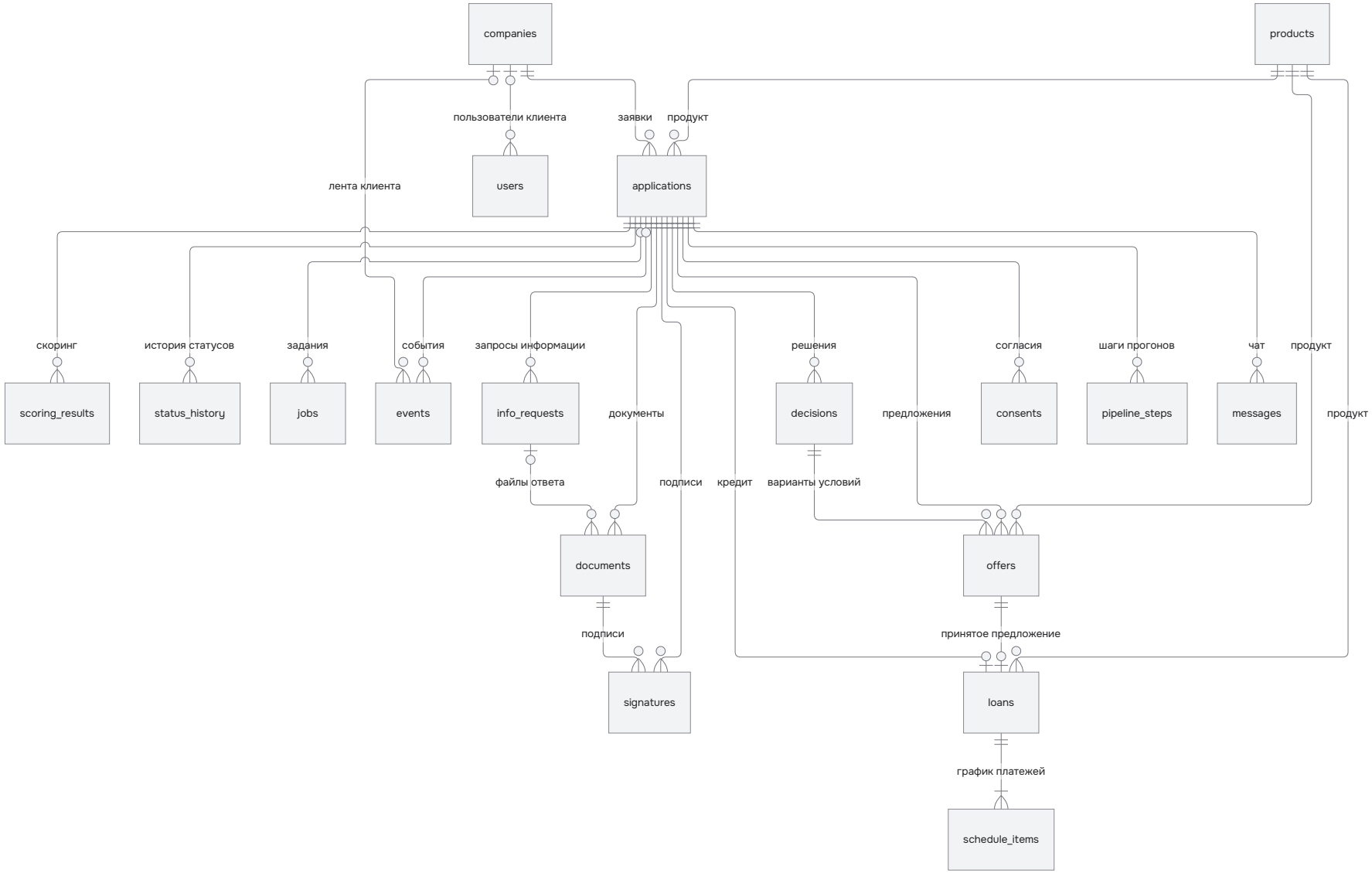
Модель данных

Таблицы PostgreSQL, связи между ними и соглашения о хранении. Источник истины — модели SQLAlchemy `backend/app/models.py` и миграция Alembic `0001_initial_schema.py`; допустимые значения статусов и кодов — `backend/app/domain/enums.py`. Процесс и статусы заявки описаны в process.md, устройство конвейера — в architecture.md.

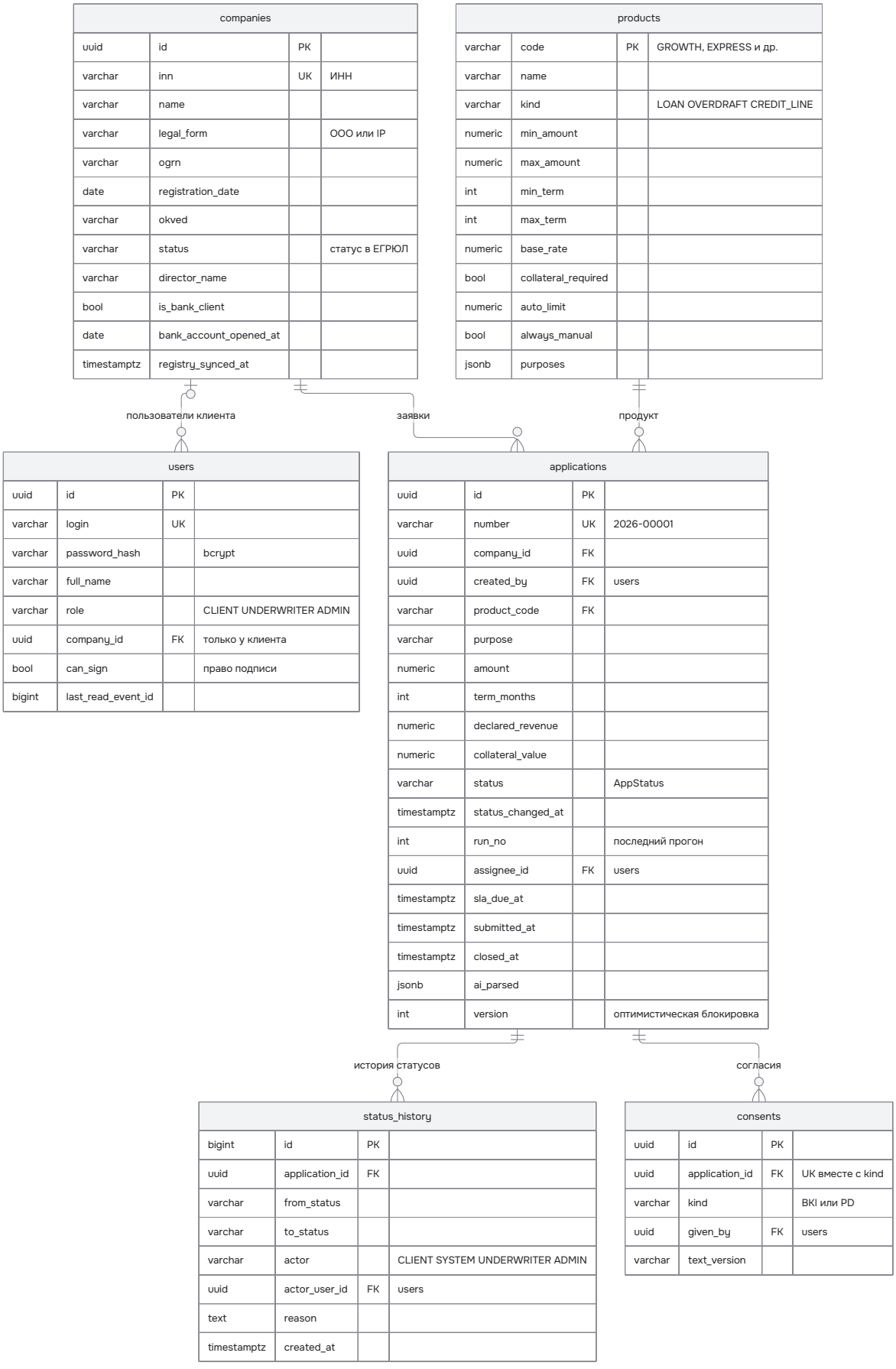
ER-диаграмма

Все 20 таблиц и связи между ними. Сначала общая схема без столбцов, затем те же таблицы по группам — с ключевыми столбцами; таблицы соседних групп показаны на схемах групп без столбцов. Ссылки на пользователя, который выполнил действие, — на отдельной схеме. Типы — типы PostgreSQL без размерностей: деньги — `NUMERIC(15,2)`, ставки — `NUMERIC(5,2)`, длины строк — в миграции. PK — первичный ключ, FK — внешний ключ, UK — уникальное поле.

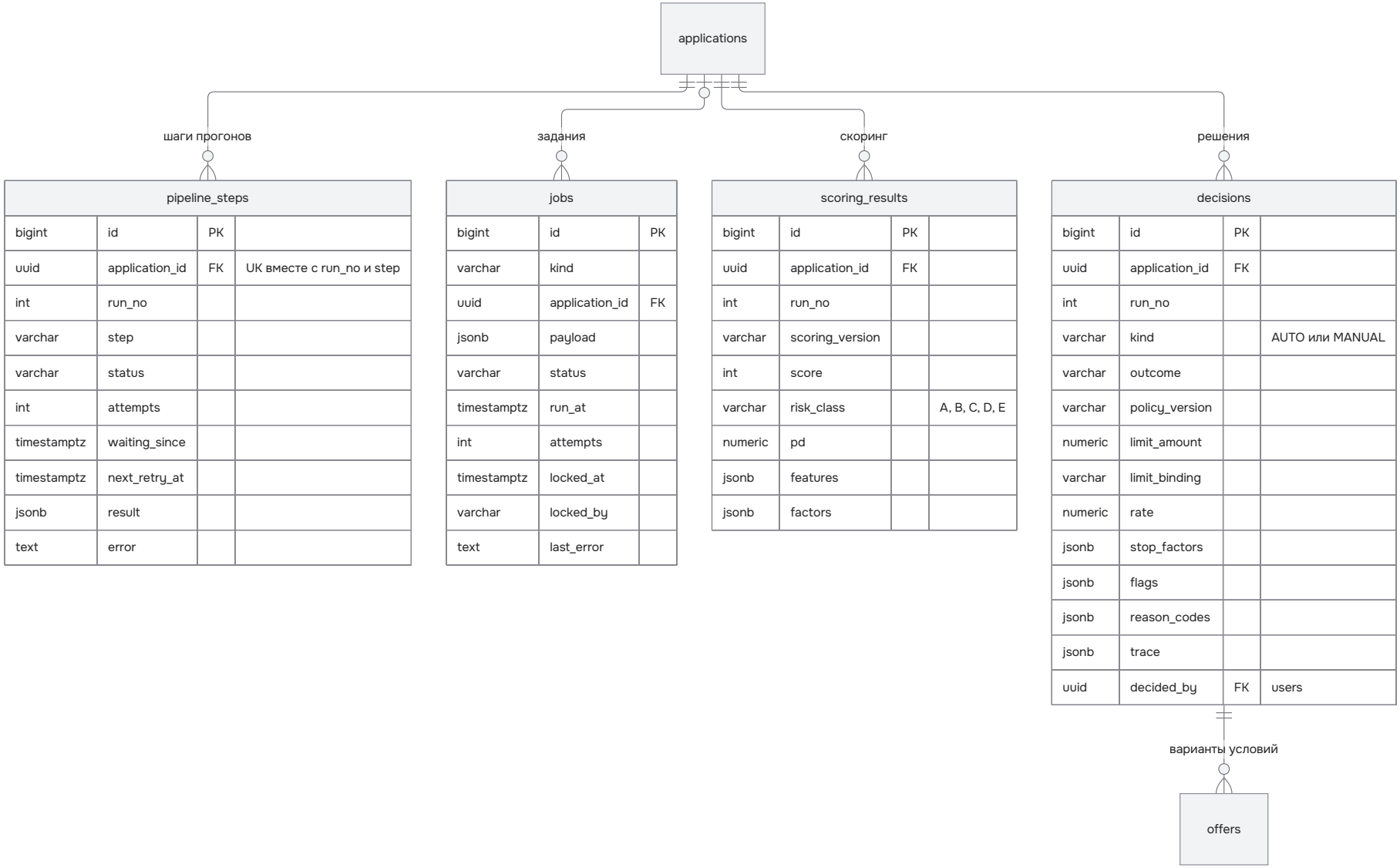
Общая схема


companies
products
пользователи клиента
заявки
продукт
лента клиента
users
applications
скоринг
история статусов
задания
события
запросы информации
решения
согласия
шаги прогонов
чат
продукт
scoring_results
status_history
jobs
events
info_requests
документы
decisions
предложения
consents
pipeline_steps
messages
продукт
файлы ответа
подписи
кредит
варианты условий
documents
offers
подписи
принятое предложение
signatures
loans
график платежей
schedule_items

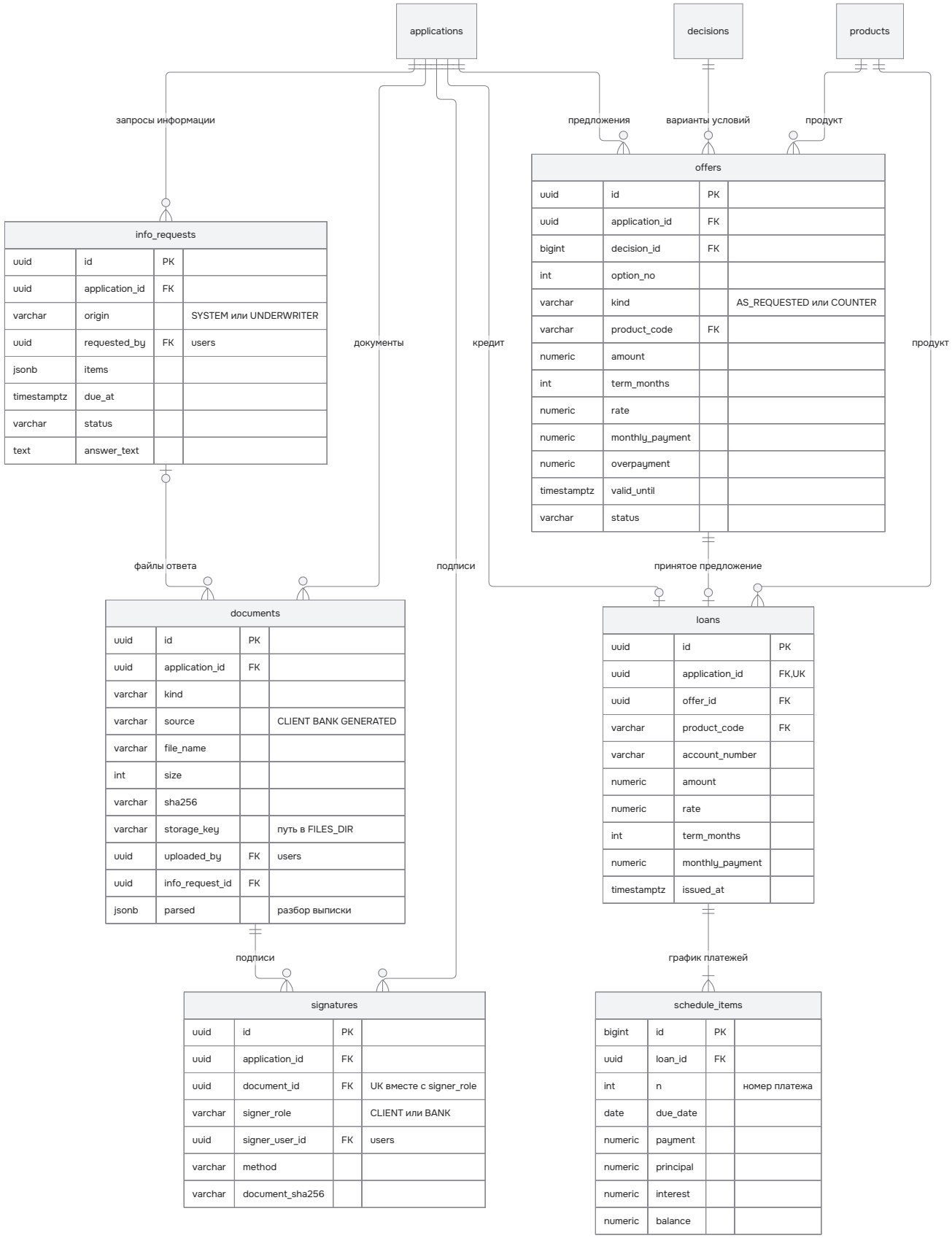
Компании, пользователи, продукты и заявка



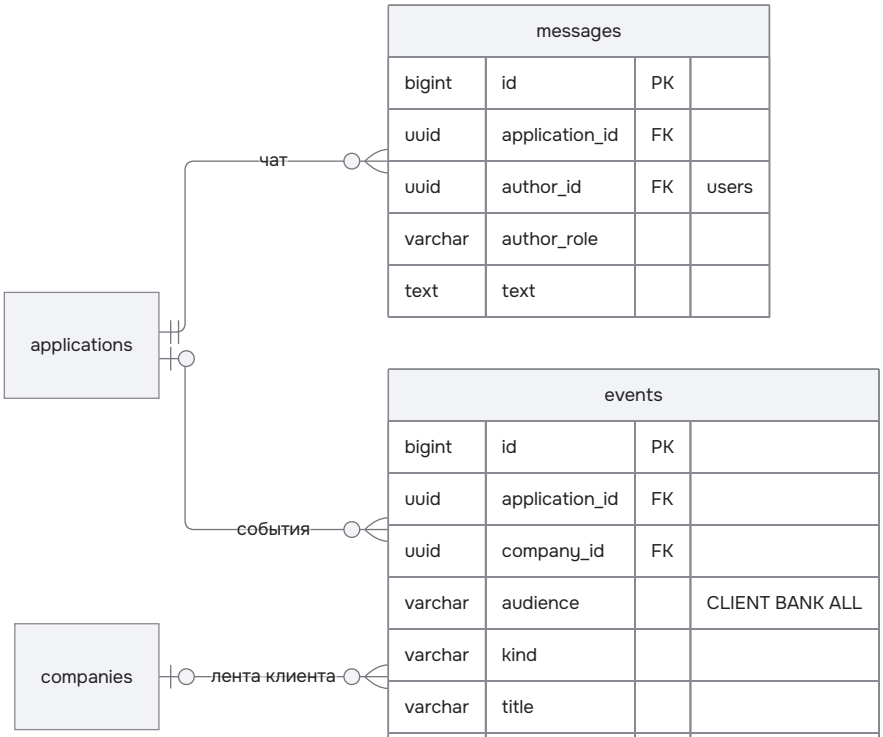
Конвейер и решение



Предложение, документы, выдача



Общение, уведомления и служебные таблицы



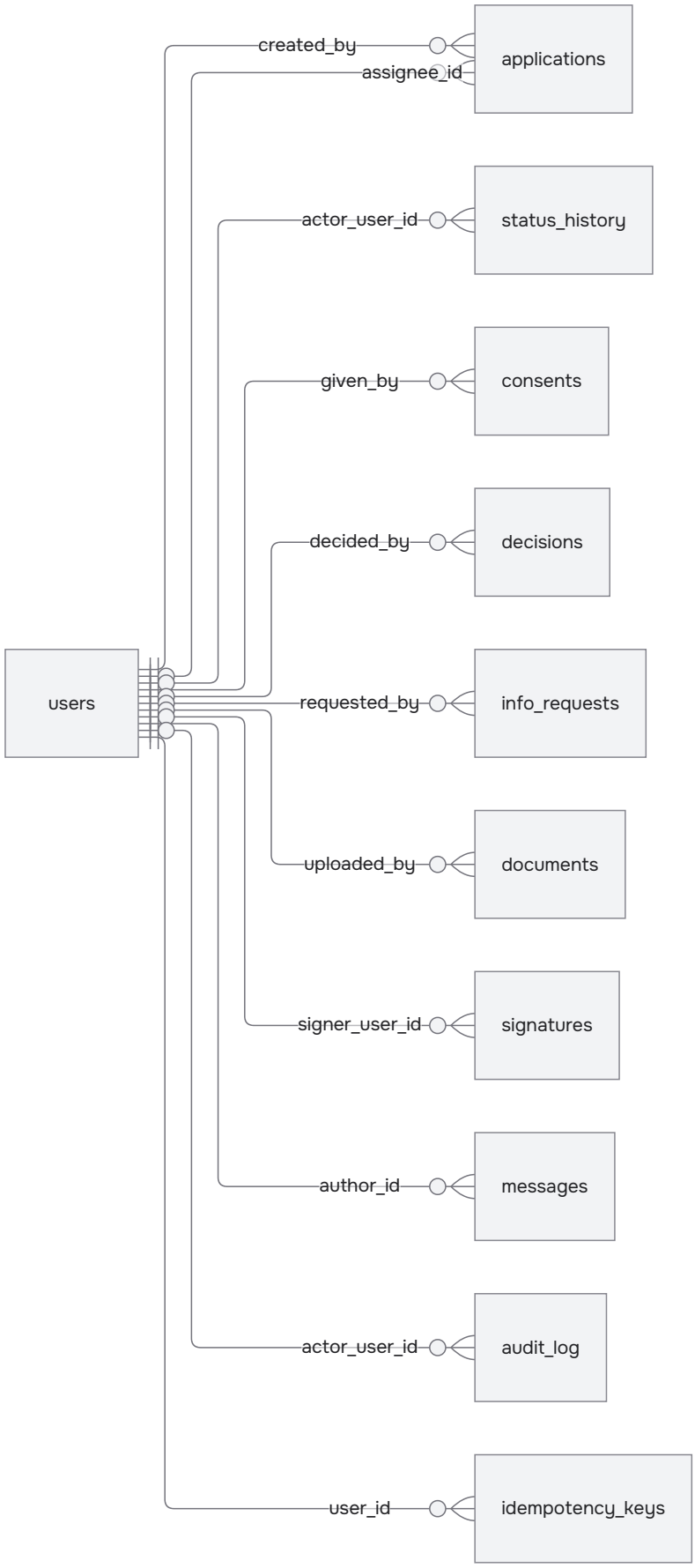
messages			
bigint	id	PK	
uuid	application_id	FK	
uuid	author_id	FK	users
varchar	author_role		
text	text		

events			
bigint	id	PK	
uuid	application_id	FK	
uuid	company_id	FK	
varchar	audience		CLIENT BANK ALL
varchar	kind		
varchar	title		
bool	notify		
jsonb	payload		

audit_log			
bigint	id	PK	
varchar	actor		
uuid	actor_user_id	FK	users
varchar	action		
varchar	entity_type		
varchar	entity_id		строка, без FK
jsonb	details		
varchar	request_id		

idempotency_keys			
bigint	id	PK	
uuid	user_id	FK	users, UK вместе с scope и key
varchar	scope		
varchar	key		
jsonb	response		

Кто выполнил действие — ссылки на `users`:



Общее для всех таблиц:

- Ключи: `uuid` — у компаний, пользователей, заявок, согласий, предложений, запросов информации, документов, подписей и кредитов; автоинкремент `bigint` — у истории статусов, шагов, заданий, результатов скоринга, решений, графика, сообщений, событий, журнала и ключей идемпотентности; у продукта ключ — код.
- Внешние ключи без каскадного удаления (`NO ACTION`): строку, на которую есть ссылки, удалить нельзя.
- Почти у всех таблиц есть `created_at timestamptz` со значением по умолчанию `now()`; его нет у `products`, `pipeline_steps` и `schedule_items`.
- Статусы, виды и коды хранятся строками (`varchar`), без перечислений PostgreSQL и ограничений CHECK. Допустимые значения задают перечисления в `enums.py`, подписи для клиента и банка — `domain/labels.py`.
- JSON хранится в `jsonb`.
- Служебная таблица `alembic_version` хранит номер версии схемы для миграций.

Таблицы

Кто пишет: «API» — сервис `api` по действиям пользователей, «worker» — обработчик конвейера, «seed» — загрузка демо-данных (`backend/app/seed`).

Компании, пользователи, продукты

`companies` — компании клиентов: профиль из ЕГРЮЛ/ЕГРИП и сведения о счёте в банке.

- Поля: `inn` (10 цифр у ООО, 12 у ИП), `legal_form` (`ООО`, `IP`), `ogrn`, `registration_date`, `okved` и `okved_name`, `address`, `status` (`ACTIVE`, `LIQUIDATING`, `REORGANIZING`, `BANKRUPTCY`, `LIQUIDATED`), `director_name`; `is_bank_client` и `bank_account_opened_at` — из внутренних данных банка; `registry_synced_at` — когда профиль последний раз обновлялся из реестра.
- Ограничения: UNIQUE `inn`.
- Пишет: seed создаёт компанию по ИНН и названию и подтягивает профиль из симулятора; API обновляет профиль, если он старше суток или клиент нажал «Обновить»; worker обновляет профиль на шагах ЕГРЮЛ и «Внутренние данные банка».

`users` — клиенты и сотрудники банка.

- Поля: `login`, `password_hash` (bcrypt), `full_name`, `position`, `role` (`CLIENT`, `UNDERWRITER`, `ADMIN`), `company_id` — компания клиента (у сотрудников пусто), `can_sign` — право подписывать договор от имени компании, `last_read_event_id` — последнее прочитанное уведомление.
- Ограничения: UNIQUE `login`; FK `company_id` → `companies`.
- Пишет: seed создаёт и обнуляет `last_read_event_id` при сбросе демо-данных; API меняет только `last_read_event_id`, когда пользователь открывает ленту уведомлений.

`products` — каталог кредитных продуктов (credit-policy.md).

- Поля: `code` (`GROWTH`, `EXPRESS`, `SECURED`, `OVERDRAFT`, `CREDIT_LINE`, `STARTUP`), `kind` (`LOAN`, `OVERDRAFT`, `CREDIT_LINE`), границы суммы и срока, `base_rate`, `collateral_required`, `min_business_months` и `max_business_months`, `requires_bank_account_months`, `auto_limit` — максимум для автоматического одобрения, `always_manual`, `purposes` — коды целей (JSON-массив), `sort_order`, `active`.

- Ограничения: PK `code`.
- Пишет: seed — из `backend/app/domain/catalog.py` при каждом запуске; API и worker только читают.

Заявка

`applications` — заявка: параметры, текущий статус и служебные поля процесса.

- Поля: `number` — номер для людей; `purpose` (`WORKING_CAPITAL`, `CASH_GAP`, `EQUIPMENT`, `EXPANSION`) и `purpose_text`; `amount`, `term_months`; `declared_revenue`; `collateral_type` и `collateral_value`; `contact_name`; `status`, `status_changed_at`, `submitted_at`, `closed_at`; `run_no` — номер последнего прогона конвейера; `assignee_id` и `sla_due_at` — аналитик и срок ручного рассмотрения; `ai_parsed` — что распознал разбор описания задачи клиента (правила или модель) и исходный текст; `version`.
- Ограничения и индексы: UNIQUE `number`; индекс по `status` для очереди; FK на `companies`, `products`, `users` (`created_by`, `assignee_id`).
- Пишет: API клиента создаёт, правит и удаляет черновик; API банка назначает аналитика (`assignee_id`); статус и связанные с ним поля меняет только машина состояний (см. ниже) — из API клиента и банка, worker и seed (демо-заявки).

`status_history` — каждый переход статуса: `from_status`, `to_status`, `actor` (`CLIENT`, `SYSTEM`, `UNDERWRITER`, `ADMIN`), `actor_user_id`, `reason`, `created_at`.

- Индекс по `application_id`.
- Пишет: только машина состояний. Строки не меняются и удаляются только при сбросе демо-данных.

`consents` — согласия на запрос в БКИ (`BKI`) и обработку персональных данных (`PD`): кто дал, когда, версия текста согласия (`text_version`).

- Ограничения: UNIQUE (`application_id`, `kind`).
- Пишет: API при отправке заявки, seed для демо-заявок. Идентификатор согласия `BKI` передаётся в запрос к БКИ.

Конвейер и решение

`pipeline_steps` — шаги прогона конвейера: `DATA_CHECK`, `EGRUL`, `FNS`, `GIRBO`, `BKI`, `BANK`, `SCORING`, `POLICY`.

- Поля: `run_no`, `step`, `status` (`PENDING`, `RUNNING`, `DONE`, `SKIPPED`, `WAITING`, `FAILED`), `attempts`, `started_at`, `finished_at`, `waiting_since` и `next_retry_at` — ожидание недоступного источника, `result` — нормализованный ответ источника или результат проверки, `error`.
- Ограничения и индексы: UNIQUE (`application_id`, `run_no`, `step`); индекс по `application_id`.
- Пишет: машина состояний создаёт строки нового прогона; worker выполняет шаги и записывает результат.

`jobs` — очередь заданий обработчику.

- Поля: `kind` (`PIPELINE_STEP`, `GENERATE_DOCS`, `DISBURSE`), `payload` (для шага — `run_no` и `step`), `status` (`QUEUED`, `RUNNING`, `DONE`, `FAILED`, `CANCELLED`), `run_at` — не раньше какого времени выполнять, `attempts`, `locked_at` и `locked_by` — кто и когда взял, `last_error`, `finished_at`.

- Индексы: `ix_jobs_ready` (`status`, `run_at`) — для выборки готовых заданий; по `application_id`.
- Пишет: машина состояний и worker ставят задания, worker их выполняет; API банка по кнопке «Повторить сейчас» переносит `run_at` ожидающих заданий на текущий момент.

`scoring_results` — результат скоринговой карты на прогоне: `scoring_version`, `score`, `risk_class` (A–E), `pd` — условная вероятность дефолта, %, `features` — входные признаки, `factors` — вклад каждого фактора в баллах с формулировкой для клиента.

- Индекс по `application_id`.
- Пишет: worker на шаге `SCORING`.

`decisions` — решения по заявке, автоматические и ручные.

- Поля: `run_no`; `kind` (`AUTO`, `MANUAL`); `outcome` (`APPROVE`, `COUNTER`, `REJECT`, `NEED_INFO`, `MANUAL_REVIEW`); `policy_version`; `limit_amount` и `limit_binding` — расчётный лимит и что его ограничило (`PRODUCT_MAX`, `REVENUE`, `COLLATERAL`, `DSCR`, `OVERDRAFT`); `product_code` (без FK); `rate`; `stop_factors` (S01–S05), `flags` (F01–F07), `reason_codes` (R01–R07); `trace` — результат каждого правила политики; `comment`; `decided_by`.
- Индекс по `application_id`.
- Пишет: worker на шаге `POLICY` (`AUTO`), API банка — решение аналитика (`MANUAL`). Строки не меняются: версия политики, результат каждого правила (`trace`) и признаки в `scoring_results` позволяют проверить решение задним числом.

Предложение, документы, выдача

`offers` — варианты условий для клиента: `option_no` (1 или 2), `kind` (`AS_REQUESTED`, `COUNTER`), `product_code`, `amount`, `term_months`, `rate`, `monthly_payment`, `overpayment`, `valid_until` (по умолчанию 7 дней), `status` (`ACTIVE`, `ACCEPTED`, `DECLINED`, `EXPIRED`, `VOID`), `note`, `accepted_at`.

- Ограничения и индексы: FK `decision_id` → `decisions`; индекс по `application_id`.
- Пишет: worker (автоматическое решение, до двух вариантов) и API банка (решение аналитика, один вариант) создают. API клиента: принятый вариант — `ACCEPTED`, остальные — `VOID`; отказ от предложения — `DECLINED`; отзыв заявки — `VOID`. Worker переводит просроченные в `EXPIRED`.

`info_requests` — запросы документов и пояснений у клиента.

- Поля: `origin` (`SYSTEM` — от конвейера, `UNDERWRITER` — от аналитика), `requested_by`, `items` — пункты запроса из справочника (`BANK_STATEMENT`, `FIN_STATEMENT`, `TAX_EXPLANATION`, `ADDRESS_CONFIRMATION`, `COLLATERAL_DOCS`, `CONTRACTS`, `OTHER`), `comment`, `due_at`, `status` (`OPEN`, `ANSWERED`, `EXPIRED`, `CANCELLED`), `answer_text`, `answered_at`.
- Индекс по `application_id`.
- Пишет: worker (нет данных о выручке) и API банка создают; API клиента отвечает или отменяет при отзыве заявки; worker закрывает просроченные.

`documents` — метаданные файлов заявки; сами файлы — в каталоге `FILES_DIR` (см. «Документы и файлы»).

- Поля: `kind` (`BANK_STATEMENT`, `FIN_STATEMENT`, `COLLATERAL`, `OTHER`, `AGREEMENT`, `SCHEDULE`), `source` (`CLIENT`, `BANK`, `GENERATED`), `file_name`, `content_type`, `size`, `sha256`, `storage_key`, `uploaded_by`, `info_request_id` — к какому запросу приложен, `parsed` — разбор выписки.

- Индекс по `application_id`; FK `info_request_id` → `info_requests`.
- Пишет: API (загрузка клиентом и аналитиком, удаление клиентом своих файлов в черновике и при ответе на запрос), worker (договор в PDF).

`signatures` — подписи документа: `signer_role` (`CLIENT`, `BANK`), `signer_user_id`, `method` (`PEP_PASSWORD` — простая электронная подпись паролем, `BANK_AUTO` — автоматическая подпись банка), `document_sha256` — хэш подписанного документа.

- Ограничения и индексы: UNIQUE (`document_id`, `signer_role`); индекс по `application_id`.
- Пишет: API при подписании — сразу две строки, клиента и банка.

`loans` — выданный кредит: `account_number` — ссудный счёт из АБС, `amount`, `rate`, `term_months`, `monthly_payment`, `issued_at`, ссылка на принятое предложение.

- Ограничения: UNIQUE `application_id` — у заявки не больше одного кредита.
- Пишет: worker на задании `DISBURSE`, после ответа АБС.

`schedule_items` — график платежей: `n`, `due_date`, `payment`, `principal`, `interest`, `balance` — остаток долга после платежа.

- Индекс по `loan_id`.
- Пишет: worker вместе с кредитом, в той же транзакции. Последний платёж корректируется так, чтобы остаток стал `0.00`.

Общение и уведомления

`messages` — чат по заявке: `author_id`, `author_role`, `text`.

- Индекс по `application_id`.
- Пишет: API клиента и банка; чат доступен после отправки заявки.

`events` — события для ленты уведомлений и потока SSE (см. ниже): `audience`, `kind`, `title`, `body`, `notify`, `payload`.

- Индексы: по `application_id` и по `company_id`.
- Пишет: API и worker в той же транзакции, что и изменение данных (`services/events.py`).

Служебные таблицы

`audit_log` — журнал действий (см. ниже).

- Индекс по `entity_id`.
- Пишет: API и worker (`services/audit.py`).

`idempotency_keys` — ключи повторных запросов: `scope`, `key`, `response` (см. ниже).

- Ограничения: UNIQUE (`user_id`, `scope`, `key`).
- Пишет: API при отправке заявки и подписании.

Статус заявки и история

- Текущий статус — `applications.status`, время последней смены — `status_changed_at`. Отдельно хранятся моменты отправки (`submitted_at`), закрытия (`closed_at` — при `ISSUED`, `REJECTED`,

`DECLINED`, `CANCELLED`, `EXPIRED`) и срок ручного рассмотрения (`sla_due_at` — при переходе в `MANUAL_REVIEW`, через 24 часа).

- Статус меняет только функция `transition()` в `backend/app/services/workflow.py`. Она проверяет переход по таблице допустимых переходов с учётом роли (недопустимый — ответ 409) и в одной транзакции записывает:
  - новый статус и служебные поля заявки;
  - строку `status_history`;
  - запись `STATUS_CHANGED` в `audit_log`;
  - события для интерфейса и уведомления в `events`;
  - задания обработчику: новый прогон конвейера, формирование договора или выдачу. При финальном статусе задания заявки в очереди снимаются.
- История только дополняется. Клиент видит переходы без внутренних формулировок банка (`reason`), сотрудники банка — полностью.

Прогоны конвейера и очередь заданий

Прогон — один проход шагов конвейера по заявке. `applications.run_no` — номер последнего прогона. Он растёт при отправке заявки, после ответа клиента на запрос конвейера и при перезапуске проверки аналитиком. В начале прогона создаются восемь строк `pipeline_steps` в статусе `PENDING` и ставится задание на первый шаг; каждый выполненный шаг ставит задание на следующий.

- UNIQUE (`application_id`, `run_no`, `step`) — одна строка на шаг в прогоне. Результаты прошлых прогонов сохраняются, интерфейсы и расчёты читают шаги текущего (`run_no = applications.run_no`).
- `scoring_results` и `decisions` тоже хранят `run_no`, поэтому видно, на каком прогоне получен результат.
- Задание устаревшего прогона (`run_no` в `payload` не совпадает с заявкой) завершается без действий.

Очередь — таблица `jobs`, без отдельного брокера (`backend/app/pipeline/queue.py`):

- Задание ставится в той же транзакции, что и изменение заявки (transactional outbox): откатилась транзакция — задания нет, зафиксировалась — задание не потеряется.
- Обработчик берёт одно готовое задание запросом `SELECT … FROM jobs WHERE status = 'QUEUED' AND run_at <= :now ORDER BY run_at, id LIMIT 1 FOR UPDATE SKIP LOCKED`, переводит его в `RUNNING`, записывает `locked_at`, `locked_by` (хост и PID) и увеличивает `attempts`. `SKIP LOCKED` позволяет запускать несколько копий `worker`: занятые строки они пропускают.
- Успех — `DONE`. Ошибка — `QUEUED` с новым `run_at`: пауза 2 с × 2^(n-1), не больше 30 с. После 8 попыток — `FAILED` и событие `JOB_FAILED` для аналитиков.
- Если внешний источник недоступен, шаг переходит в `WAITING` (`waiting_since`, `next_retry_at`, `error`), а задание переносится на `next_retry_at`. После 15 минут ожидания шаг становится `FAILED`, конвейер идёт дальше, заявка попадёт к аналитику с флагом F05.
- Раз в 10 секунд обработчик возвращает в очередь задания, которые висят в `RUNNING` дольше 120 секунд (упал процесс), и закрывает просроченные предложения и запросы информации.

Оптимистическая блокировка

`applications.version` объявлен как `version_id_col` SQLAlchemy. Каждое изменение заявки выполняется запросом `UPDATE … WHERE id = :id AND version = :прочитанная` и увеличивает версию. Если строку уже изменили, запрос не обновит ни одной строки, SQLAlchemy выбросит `StaleDataError`, и API ответит 409 «Заявка изменилась, обновите страницу».

Кроме того, все изменяющие обработчики API и worker читают заявку с блокировкой строки (`SELECT … FOR UPDATE`), поэтому одновременные действия выполняются по очереди. Второе действие видит уже новый статус и получает 409 от машины состояний: например, решение аналитика по заявке, которую клиент только что отозвал. Таймер истечения предложений выбирает заявки с `FOR UPDATE SKIP LOCKED`, чтобы не ждать занятые.

События: уведомления и SSE

Таблица `events` — общая лента для уведомлений и обновления интерфейса без перезагрузки.

- `audience` — кому: `CLIENT` — пользователям компании заявки (отбор по `company_id`), `BANK` — сотрудникам банка, `ALL` — обеим сторонам.
- `notify = true` — событие видно в ленте уведомлений и учитывается в счётчике непрочитанных. `false` — только сигнал интерфейсу перезапросить данные.
- `kind`: `STATUS_CHANGED` и `STATUS_<статус>` — смена статуса и уведомление о ней; `PIPELINE_STEP` и `UPDATED` — изменились шаги или документы; `MESSAGE` — сообщение в чате; `INFO_ANSWERED` — клиент ответил на запрос; `SOURCE_WAITING`, `SOURCE_FAILED`, `JOB_FAILED`, `DISBURSE_RETRY` — проблемы конвейера для банка.
- Непрочитанные — события с `notify = true` и `id` больше `users.last_read_event_id`. Когда пользователь открывает ленту, в `last_read_event_id` сохраняется наибольший `id` из доступных ему событий.
- Поток `GET /api/stream` раз в секунду выбирает события с `id` больше последнего отправленного и отдаёт их как Server-Sent Events, где `id` события — `events.id`. При переподключении поток продолжается с `Last-Event-ID`. `id` растёт монотонно: это автоинкремент `bigint`.

Журнал действий

`audit_log` фиксирует, кто, что и над чем сделал:

- `actor` и `actor_user_id` — роль и пользователь (у действий конвейера — `SYSTEM` без пользователя);
- `action`: `LOGIN`, `DRAFT_CREATED`, `DRAFT_DELETED`, `STATUS_CHANGED`, `DOCUMENT_UPLOADED`, `AGREEMENT_SIGNED`, `ASSIGNED`, `PIPELINE_RETRY`, `FAULT_SET`, `DEMO_RESET`;
- `entity_type` и `entity_id` — объект (`application`, `user`, `simulator`, `system`) и его идентификатор строкой. Внешнего ключа нет: запись переживает удаление черновика и подходит для объектов разных типов;
- `details` — подробности: переход и причина, имя файла и SHA-256, идентификатор и SHA-256 подписанного договора;
- `request_id` — идентификатор HTTP-запроса (заголовок `X-Request-ID` или сгенерированный), по нему запись связывается с логами. У записей worker он пустой.

Записи пишутся в той же транзакции, что и действие, и не меняются. Журнал виден в досье заявки и на странице «Журнал действий»; очищается только при сбросе демо-данных.

Идемпотентность

Отправка заявки и подписание договора принимают заголовок `Idempotency-Key` (`services/idempotency.py`).

- Ключ сохраняется в `idempotency_keys` с областью действия: `submit:<id заявки>` или `sign:<id заявки>`. UNIQUE (`user_id`, `scope`, `key`).
- Если ключ уже есть, API возвращает текущее состояние заявки и не выполняет действие повторно.
- Ключ записывается в той же транзакции, что и действие. Повтор, пришедший одновременно с первым запросом, ждёт блокировку строки заявки и затем находит ключ.
- Без заголовка повтор отклоняет машина состояний: заявка уже не в нужном статусе, ответ 409.
- Срока жизни у ключей нет, они удаляются при сбросе демо-данных.

Документы и файлы

- Метаданные — в `documents`, содержимое — в каталоге `FILES_DIR` (`/data/files`). В Docker это том `files`, общий для `api` и `worker`. Файл лежит по пути `<id заявки>/<случайный hex>_<имя>`, путь хранится в `storage_key`. При сохранении считаются размер и SHA-256.
- Клиент загружает PDF, JPG, PNG, TXT и CSV до 10 МБ (`MAX_UPLOAD_MB`). Имя файла очищается от служебных символов и обрезается до 120 знаков.
- Выписка по счёту (`BANK_STATEMENT`) в формате 1С или CSV разбирается при загрузке. В `parsed` — период, число операций, поступления и списания по месяцам, среднемесячные поступления; эти данные использует скоринг.
- Договор с графиком платежей worker формирует в PDF (`AGREEMENT`, источник `GENERATED`) шрифтом DejaVu Sans. Вид `SCHEDULE` зарезервирован: график входит в договор и хранится в `schedule_items`.
- При подписании SHA-256 договора копируется в `signatures.document_sha256` и в запись журнала `AGREEMENT_SIGNED`.
- Файлы отдаются только через API (`GET /api/documents/{id}/download`) после проверки доступа к заявке.
- При удалении документа или черновика файлы удаляются с диска после фиксации транзакции; сброс демо-данных очищает каталог `FILES_DIR` целиком.

Деньги и даты

- **Деньги** — `NUMERIC(15,2)`, до 9 999 999 999 999,99 ₽; в Python — `Decimal`. Расчёты (`domain/finance.py`) округляют до копеек по правилу ROUND_HALF_UP, лимит округляется вниз до 10 000 ₽.
- **Ставки** и вероятность дефолта — `NUMERIC(5,2)`, проценты: `19.00` — 19 % годовых.
- **В API** суммы и ставки в схемах ответов объявлены как `Decimal` и передаются строками (`"3000000.00"`), без потери точности. Расчётные показатели в досье аналитика отдаются числами.
- **В JSONB** — ответы источников, признаки скоринга, детали журнала — числа хранятся как в JSON. Суммы при расчётах переводятся в `Decimal`; коэффициенты (долговая нагрузка, рентабельность,

доли) — числа с плавающей точкой.

- **Моменты времени** — `timestamptz`. Код записывает время в UTC (`datetime.now(UTC)`), значения по умолчанию ставит база (`now()`), часовой пояс базы — UTC. API отдаёт время в ISO 8601 с часовым поясом, интерфейс показывает его в часовом поясе браузера.
- **Календарные даты** без времени — `date`: регистрация компании, открытие счёта, даты платежей.
- **Сроки** кредита — целые месяцы (`term_months`). Таймеры процесса задаются в часах и минутах в настройках (`backend/app/config.py`).

Номер заявки

Номер выдаёт последовательность PostgreSQL `application_number_seq` при создании черновика: `<год>-<номер>`, номер — пять цифр с ведущими нулями, например `2026-00001`. Год берётся по UTC. Последовательность общая для всех лет. Номер удалённого черновика или откатившейся транзакции повторно не выдаётся, поэтому в нумерации бывают пропуски. Уникальность гарантирует UNIQUE `number`. При сбросе демо-данных нумерация начинается с 1.

Персональные данные

Какие поля относятся к персональным данным:

- `users` — логин, ФИО, должность;
- `companies` — ФИО руководителя; у ИП ещё ИНН, ОГРНИП и адрес, потому что это данные человека;
- `applications` — ФИО заявителя (`contact_name`), описание цели и исходный текст в `ai_parsed`;
- `consents` — факт, время и версия текста согласий на обработку персональных данных и запрос в БКИ;
- `messages`, `info_requests.answer_text`, загруженные файлы (выписки содержат операции и контрагентов), `events.body` (текст сообщений чата и ответов на запросы), `audit_log.details` (имена файлов, причины решений).

Как защищены:

- В демо все данные синтетические, реальных людей и компаний нет (см. «Тестовые данные»).
- Пароли хранятся только как bcrypt-хеш. При подписании пароль сверяется с хешем и не сохраняется.
- Клиент видит только заявки, документы и события своей компании; чужая заявка отвечает 404. Методы рабочего места банка доступны только ролям `UNDERWRITER` и `ADMIN`.
- Файлы лежат вне веб-каталога и выдаются только через API после проверки доступа.
- В логи `api` и `worker` пишутся метод, путь, код ответа, длительность, `request_id` и идентификаторы заданий; тела запросов и ответы внешних систем в логи не попадают.
- В модель ИИ, если она включена, паспортные данные и контакты не передаются (ai.md).
- Секреты (`JWT_SECRET`, ключ модели, пароль базы) задаются переменными окружения и не хранятся в репозитории.

Тестовые данные

Все данные синтетические и созданы командой:

- `backend/app/seed/demo.py` — 10 компаний (ИНН и название), 13 пользователей, две демо-заявки; продукты — из `domain/catalog.py`;
- `simulator/sim/data/companies.json` — ответы внешних систем для каждой компании: реестр, ФНС, отчётность, БКИ, поступления на счёт. Поле `persona` — какой сценарий показывает компания. Даты заданы в месяцах до текущего дня (`*_months_ago`), поэтому сценарии не устаревают;
- `frontend/public/demo-files` — выписка ИП Орлова в формате 1С (кодировка Windows-1251, как у выгрузки из 1С) и CSV.

ИНН тестовых компаний начинаются с `00` — такого кода налогового органа нет, поэтому совпадение с реальными компаниями исключено. Контрольные цифры ИНН верные, проверка данных их пропускает.

Загрузка. При старте `api` выполняет миграции и `python -m app.seed`: продукты обновляются, компании и пользователи создаются, если их нет, профили компаний подтягиваются из ЕГРЮЛ симулятора, демо-заявки — только если заявок ещё нет. Повторный запуск ничего не дублирует.

Сброс. `make reset`, `python -m app.seed --reset` или кнопка «Сбросить демо-данные» под `admin` (`POST /api/admin/demo/reset`): очищаются `applications` и все зависимые таблицы, `audit_log` и `idempotency_keys`, нумерация заявок начинается с 1, удаляются файлы, снимаются сбои симулятора, снова создаются две демо-заявки. Идентификаторы событий и заданий продолжают расти, чтобы открытые вкладки после сброса получали новые события по `Last-Event-ID`. Компании, пользователи и продукты сохраняются.

Тестовые компании

Пароль у всех — `demo`. У каждой компании один пользователь-клиент с правом подписи. Исход заявки определяют данные компании в симуляторе; пошаговые сценарии — в test-scenarios.md.

Логин	Компания	ИНН	Что показывает	Что в данных к этому ведёт
`veter`	ООО «Северный ветер»	0010000017	Автоодобрение, подписание и выдача	6 лет работы, выручка 120 млн ₽ с ростом, без просрочек, клиент банка 5 лет со стабильными поступлениями
`smirnova`	ИП Смирнова Анна Викторовна	001000000436	Контрпредложение по овердрафту: 440 000 ₽ вместо 1 млн ₽	Поступления на счёт около 1,1 млн ₽ в месяц; лимит овердрафта — 40 % поступлений
`tehnomaster`	ООО «ТехноМастер»	0010000024	Два варианта условий: та же сумма на больший срок или меньшая сумма	Платежи по двум действующим кредитам — 300 тыс. ₽ в месяц, не клиент банка
`orlov`	ИП Орлов Дмитрий Сергеевич	001000001006	Запрос выписки, затем автоматическое одобрение	ИП не сдаёт отчётность в ГИР БО и не обслуживается в банке — выручку нечем подтвердить; в выписке около 700 тыс. ₽ поступлений в месяц
`vektor`	ООО «Вектор Логистик»	0010000031	Ручное рассмотрение: флаги F01 и F02, запрос пояснений, решение аналитика	Адрес массовой регистрации, налоговая задолженность 82 тыс. ₽; демо-заявка — кредит на развитие 12 млн ₽ на 36 мес.
`romashka`	ООО «Ромашка-Строй»	0010000056	Отказ по стоп-фактору S01	Статус в ЕГРЮЛ — в процессе ликвидации

Логин	Компания	ИНН	Что показывает	Что в данных к этому ведёт
`grand`	ООО «Гранд Девелопмент»	0010000063	Кредит под залог — всегда решает аналитик	Продукт `SECURED`; демо-заявка — 60 млн ₽ на 84 мес. под залог недвижимости стоимостью 120 млн ₽
`start`	ООО «Новый старт»	0010000070	Молодой бизнес: доступен только кредит для начинающих, решение за аналитиком	5 месяцев с регистрации, нет кредитной истории
`altair`	ООО «Альтаир»	0010000088	Сбой внешнего источника: заявка ждёт и продолжает обработку	Надёжный профиль: 8 лет работы, без долгов, стабильные поступления — исход зависит только от сбоя
`meridian`	ООО «Меридиан»	0010000095	Отказ по стоп-фактору S03	Текущая просрочка 47 дней

У `vektor` в кабинет входит финансовый директор с правом подписи по доверенности; руководитель по ЕГРЮЛ — другой человек.

Сотрудники банка

Логин	Пользователь	Роль	Что может
`analyst`	Иванова Ольга Дмитриевна, кредитный аналитик	`UNDERWRITER`	Очередь, досье, решения, запросы информации, чат
`analyst2`	Ковалёв Павел Игоревич, кредитный аналитик	`UNDERWRITER`	То же: второй аналитик для параллельной работы
`admin`	Администратор демо-стенда	`ADMIN`	То же, что аналитик, плюс сброс демо-данных и сбои внешних систем

Документ 8 из 14 · docs/api.md

API

REST API кредитного конвейера: кабинет клиента, рабочее место аналитика и администрирование демо-стенда работают через один API. Изменения по заявке приходят в интерфейс потоком событий (SSE). Процесс и статусы описаны в process.md, правила решения — в credit-policy.md.

Все примеры ниже сняты с работающего стенда. Ответы сокращены: показаны ключевые поля, остальные опущены.

Где смотреть

Адрес	Что там
`http://localhost:8080/api`	Базовый адрес API. Запросы идут через nginx контейнера `frontend`, порт меняется переменной `APP_PORT`
http://localhost:8080/api/docs	Swagger UI: все методы, схемы, запросы из браузера (кнопка Authorize — токен из `POST /api/auth/login`)
http://localhost:8080/api/redoc	То же описание в ReDoc
http://localhost:8080/api/openapi.json	Спецификация OpenAPI 3.1 со стенда
api/openapi.json	Та же спецификация в репозитории. Обновляется командой `make openapi` (или `cd backend && .venv/bin/python -m app.export_openapi ../docs/api/openapi.json`), CI проверяет, что файл совпадает с кодом
api/postman_collection.json	Коллекция Postman: ключевой сценарий клиента, запрос информации, решение аналитика, администрирование, ИИ — с проверками ответов
api/requests.http	Ключевой сценарий клиента и решение аналитика для HTTP Client в IntelliJ IDEA / PyCharm / WebStorm и расширения httpYac в VS Code

Запуск коллекции из корня репозитория на поднятом стенде (38 запросов, около 150 проверок, около 15 секунд):

```bash
npx newman@6.2.2 run docs/api/postman_collection.json
```

Коллекция сама сбрасывает демо-данные, поэтому её можно запускать повторно. В Postman: Import → файл коллекции → Run collection. Для загрузки выписки укажите корень репозитория в Settings → General → Working directory.

Аутентификация и роли

Вход — `POST /api/auth/login` с логином и паролем. В ответе JWT (HS256, действует 12 часов), его передают в заголовке `Authorization: Bearer <token>`. Демо-пользователи и их сценарии перечислены в README, пароль у всех — `demo`.

```http
POST /api/auth/login
Content-Type: application/json

{"login": "veter", "password": "demo"}
```

```json
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5…",
  "user": {
    "id": "7f052893-dc43-4df5-a22b-91718eb8c9fd",
    "login": "veter",
    "full_name": "Ветров Андрей Сергеевич",
    "role": "CLIENT",
    "can_sign": true,
    "company": {"inn": "0010000017", "name": "ООО «Северный ветер»"}
  }
}
```

Роль	Демо-логины	Что доступно
Без входа	—	Вход, каталог продуктов, справочники, подбор продукта, проверки `health` и `ready`
`CLIENT`	`veter`, `smirnova`, `tehnomaster`, `orlov`, `vektor`, `romashka`, `grand`, `start`, `altair`, `meridian`	Профиль своей компании, заявки, документы, ответы на запросы банка, предложения, подписание, кредит, чат, уведомления, разбор описания задачи
`UNDERWRITER`	`analyst`, `analyst2`	Рабочее место банка `/api/bank/*`, карточка и документы любой заявки, чат, уведомления банка, просмотр состояния внешних систем и журнала действий, ИИ-сводка досье
`ADMIN`	`admin`	Всё, что доступно аналитику, плюс сброс демо-данных и управление сбоями внешних систем

Правила доступа:

- **Клиент видит только заявки своей компании.** Чужая заявка, документ, предложение или запрос возвращают 404 «Заявка не найдена» — API не раскрывает, что объект существует.
- **Сотрудники банка видят заявки всех компаний.** Черновики в рабочее место банка не попадают: методы `/api/bank/*` возвращают для них 404.
- **Карточка заявки зависит от роли.** Клиенту приходят формулировки статусов для клиента, без технических ошибок шагов (`steps[].error`) и без причин переходов в истории (`history[].reason`). Сотрудник банка видит всё.
- **Методы чужой роли** возвращают 403 «Недостаточно прав». Подписать договор может только пользователь с правом подписи (`can_sign`), иначе 403.
- **Нет токена, он неверный или истёк** — 401 «Нужно войти заново» с заголовком `WWW-Authenticate: Bearer`.

Соглашения

Формат данных

- Запросы и ответы — JSON в UTF-8. Загрузка файлов — `multipart/form-data`.
- Идентификаторы заявок, предложений, документов, запросов информации, пользователей и кредитов — UUID. Номер заявки для людей — `number` вида `2026-00003`.
- Суммы и ставки в основных схемах — строки с двумя знаками после запятой (`"3000000.00"`, `"19.00"`), чтобы не терять точность. В запросах можно передавать число или строку. Числами приходят расчётные показатели в досье, `constraints` предварительного расчёта, разобранная выписка (`parsed`) и сумма в разборе описания задачи.

- Даты — `YYYY-MM-DD`, время — ISO 8601 в UTC: `2026-10-21T11:57:20.867479Z`.
- Коды (статусы, продукты, цели, типы документов) — латиницей в верхнем регистре. Подписи для людей — в полях `*_label` и в справочниках `GET /api/meta`.
- Списки без постраничной навигации, с фиксированным пределом: очередь банка — 300 заявок, уведомления — 50, журнал действий — 200 записей.

Ошибки

Ошибка возвращается с телом `{"detail": "…"}`. Текст на русском и готов к показу пользователю:

```json
{"detail": "Без согласий на запрос в БКИ и обработку персональных данных мы не сможем рассмотреть
  заявку"}
```

Если запрос не прошёл проверку схемы (нет обязательного поля, неверный тип или значение не из списка), FastAPI возвращает 422, а `detail` — список нарушений:

```json
{
  "detail": [
    {
      "type": "literal_error",
      "loc": ["body", "action"],
      "msg": "Input should be 'APPROVE', 'COUNTER' or 'REJECT'",
      "input": "MAYBE",
      "ctx": {"expected": "'APPROVE', 'COUNTER' or 'REJECT'"}
    }
  ]
}
```

Код	Когда
200	Успешный запрос
201	Создан черновик, загружен документ, отправлено сообщение в чат
204	Удалён черновик или документ, уведомления отмечены прочитанными. Тела нет
401	Нет токена, токен неверный или истёк; неверный логин или пароль при входе
403	Метод недоступен роли; нет права подписи
404	Объект не найден или принадлежит другой компании; черновик для сотрудника банка (черновик видит только клиент); неизвестная внешняя система; ИИ-сводка при выключенном ИИ
409	Действие недоступно в текущем статусе: «Заявка уже отправлена», «Предложение больше не действует», «Действие недоступно: заявка в статусе «Черновик»»; одновременное изменение заявки — «Заявка изменилась, обновите страницу»
422	Ошибка схемы запроса или бизнес-правила: сумма и срок вне продукта, нет согласий или залога, неверный пароль при подписании, выписка не разобрана, решение аналитика вне полномочий
503	ИИ-сводка: модель не ответила

Файл больше 12 МБ отклоняет nginx (413). Собственный предел API — 10 МБ (422 «Файл больше 10 МБ»). Непредвиденная ошибка сервера — 500 с текстом `Internal Server Error`.

Идемпотентность: `Idempotency-Key`

Отправка заявки (`POST /api/applications/{id}/submit`) и подписание договора (`POST /api/applications/{id}/sign`) принимают заголовок `Idempotency-Key` — уникальную строку до 128 символов, например UUID; более длинный ключ отклоняется с кодом 422. Интерфейс создаёт новый ключ на каждое действие пользователя.

- Ключ запоминается после успешного действия, отдельно для пользователя, действия и заявки.
- **Повтор с тем же ключом** (двойной клик, повтор после обрыва связи) возвращает 200 и текущую карточку заявки. Действие не повторяется: конвейер не запускается второй раз (`run_no` не меняется), вторая подпись не создаётся.
- **Повтор без ключа или с новым ключом** проходит обычные проверки и получает 409: «Заявка уже отправлена» или «Договор сейчас нельзя подписать».
- **Неуспешная попытка не запоминается.** Например, после 422 «Неверный пароль» можно повторить подписание с тем же ключом.
- Заголовок необязателен, но без него повтор не отличить от новой попытки.

Одновременные изменения

Действия с заявкой блокируют её строку в базе (`SELECT … FOR UPDATE`), а версия заявки (`version`) страхует от потерянных обновлений. Если аналитик принимает решение, а клиент в это же время отзывает заявку, пройдёт только первое действие. Второй участник получит 409: текст о недоступном действии («Решение можно принять только по заявке на ручном рассмотрении») или «Заявка изменилась, обновите страницу». После 409 интерфейс перечитывает карточку заявки.

Идентификатор запроса: `X-Request-ID`

Каждый ответ содержит заголовок `X-Request-ID`. Клиент может передать свой идентификатор в этом же заголовке, иначе API создаёт его сам (16 шестнадцатеричных символов). Идентификатор пишется в JSON-логи `api` (поле `request_id`) и в журнал действий в базе (`audit_log.request_id`): по нему находят все следы конкретного запроса.

Асинхронная обработка

Отправка заявки, ответ на запрос конвейера, принятие предложения, подписание и перезапуск проверки отвечают сразу, а работу выполняет обработчик `worker`. Заявка проходит промежуточные статусы:

Действие	Промежуточный статус	Результат
Отправка, ответ на запрос конвейера, перезапуск	`SUBMITTED`, `IN_PROCESSING`	`OFFER`, `NEED_INFO`, `MANUAL_REVIEW` или `REJECTED`
Принятие предложения	`ACCEPTED`	`SIGNING`: договор сформирован
Подписание	`SIGNED`	`ISSUED`: кредит выдан

На демо-стенде это занимает несколько секунд. Результат получают одним из способов: опрашивают `GET /api/applications/{id}` (раз в секунду достаточно) или слушают поток `GET /api/stream` и перечитывают карточку при событии с нужным `application_id`.

CORS

На стенде интерфейс и API отдаются с одного адреса, CORS не нужен. Он разрешён только для дев-сервера Vite: `http://localhost:5173` и `http://127.0.0.1:5173`.

Методы

46 операций в 8 группах — как теги в Swagger UI. «Банк» — роли `UNDERWRITER` и `ADMIN`.

Вход

Метод	Путь	Кто	Что делает
`POST`	`/api/auth/login`	Без входа	Вход по логину и паролю: токен и профиль пользователя
`GET`	`/api/me`	Любая роль	Текущий пользователь, его роль, право подписи и компания

Каталог и справочники

Метод	Путь	Кто	Что делает
`GET`	`/api/meta`	Без входа	Справочники: подписи статусов и шагов конвейера, цели, типы документов, пункты запроса информации, коды причин отказа, флаги, стоп-факторы
`GET`	`/api/products`	Без входа	Каталог продуктов: суммы, сроки, базовая ставка, залог, требования к сроку бизнеса, автолимит
`POST`	`/api/calculator/quote`	Без входа	Подбор продуктов под цель, сумму и срок: доступность и причины недоступности, платёж и переплата по базовой ставке
`GET`	`/api/company`	`CLIENT`	Профиль компании из ЕГРЮЛ / ЕГРИП и сведения о счёте в банке. Данные старше суток (или с `?refresh=true`) обновляются из реестра

Заявки клиента

Метод	Путь	Кто	Что делает
`GET`	`/api/applications`	`CLIENT`	Заявки своей компании, новые сверху; `needs_action` — ждём действия клиента
`POST`	`/api/applications`	`CLIENT`	Создать черновик (201). Границы продукта проверяются при отправке
`GET`	`/api/applications/{application_id}`	Любая роль [1]	Карточка заявки: статус, шаги конвейера, предложения, запросы информации, документы, история, объяснение решения, кредит
`PATCH`	`/api/applications/{application_id}`	`CLIENT`	Изменить черновик: передаются только изменяемые поля
`DELETE`	`/api/applications/{application_id}`	`CLIENT`	Удалить черновик (204). Отправленную заявку не удаляют, а отзывают
`POST`	`/api/applications/{application_id}/submit`	`CLIENT`	Отправить заявку: согласия `consent_bki` и `consent_pd`, заголовок `Idempotency-Key`
`POST`	`/api/applications/{application_id}/cancel`	`CLIENT`	Отозвать заявку до подписания: предложения аннулируются, запросы закрываются

Метод	Путь	Кто	Что делает
`POST`	`/api/applications/{application_id}/documents`	Любая роль [2]	Загрузить файл (201): поля `file`, `kind`, `info_request_id` (только открытый запрос этой же заявки, иначе 422). Выписка `BANK_STATEMENT` разбирается сразу
`GET`	`/api/documents/{document_id}/download`	Любая роль [1]	Скачать документ; договор с графиком платежей — PDF
`DELETE`	`/api/documents/{document_id}`	`CLIENT`	Удалить свой документ в черновике или при запросе информации (204)
`POST`	`/api/info-requests/{request_id}/answer`	`CLIENT`	Ответить на запрос банка: текст и документы, загруженные с этим `info_request_id`
`GET`	`/api/applications/{application_id}/messages`	Любая роль [1]	Сообщения чата по заявке
`POST`	`/api/applications/{application_id}/messages`	Любая роль [1]	Написать в чат (201). Для черновика недоступно
`POST`	`/api/offers/{offer_id}/accept`	`CLIENT`	Принять вариант предложения; остальные варианты аннулируются
`POST`	`/api/applications/{application_id}/decline`	`CLIENT`	Отказаться от предложения
`POST`	`/api/applications/{application_id}/sign`	`CLIENT` с правом подписи	Подписать договор повторным вводом пароля, заголовок `Idempotency-Key`
`GET`	`/api/applications/{application_id}/loan`	Любая роль [1]	Выданный кредит и график платежей; до выдачи — `null`

[1] Клиент — только по заявкам своей компании, сотрудники банка — по любым. [2] Клиент — в статусах `DRAFT` и `NEED_INFO`, сотрудник банка — в `MANUAL_REVIEW`.

Уведомления

Метод	Путь	Кто	Что делает
`GET`	`/api/notifications`	Любая роль	Лента: 50 последних уведомлений и число непрочитанных
`POST`	`/api/notifications/read`	Любая роль	Отметить все уведомления прочитанными (204)
`GET`	`/api/stream`	Любая роль	Поток событий (SSE). Токен — только в заголовке `Authorization`

Рабочее место банка

Метод	Путь	Кто	Что делает
`GET`	`/api/bank/summary`	Банк	Счётчики: ручное рассмотрение, мои, просроченные, ожидают клиента, в обработке, ждут источник, предложения, выдано сегодня

Метод	Путь	Кто	Что делает
`GET`	`/api/bank/applications`	Банк	Очередь. Фильтры: `status` (можно несколько), `product`, `risk_class`, `mine`, `q` (название, ИНН или номер), `include_closed`. Без фильтра по статусу — активные заявки. Сверху ручное рассмотрение (по сроку рассмотрения), затем автоматическая обработка, ожидание клиента и остальные (по времени смены статуса)
`POST`	`/api/bank/applications/{application_id}/assign`	Банк	Взять заявку в работу (`MANUAL_REVIEW`, `NEED_INFO`)
`GET`	`/api/bank/applications/{application_id}/dossier`	Банк	Досье: ответы источников, скоринг с факторами, автоматическое решение с трассировкой правил, расчётный лимит, журнал действий
`POST`	`/api/bank/applications/{application_id}/terms-preview`	Банк	Предварительный расчёт условий: лимит, минимальная ставка, платёж, переплата, нарушения полномочий
`POST`	`/api/bank/applications/{application_id}/decision`	Банк	Решение: `APPROVE`, `COUNTER` или `REJECT`
`POST`	`/api/bank/applications/{application_id}/info-requests`	Банк	Запросить у клиента документы или пояснения
`POST`	`/api/bank/applications/{application_id}/restart`	Банк	Перезапустить автоматическую проверку из `MANUAL_REVIEW`, например после восстановления источника. Новый прогон, `run_no` + 1
`POST`	`/api/bank/applications/{application_id}/pipeline/retry`	Банк	Повторить ожидающий шаг конвейера сейчас, не дожидаясь паузы
`GET`	`/api/bank/pipeline/issues`	Банк	Шаги, которые ждут внешний источник или завершились ошибкой, и упавшие задания

Администрирование демо

Метод	Путь	Кто	Что делает
`POST`	`/api/admin/demo/reset`	`ADMIN`	Сбросить демо-данные: удалить заявки, снять сбои, создать две демо-заявки для аналитика
`GET`	`/api/admin/simulator/faults`	Банк	Режим каждой внешней системы в симуляторе
`PUT`	`/api/admin/simulator/faults/{source}`	`ADMIN`	Включить сбой внешней системы или вернуть её в рабочий режим
`POST`	`/api/admin/simulator/faults/reset`	`ADMIN`	Вернуть все внешние системы в рабочий режим
`GET`	`/api/admin/audit`	Банк	Журнал действий: 200 последних записей

ИИ-помощник

Метод	Путь	Кто	Что делает
`GET`	`/api/ai/status`	Любая роль	Включены ли ИИ-функции и какая модель: `{"enabled": false, "model": null, "mode": "rules"}`

Метод	Путь	Кто	Что делает
`POST`	`/api/ai/parse-need`	Любая роль	Цель, сумма и срок из описания задачи своими словами. Без модели работает на правилах
`GET`	`/api/bank/applications/{application_id}/ai-summary`	Банк	Сводка досье «сильные стороны / риски». 404, если ИИ выключен; 503, если модель не ответила

Подробнее — в ai.md.

Служебное

Метод	Путь	Кто	Что делает
`GET`	`/api/health`	Без входа	Процесс жив: `{"status": "ok"}`
`GET`	`/api/ready`	Без входа	База данных доступна: `{"status": "ready"}`

Ключевой сценарий клиента

ООО «Северный ветер» (`veter`) берёт кредит на развитие бизнеса: 3 000 000 ₽ на 24 месяца. Заявка одобряется автоматически, клиент подписывает договор и получает деньги. Вход показан в разделе «Аутентификация и роли»; во всех запросах после него — заголовок `Authorization: Bearer <token>`. Те же шаги с проверками — в папке «1. Ключевой сценарий клиента» коллекции Postman и в api/requests.http.

1. Профиль компании. Данные из ЕГРЮЛ и сведения о счёте подставляются в подбор продукта.

```http
GET /api/company
```

```json
{
  "inn": "0010000017",
  "name": "ООО «Северный ветер»",
  "legal_form": "ООО",
  "ogrn": "1007700000010",
  "registration_date": "2020-08-21",
  "status": "ACTIVE",
  "status_label": "Действующая",
  "director_name": "Ветров Андрей Сергеевич",
  "is_bank_client": true,
  "bank_account_opened_at": "2021-10-21",
  "business_months": 74,
  "account_months": 60
}
```

2. Подбор продукта. Публичный метод: продукты под цель с предварительным платежом по базовой ставке. Цель — `WORKING_CAPITAL`, `CASH_GAP`, `EQUIPMENT` или `EXPANSION`.

```http
POST /api/calculator/quote
Content-Type: application/json

{"purpose": "WORKING_CAPITAL", "amount": 3000000, "term_months": 24,
 "business_months": 74, "is_bank_client": true, "account_months": 60}
```

JSON
```
[
  {
    "product": {"code": "GROWTH", "name": "Кредит на развитие бизнеса"},
    "eligible": true,
    "issues": [],
    "rate_from": "19.00",
    "monthly_payment": "151225.85",
    "overpayment": "629420.40",
    "payment_label": "Ежемесячный платёж"
  }
]
```

3. Черновик заявки. Ответ 201 и карточка заявки.

HTTP
```
POST /api/applications
Content-Type: application/json

{"product_code": "GROWTH", "purpose": "WORKING_CAPITAL", "purpose_text": "Закупка товара к сезону",
 "amount": 3000000, "term_months": 24}
```

JSON
```
{
  "id": "74deba13-28af-41b2-89d9-8a80aef68715",
  "number": "2026-00003",
  "status": "DRAFT",
  "status_label": "Черновик",
  "next_step": "Заполните и отправьте заявку",
  "needs_action": true,
  "can_cancel": false,
  "amount": "3000000.00",
  "term_months": 24,
  "run_no": 0,
  "steps": [],
  "offers": []
}
```

Для кредита под залог (`SECURED`) в черновике нужны `collateral_type` и `collateral_value`, можно указать заявленную выручку `declared_revenue`.

4. Отправка. Согласия обязательны. Статус `SUBMITTED`, создан первый прогон конвейера: восемь шагов в статусе `PENDING`.

HTTP
```
POST /api/applications/74deba13-28af-41b2-89d9-8a80aef68715/submit
Content-Type: application/json
Idempotency-Key: fa06be31-d247-45f6-bd2d-c8a3d083afa3

{"consent_bki": true, "consent_pd": true}
```

JSON


```
{
  "id": "74deba13-28af-41b2-89d9-8a80aef68715",
  "number": "2026-00003",
  "status": "SUBMITTED",
  "status_label": "Заявка отправлена",
  "submitted_at": "2026-10-21T11:57:20.867479Z",
  "run_no": 1,
  "steps": [
    {"step": "DATA_CHECK", "title": "Проверка данных заявки", "status": "PENDING", "attempts": 0,
      "waiting_text": null},
    {"step": "EGRUL", "title": "Сведения из ЕГРЮЛ / ЕГРИП", "status": "PENDING", "attempts": 0,
      "waiting_text": null}
  ]
}
```


Шаги идут в порядке `DATA_CHECK`, `EGRUL`, `FNS`, `GIRBO`, `BKI`, `BANK`, `SCORING`, `POLICY`. Статус шага: `PENDING`, `RUNNING`, `DONE`, `SKIPPED` (например, ГИР БО для ИП), `WAITING` (источник недоступен, будет повтор) или `FAILED`. Пока шаг выполняется или ждёт, в `waiting_text` — текст для клиента.

Ошибки отправки: 422 без согласий, при сумме или сроке вне продукта («Сумма от 1 000 000 ₽ до 30 000 000 ₽»), если бизнес не подходит под требования продукта или не указан залог; 409, если заявка уже отправлена.

5. Решение. Через несколько секунд конвейер проходит все шаги. Узнать результат можно опросом карточки или по потоку событий.

HTTP

```
GET /api/applications/74deba13-28af-41b2-89d9-8a80aef68715
```

JSON


```
{
  "status": "OFFER",
  "status_label": "Предложение готово",
  "next_step": "Примите условия или откажитесь",
  "needs_action": true,
  "offers": [
    {
      "id": "f341ca0d-e97d-4239-925c-93796be2a8ed",
      "option_no": 1,
      "kind": "AS_REQUESTED",
      "product_code": "GROWTH",
      "amount": "3000000.00",
      "term_months": 24,
      "rate": "19.00",
      "monthly_payment": "151225.85",
      "overpayment": "629420.40",
      "valid_until": "2026-10-28T11:57:23.147100Z",
      "status": "ACTIVE",
      "note": null
    }
  ],
  "explanation": {
    "positive": ["Кредиты погашаются без просрочек", "Невысокая текущая долговая нагрузка", "Бизнес
      работает больше 5 лет"],
    "negative": [],
    "limit_phrase": "Текущие платежи по кредитам и прибыль ограничивают сумму",
    "rejection": [],
    "advice": []
  }
}
```

Другие исходы автоматической проверки: `NEED_INFO` — нужна выписка или пояснение (см. Запрос информации), `MANUAL_REVIEW` — заявку рассмотрит аналитик, `REJECTED` — отказ, причины простыми словами в `explanation.rejection`. Контрпредложение — тот же статус `OFFER` с вариантами `kind: "COUNTER"`, их может быть два.

6. Принятие предложения. Статус `ACCEPTED`, обработчик формирует договор.

```http
POST /api/offers/f341ca0d-e97d-4239-925c-93796be2a8ed/accept
```

```json
{
  "status": "ACCEPTED",
  "status_label": "Готовим документы",
  "offers": [{"id": "f341ca0d-e97d-4239-925c-93796be2a8ed", "status": "ACCEPTED"}]
}
```

Если срок предложения истёк или выбран другой вариант — 409 «Предложение больше не действует».

7. Договор. Через одну-две секунды заявка в статусе `SIGNING`, договор с графиком платежей — в `documents`.

```json
{
  "status": "SIGNING",
  "documents": [
    {
      "id": "5100d8cd-db59-45b0-b381-19334c3da107",
      "kind": "AGREEMENT",
      "source": "GENERATED",
      "file_name": "Кредитный договор 2026-00003.pdf",
      "content_type": "application/pdf",
      "sha256": "d525c0f7e774dae6d37c4a4d2c44d6ca012d3f64cceae38f03acb740c62b7442",
      "signed_by_client": false,
      "signed_by_bank": false
    }
  ]
}
```

Файл: `GET /api/documents/5100d8cd-db59-45b0-b381-19334c3da107/download` (PDF).

8. Подписание. Простая электронная подпись — повторный ввод пароля. Сохраняются хэш договора, время и пользователь, банк подписывает автоматически. Статус `SIGNED`, ставится задание на выдачу.

```http
POST /api/applications/74deba13-28af-41b2-89d9-8a80aef68715/sign
Content-Type: application/json
Idempotency-Key: 5b0b8d0e-1f7a-4a53-9a51-2f3c5b7f9e10

{"password": "demo"}
```

```json
{
  "status": "SIGNED",
  "status_label": "Документы подписаны",
  "documents": [{"kind": "AGREEMENT", "signed_by_client": true, "signed_by_bank": true}]
}
```

Неверный пароль — 422 «Неверный пароль», статус не `SIGNING` — 409.

9. Выдача. АБС открывает ссудный счёт и перечисляет деньги, заявка переходит в `ISSUED`.

```http
GET /api/applications/74deba13-28af-41b2-89d9-8a80aef68715/loan
```

```json
{
  "id": "b549c40e-227e-48a8-9f79-e6b0deeb6028",
  "product_code": "GROWTH",
  "account_number": "45207810000202600003",
  "amount": "3000000.00",
  "rate": "19.00",
  "term_months": 24,
  "monthly_payment": "151225.85",
  "issued_at": "2026-10-21T11:57:41.839490Z",
  "schedule": [
    {"n": 1, "due_date": "2026-11-21", "payment": "151225.85", "principal": "103725.85", "interest":
      "47500.00", "balance": "2896274.15"},
    {"n": 24, "due_date": "2028-10-21", "payment": "151225.88", "principal": "148868.79", "interest":
      "2357.09", "balance": "0.00"}
  ]
}
```

Кроме того, клиент может отказаться от предложения (`POST /api/applications/{id}/decline`) или отозвать заявку на любом этапе до подписания (`POST /api/applications/{id}/cancel`). Действие в неподходящем статусе — 409 «Действие недоступно: заявка в статусе «…»».

Запрос информации

ИП Орлов (`orlov`) не обслуживается в банке и не сдаёт бухгалтерскую отчётность, поэтому конвейер не может оценить выручку и запрашивает выписку по счёту. Заявка `EXPRESS` на 1 500 000 ₽ и 24 месяца переходит в `NEED_INFO`:

```json
{
  "id": "c368aa16-caa5-4fbd-9aac-c7460d231524",
  "number": "2026-00004",
  "status": "NEED_INFO",
  "status_label": "Нужна информация",
  "info_requests": [
    {
      "id": "08653aa1-bc44-44c6-8d90-96968864793a",
      "origin": "SYSTEM",
      "items": [
        {
          "code": "BANK_STATEMENT",
          "title": "Выписка по расчётному счёту за последние 12 месяцев",
          "hint": "Файл выгрузки из банка в формате 1С (.txt) или таблица CSV. По ней мы оценим
            выручку."
        }
      ],
      "comment": "Чтобы оценить выручку, нам не хватает данных о поступлениях на счёт.",
      "due_at": "2026-10-24T11:58:00.444896Z",
      "status": "OPEN"
    }
  ]
}
```

Загрузка документа к запросу — `multipart/form-data` с полями `file`, `kind` и `info_request_id`:

```bash
curl -X POST http://localhost:8080/api/applications/c368aa16-caa5-4fbd-9aac-c7460d231524/documents
  \
  -H "Authorization: Bearer $TOKEN" \
  -F "file=@frontend/public/demo-files/vypiska_orlov_1c.txt;type=text/plain" \
  -F "kind=BANK_STATEMENT" \
  -F "info_request_id=08653aa1-bc44-44c6-8d90-96968864793a"
```

```json
{
  "id": "dc63f9b7-15fc-4b49-919c-62cf2461d47c",
  "kind": "BANK_STATEMENT",
  "kind_label": "Выписка по расчётному счёту",
  "source": "CLIENT",
  "file_name": "vypiska_orlov_1c.txt",
  "size": 22459,
  "sha256": "eeda7e2f5c6ea328bea86f01ea77b8fef22f87dc7975d2643dbcbef0e70d24fd",
  "info_request_id": "08653aa1-bc44-44c6-8d90-96968864793a",
  "parsed": {
    "source": "1C",
    "account": "40802810000000001006",
    "period_from": "2025-10-05",
    "period_to": "2026-09-25",
    "operations": 60,
    "months": [{"month": "2025-10", "inflow": 651000.0, "outflow": 124060.0}],
    "total_inflow": 8400000.0,
    "avg_monthly_inflow": 700000.0
  }
}
```

Правила загрузки:

- `kind`: `BANK_STATEMENT`, `FIN_STATEMENT`, `COLLATERAL` или `OTHER` (по умолчанию). Договор и график формирует банк.
- Клиент загружает PDF, JPG, PNG, TXT или CSV до 10 МБ. Хранится SHA-256 файла.
- Выписка разбирается сразу: формат обмена 1С (`1CClientBankExchange`, `.txt`) или CSV с колонками `Дата; Приход; Расход`, кодировки UTF-8 и Windows-1251. Если разобрать не удалось — 422 «Не удалось разобрать выписку: …». Примеры — в frontend/public/demo-files.

Ответ на запрос. Документы привязываются к запросу при загрузке через `info_request_id`, в теле ответа — только текст.

```http
POST /api/info-requests/08653aa1-bc44-44c6-8d90-96968864793a/answer
Content-Type: application/json

{"text": "Выписка за 12 месяцев"}
```

```json
{
  "status": "IN_PROCESSING",
  "run_no": 2,
  "info_requests": [{"status": "ANSWERED", "answer_text": "Выписка за 12 месяцев", "answered_at": "2026-
    10-21T11:58:11.487059Z"}]
}
```

Ответ на запрос конвейера (`origin: "SYSTEM"`) запускает новый прогон проверки: выручка оценивается по выписке, и заявка Орлова через несколько секунд переходит в `OFFER`. Ответ на запрос аналитика (`origin: "UNDERWRITER"`) возвращает заявку тому же аналитику в `MANUAL_REVIEW`.

Ошибки: 422 «Приложите выписку по счёту в формате 1С или CSV», если запрошена выписка, а разобранной выписки нет; 422 «Добавьте ответ или приложите документ» — пустой ответ без документов; 409 «На этот запрос уже нельзя ответить» — запрос закрыт.

Рабочее место аналитика

Аналитик (`analyst`) работает с заявками, которые конвейер не решил сам. После сброса демо-данных в очереди две такие заявки: ООО «Вектор Логистик» (флаги F01, F02) и ООО «Гранд Девелопмент» (кредит под залог — всегда решает аналитик).

1. Очередь.

```HTTP
GET /api/bank/applications?status=MANUAL_REVIEW
```

```JSON
[
  {
    "id": "9769358c-ba00-4d64-8881-eb7d97bb5184",
    "number": "2026-00002",
    "company_name": "ООО «Гранд Девелопмент»",
    "inn": "0010000063",
    "product_code": "SECURED",
    "amount": "60000000.00",
    "term_months": 84,
    "status": "MANUAL_REVIEW",
    "status_label": "Ручное рассмотрение",
    "risk_class": "A",
    "score": 830,
    "flags": [],
    "assignee_name": null,
    "assignee_is_me": false,
    "sla_due_at": "2026-10-22T11:57:04.214281Z",
    "overdue": false,
    "waiting_source": null
  }
]
```

2. Досье. `GET /api/bank/applications/{id}/dossier` собирает всё для решения:

Поле	Что в нём
`application`	Карточка заявки в представлении банка
`sources`	Результат каждого шага конвейера: `status`, `result` (нормализованный ответ источника), `error`, `attempts`, `finished_at`
`scoring`	Балл, класс риска, условная вероятность дефолта `pd` (%), версия карты, факторы с баллами и формулировками, входные признаки
`auto_decision`, `last_decision`	Автоматическое и последнее решение: исход, версия политики, лимит и его ограничитель, ставка, флаги и стоп-факторы с расшифровкой, трассировка правил
`limit`	Расчётный лимит, ограничитель, все ограничения и ставка для текущих условий
`assignee`, `assignee_is_me`, `sla_due_at`	Кто ведёт заявку и до какого времени её нужно рассмотреть
`audit`	Журнал действий по заявке
`info_item_presets`	Пункты, которые можно запросить у клиента

```json
{
  "application": {"id": "9769358c-ba00-4d64-8881-eb7d97bb5184", "number": "2026-00002", "status":
    "MANUAL_REVIEW"},
  "sources": {
    "BKI": {
      "status": "DONE",
      "result": {
        "has_history": true,
        "max_overdue_days_24m": 0,
        "current_overdue_days": 0,
        "loans": [{"lender": "Банк «Восток»", "kind": "Кредитная линия", "amount": 80000000, "balance":
          52000000, "monthly_payment": 2500000, "opened_at": "2024-08-21"}]
      },
      "error": null,
      "attempts": 1
    }
  },
  "scoring": {
    "score": 830,
    "risk_class": "A",
    "pd": 1.0,
    "version": "v1.0",
    "factors": [{"code": "BUSINESS_AGE", "title": "Срок ведения бизнеса", "value": "8 лет 4 мес.",
      "phrase": "Бизнес работает больше 5 лет", "points": 50}]
  },
  "auto_decision": {
    "kind": "AUTO",
    "outcome": "MANUAL_REVIEW",
    "policy_version": "v1.0",
    "limit": 78570000.0,
    "limit_binding": "DSCR",
    "rate": 15.5,
    "flags": [],
    "stop_factors": [],
    "trace": [{"rule": "Флаги ручного рассмотрения", "detail": "продукт рассматривает аналитик",
      "result": "есть"}]
  },
  "limit": {"limit": 78570000.0, "binding": "DSCR", "rate": 15.5},
  "sla_due_at": "2026-10-22T11:57:04.214281Z"
}
```

3. Взять в работу — `POST /api/bank/applications/{id}/assign`. Решение или запрос информации тоже назначают заявку на аналитика, если она ещё ничья.

4. Предварительный расчёт условий. Пересчитывает лимит, минимальную ставку по классу риска, платёж и переплату и проверяет полномочия. Ставку можно передать в `rate`, иначе берётся расчётная. Нарушения возвращаются в `errors`, код ответа — 200.

```http
POST /api/bank/applications/9769358c-ba00-4d64-8881-eb7d97bb5184/terms-preview
Content-Type: application/json

{"product_code": "SECURED", "amount": 60000000, "term_months": 84}
```

JSON

```
{
  "limit": "78570000",
  "limit_binding": "DSCR",
  "limit_phrase": "Текущие платежи по кредитам и прибыль ограничивают сумму",
  "min_rate": "15.50",
  "rate": "15.50",
  "monthly_payment": "1174700.81",
  "overpayment": "38674868.04",
  "payment_label": "Ежемесячный платёж",
  "errors": [],
  "constraints": [
    {"code": "PRODUCT_MAX", "amount": 150000000.0},
    {"code": "REVENUE", "amount": 420000000.0},
    {"code": "COLLATERAL", "amount": 96000000.0},
    {"code": "DSCR", "amount": 78579746.81, "max_payment": 1538461.54}
  ]
}
```


С `"rate": 14` в ответе было бы `"errors": ["Ставка не может быть ниже расчётной: 15.50%"]`.

5. Решение. Только для заявки в `MANUAL_REVIEW`, иначе 409.

action	Поля	Результат
`APPROVE`	`rate` и `comment` — необязательно	`OFFER`, предложение на запрошенных условиях (`kind: "AS_REQUESTED"`)
`COUNTER`	`product_code`, `amount`, `term_months`, `rate` — что меняется (остальное берётся из заявки), `comment`	`OFFER`, контрпредложение (`kind: "COUNTER"`, `note: "Условия предложены специалистом банка"`)
`REJECT`	`reason_code` (`R01` – `R07`) и `comment` обязательны	`REJECTED`, клиент видит причину по коду, для `R07` — текст комментария

HTTP
```
POST /api/bank/applications/9769358c-ba00-4d64-8881-eb7d97bb5184/decision
Content-Type: application/json

{"action": "APPROVE", "comment": "Залог покрывает сумму, класс риска A"}
```

JSON

```
{
  "status": "OFFER",
  "status_label": "Предложение выдано",
  "offers": [
    {"kind": "AS_REQUESTED", "product_code": "SECURED", "amount": "60000000.00", "term_months": 84,
      "rate": "15.50", "monthly_payment": "1174700.81", "status": "ACTIVE"}
  ],
  "history": [
    {"to_status": "MANUAL_REVIEW", "actor": "SYSTEM", "actor_name": null, "reason": "Продукт
      рассматривает аналитик"},
    {"to_status": "OFFER", "actor": "UNDERWRITER", "actor_name": "Иванова Ольга Дмитриевна", "reason":
      "Залог покрывает сумму, класс риска A"}
  ]
}
```


Полномочия аналитика (credit-policy.md) проверяются при решении, нарушение — 422 с текстом:

- сумма и срок — в границах продукта, ставка — не ниже расчётной;

- сумма выше расчётного лимита — только с комментарием и не больше чем на 20%. Предварительный расчёт наличие комментария не проверяет;
- отказ без причины или без комментария — 422 «Укажите причину отказа», «Добавьте комментарий к отказу».

Пример: у ООО «Вектор Логистик» лимит 5 650 000 ₽. `{"action": "COUNTER", "amount": 6000000, "term_months": 36}` вернёт 422 «Сумма выше расчётного лимита — добавьте комментарий с обоснованием»; с полем `comment` клиент получит контрпредложение на 6 000 000 ₽ под 20,5%.

6. Запрос информации у клиента. Пункты — коды из `info_item_presets` досье: `BANK_STATEMENT`, `FIN_STATEMENT`, `TAX_EXPLANATION`, `ADDRESS_CONFIRMATION`, `COLLATERAL_DOCS`, `CONTRACTS`, `OTHER`. Срок ответа `due_hours` — от 1 до 720 часов, по умолчанию 72.

HTTP
```
POST /api/bank/applications/b5a86aab-9f39-4ab4-b000-0b2c0065a8c0/info-requests
Content-Type: application/json

{"items": ["TAX_EXPLANATION"], "comment": "Поясните, когда погасите налоговую задолженность",
  "due_hours": 48}
```

JSON
```
{
  "status": "NEED_INFO",
  "status_label": "Ожидает клиента",
  "info_requests": [
    {
      "id": "81ff75fc-b5a3-40d4-afc5-1333fd3daf4e",
      "origin": "UNDERWRITER",
      "items": [{"code": "TAX_EXPLANATION", "title": "Пояснение по налоговой задолженности", "hint":
        "Когда и как планируете погасить. Можно приложить платёжное поручение."}],
      "comment": "Поясните, когда погасите налоговую задолженность",
      "due_at": "2026-10-23T11:58:53.006169Z",
      "status": "OPEN"
    }
  ]
}
```

Клиент отвечает через `POST /api/info-requests/{id}/answer`, и заявка возвращается аналитику в `MANUAL_REVIEW`. Если клиент не ответил в срок, заявка закрывается со статусом `EXPIRED`. Неизвестный код пункта — 422.

7. Сбои источников. Пока внешний источник недоступен, шаг конвейера в статусе `WAITING` и повторяется с растущей паузой. Банк видит такие заявки в `GET /api/bank/pipeline/issues` и в очереди (`waiting_source`):

JSON
```
[
  {
    "application_id": "ffe3ecc7-58d5-457a-ba43-5f64c4fa12b6",
    "number": "2026-00004",
    "company_name": "ООО «Альтаир»",
    "status": "IN_PROCESSING",
    "step": "BKI",
    "step_title": "Кредитная история",
    "step_status": "WAITING",
    "attempts": 1,
    "error": "bki: сервис ответил ошибкой 503",
    "waiting_since": "2026-10-21T12:17:19.282221Z",
    "next_retry_at": "2026-10-21T12:17:21.282269Z"
  }
]
```

После восстановления источника `POST /api/bank/applications/{id}/pipeline/retry` повторяет шаг сразу, не дожидаясь паузы. Если ожидание превысило лимит (15 минут), шаг завершается с ошибкой, заявка уходит аналитику с флагом F05, а после восстановления источника её проверку перезапускают: `POST /api/bank/applications/{id}/restart`.

Управление демо-стендом

Методы для подготовки стенда к показу и проверки устойчивости конвейера.

Сброс демо-данных — `POST /api/admin/demo/reset`, только `ADMIN`. Удаляет заявки со всеми документами, событиями и ключами идемпотентности, очищает журнал действий, начинает нумерацию заявок заново и снимает сбои симулятора. Затем создаёт две демо-заявки — ООО «Вектор Логистик» и ООО «Гранд Девелопмент», — которые сразу проходят конвейер и через несколько секунд попадают к аналитику. Пользователи, компании и продукты сохраняются.

JSON
```
{"status": "ok", "preseeded": 2}
```

То же из командной строки: `make reset` или `docker compose exec api python -m app.seed --reset`.

Сбои внешних систем. `GET /api/admin/simulator/faults` показывает режим каждой системы, `PUT /api/admin/simulator/faults/{source}` меняет его, `POST /api/admin/simulator/faults/reset` возвращает все системы в рабочий режим. Неизвестная система — 404. Ответ всех трёх методов — список систем:

HTTP
```
PUT /api/admin/simulator/faults/bki
Content-Type: application/json

{"mode": "error"}
```

JSON
```
[
  {"source": "egrul", "title": "ЕГРЮЛ / ЕГРИП", "mode": "ok", "latency_ms": 0},
  {"source": "bki", "title": "Бюро кредитных историй", "mode": "error", "latency_ms": 0}
]
```

source	Система	Где используется
`egrul`	ЕГРЮЛ / ЕГРИП	Шаг `EGRUL`, профиль компании
`fns`	ФНС «Прозрачный бизнес»	Шаг `FNS`
`girbo`	ГИР БО (отчётность)	Шаг `GIRBO`
`bki`	Бюро кредитных историй	Шаг `BKI`
`bank`	Внутренние данные банка	Шаг `BANK`, профиль компании
`core`	Выдача кредита (АБС)	Выдача после подписания

mode	Что делает симулятор	Что видит конвейер
`ok`	Отвечает с обычной задержкой	Шаг выполняется
`error`	Отвечает 503	Шаг ждёт источник (`WAITING`) и повторяется через 2, 4, 8, 16, 30 секунд. После 5 ошибок подряд вызовы источника приостанавливаются на 20 секунд
`timeout`	Отвечает через 30 секунд	Вызов прерывается по таймауту 3 секунды, дальше — как при `error`
`slow`	Добавляет к задержке `latency_ms` (0–20 000 мс)	Если ответ дольше 3 секунд — как при `timeout`

Пока источник недоступен, клиент видит «Ждём ответа бюро кредитных историй», заявка остаётся в `IN_PROCESSING` и не теряется. Сбой `core` задерживает выдачу: заявка остаётся в `SIGNED`, выдача повторяется, пока АБС не ответит. Если при сбое ЕГРЮЛ клиент открывает профиль компании, API возвращает последние сохранённые данные.

Журнал действий — `GET /api/admin/audit`: 200 последних записей, новые сверху. Записываются вход, создание и удаление черновиков, смена статусов, загрузка документов, подписание, назначение, повтор шага, сброс демо, смена режима источников и снятие всех сбоев.

JSON


```
[
  {
    "id": 25,
    "action": "FAULT_SET",
    "actor": "ADMIN",
    "user": "Администратор демо-стенда",
    "entity_type": "simulator",
    "entity_id": "bki",
    "details": {"mode": "error", "latency_ms": 0},
    "created_at": "2026-10-21T12:17:17.932235Z"
  },
  {
    "id": 15,
    "action": "AGREEMENT_SIGNED",
    "actor": "CLIENT",
    "user": "Ветров Андрей Сергеевич",
    "entity_type": "application",
    "entity_id": "3e90e271-3dfe-40fd-ab69-de24a0827322",
    "details": {"document": "685685dc-f82d-40d1-b31b-28ab5ed8a144", "sha256":
      "e76e0f4c68dea5d8093ed93c9ae4547f3cbbd510cff6bea7a0ed9bf212c45472"},
    "created_at": "2026-10-21T12:16:40.965599Z"
  }
]
```

Поток событий (SSE)

`GET /api/stream` — поток Server-Sent Events: интерфейс обновляется без перезагрузки страницы. Токен передаётся **только** в заголовке `Authorization: Bearer <token>`, в адресе он не принимается — так токен не попадает в логи. Стандартный `EventSource` браузера заголовки не передаёт, поэтому интерфейс читает поток через `fetch` (frontend/src/api/stream.ts).

```bash
TOKEN=$(curl -s http://localhost:8080/api/auth/login -H 'Content-Type: application/json' \
  -d '{"login": "veter", "password": "demo"}' | jq -r .token)
curl -N http://localhost:8080/api/stream -H "Authorization: Bearer $TOKEN"
```

Поток клиента `veter` при отправке заявки и автоматическом одобрении (запись со стенда, часть однотипных событий опущена):

```
retry: 3000

: ping

id: 29
event: message
data: {"id": 29, "kind": "STATUS_CHANGED", "application_id": "74deba13-28af-41b2-89d9-8a80aef68715",
  "title": "Заявка отправлена", "notify": false}

id: 30
event: message
data: {"id": 30, "kind": "STATUS_SUBMITTED", "application_id": "74deba13-28af-41b2-89d9-8a80aef68715",
  "title": "Заявка № 2026-00003 отправлена", "notify": true}

id: 31
event: message
data: {"id": 31, "kind": "STATUS_CHANGED", "application_id": "74deba13-28af-41b2-89d9-8a80aef68715",
  "title": "Проверяем заявку", "notify": false}

id: 32
event: message
data: {"id": 32, "kind": "PIPELINE_STEP", "application_id": "74deba13-28af-41b2-89d9-8a80aef68715",
  "title": "PIPELINE_STEP", "notify": false}

id: 39
event: message
data: {"id": 39, "kind": "STATUS_CHANGED", "application_id": "74deba13-28af-41b2-89d9-8a80aef68715",
  "title": "Предложение готово", "notify": false}

id: 40
event: message
data: {"id": 40, "kind": "STATUS_OFFER", "application_id": "74deba13-28af-41b2-89d9-8a80aef68715",
  "title": "Предложение по заявке № 2026-00003 готово", "notify": true}
```

Формат:

- первой строкой сервер присылает `retry: 3000` — пауза перед переподключением;
- событие — `id` (сквозной номер), `event: message` и `data` с JSON: `id`, `kind`, `application_id`, `title`, `notify`;
- раз в 15 секунд приходит комментарий `: ping`, чтобы соединение не закрывалось;
- новые события сервер проверяет раз в секунду.

Получив событие, интерфейс перечитывает затронутые данные: карточку заявки по `application_id`, очередь, ленту уведомлений. События с `notify: true` попадают и в ленту `GET /api/notifications` вместе с текстом `body`.

kind	Кому	notify	Когда
`STATUS_CHANGED`	Клиенту и банку	нет	Любая смена статуса; `title` — новый статус для клиента
`STATUS_<статус>`	Клиенту	да	Статусы `SUBMITTED`, `NEED_INFO`, `MANUAL_REVIEW`, `OFFER`, `REJECTED`, `SIGNING`, `ISSUED`, `EXPIRED`
`STATUS_<статус>`	Банку	да	Статусы `MANUAL_REVIEW`, `ACCEPTED`, `DECLINED`, `CANCELLED`
`PIPELINE_STEP`	Клиенту и банку	нет	Шаг конвейера завершился или ждёт источник
`UPDATED`	Клиенту и банку	нет	Загружен или удалён документ
`INFO_ANSWERED`	Банку	да	Клиент ответил на запрос
`MESSAGE`	Другой стороне	да	Новое сообщение в чате
`SOURCE_WAITING`	Банку	нет	Шаг начал ждать внешний источник
`SOURCE_FAILED`	Банку	да	Источник недоступен дольше лимита
`DISBURSE_RETRY`	Банку	да	Выдача кредита повторяется из-за сбоя АБС
`JOB_FAILED`	Банку	да	Задание конвейера не выполнилось после всех попыток

Клиент получает события только своей компании, сотрудники банка — события для банка по всем заявкам.

Переподключение. Без заголовка `Last-Event-ID` поток начинается с текущего момента. Чтобы не пропустить события за время обрыва, передайте номер последнего полученного события — сервер пришлёт всё, что было после него:

```bash
curl -N http://localhost:8080/api/stream -H "Authorization: Bearer $TOKEN" -H "Last-Event-ID: 44"
```

Сброс демо-данных не начинает нумерацию событий заново, поэтому открытые вкладки после сброса продолжают получать события без перезагрузки.

nginx проксирует поток без буферизации, таймаут чтения — час (frontend/nginx.conf); пинги не дают соединению простаивать.

Внутренний API симулятора

> **Внутренний API.** Симулятор не опубликован наружу: у сервиса `simulator` нет проброшенных портов в docker-compose.yml, а nginx (frontend/nginx.conf) проксирует только `/api/` в сервис `api`. Симулятор доступен лишь внутри сети Docker по адресу `http://simulator:8100`. Его вызывают `worker` (шаги конвейера и выдача) и `api` (профиль компании и управление сбоями). Для показа сбоев используйте `/api/admin/simulator/faults`.

Симулятор (simulator/sim/main.py) заменяет внешние системы синтетическими данными 10 тестовых компаний (simulator/sim/data/companies.json). Даты в ответах считаются от текущего дня, поэтому сценарии не устаревают. Контракты упрощены, смысл интеграций сохранён.

Метод	Путь	Система	Что возвращает
`GET`	`/health`	—	`{"status": "ok"}`
`GET`	`/egrul/{inn}`	ЕГРЮЛ / ЕГРИП	Название, форма (`ООО` или `IP`), ОГРН, дата регистрации, статус (`ACTIVE`, `LIQUIDATING` и др.), ОКВЭД, адрес, руководитель и дата его назначения
`GET`	`/fns/{inn}`	ФНС «Прозрачный бизнес»	Налоговая задолженность, массовый адрес, дисквалификация руководителя, численность, налоговый режим
`GET`	`/girbo/{inn}`	ГИР БО	Выручка, прибыль от продаж и чистая прибыль за два последних года. Для ИП и компаний без отчётности — `{"available": false, "years": []}`
`POST`	`/bki/report`	Бюро кредитных историй	Тело `{"inn", "consent_id"}`; без согласия — 400. Наличие истории, максимальная просрочка за 24 месяца, текущая просрочка, действующие кредиты: кредитор, вид, сумма, остаток, платёж, дата
`GET`	`/bank/clients/{inn}`	Внутренние данные банка	Клиент ли банка, дата открытия счёта, поступления по месяцам за 12 месяцев. Не клиент — `{"is_client": false, "months": []}`
`POST`	`/core/loans`	АБС, выдача кредита	Тело `{"inn", "application_number", "amount", "rate", "term_months"}`. Номер ссудного счёта, время перечисления, `"status": "DISBURSED"`
`GET`	`/admin/faults`	Управление	Режимы всех систем
`PUT`	`/admin/faults/{source}`	Управление	Тело `{"mode", "latency_ms"}`; неизвестная система — 404
`POST`	`/admin/faults/reset`	Управление	Все системы в режиме `ok`
`GET`	`/admin/companies`	Справка	ИНН, название и сценарий каждой тестовой компании

Неизвестный ИНН — 404 «Компания с таким ИНН не найдена в реестре»: конвейер передаёт такую заявку аналитику. Режимы сбоев — те же, что в разделе «Управление демо-стендом». Обычная задержка ответа: ЕГРЮЛ 300 мс, ФНС 250 мс, ГИР БО 300 мс, БКИ 600 мс, банк 200 мс, АБС 400 мс, разброс ±30%. Множитель задержки задаёт переменная `SIMULATOR_LATENCY_SCALE`.

Примеры для ООО «Северный ветер». `GET /egrul/0010000017`:

JSON

```
{
  "inn": "0010000017",
  "name": "ООО «Северный ветер»",
  "legal_form": "ООО",
  "ogrn": "1007700000010",
  "registration_date": "2020-08-21",
  "status": "ACTIVE",
  "okved": "46.39",
  "okved_name": "Торговля оптовая неспециализированная пищевыми продуктами",
  "address": "г. Москва, Северный проезд, д. 12, офис 4",
  "director": {"name": "Ветров Андрей Сергеевич", "since": "2020-08-21"}
}
```

`POST /bki/report` с телом `{"inn": "0010000017", "consent_id": "<идентификатор согласия>"}`:

```json
{
  "inn": "0010000017",
  "has_history": true,
  "max_overdue_days_24m": 0,
  "current_overdue_days": 0,
  "loans": [{"lender": "Банк «Восток»", "kind": "Кредит", "amount": 5000000, "balance": 2100000,
    "monthly_payment": 150000, "opened_at": "2024-04-21"}]
}
```

Обратиться к симулятору для отладки можно из контейнера `api`:

```bash
docker compose exec api python -c "import urllib.request;
  print(urllib.request.urlopen('http://simulator:8100/egrul/0010000017').read().decode())"
```

Документ 9 из 14 · docs/test-scenarios.md

Тестовые сценарии

Сценарии для ручной проверки и их автоматические аналоги. Данные синтетические, у каждой тестовой компании свой сценарий: исход определяется её данными в симуляторе внешних систем, а не подменой в коде.

Тестовые учётные записи

Пароль у всех — `demo`. На странице входа есть кнопки быстрого входа.

Логин	Компания	Что показывает	Заявка	Ожидаемый результат
`veter`	ООО «Северный ветер», ИНН 0010000017	Ключевой сценарий: автоодобрение и выдача	Кредит на развитие бизнеса, 3 000 000 ₽ на 24 мес.	Одобрено автоматически на запрошенных условиях → подписание → кредит выдан
`smirnova`	ИП Смирнова А. В., ИНН 001000000436	Контрпредложение, лимит по оборотам	Овердрафт, 1 000 000 ₽	Предложение на 440 000 ₽: 40 % среднемесячных поступлений на счёт
`tehnomaster`	ООО «ТехноМастер», ИНН 0010000024	Два варианта условий	Кредит на развитие, 6 000 000 ₽ на 24 мес.	Два варианта: та же сумма на 60 мес. или меньшая сумма на 24 мес.
`orlov`	ИП Орлов Д. С., ИНН 001000001006	Банку не хватает данных	Экспресс-кредит, 1 500 000 ₽ на 24 мес.	«Нужна информация» → клиент загружает выписку → автоматическое одобрение
`vektor`	ООО «Вектор Логистик», ИНН 0010000031	Ручное рассмотрение	Кредит на развитие, 12 000 000 ₽ на 36 мес. (создаётся при сбросе данных)	У аналитика: флаги F01 (массовый адрес) и F02 (налоговая задолженность), класс B, лимит ниже запроса
`grand`	ООО «Гранд Девелопмент», ИНН 0010000063	Кредит под залог — всегда решает аналитик	Кредит под залог, 60 000 000 ₽ на 84 мес., залог 120 млн ₽ (создаётся при сбросе данных)	У аналитика: класс A, лимит выше запроса — можно одобрить
`start`	ООО «Новый старт», ИНН 0010000070	Молодой бизнес	Кредит для начинающего бизнеса	Подбор предлагает продукт для бизнеса младше года; заявка уходит аналитику (класс C)
`altair`	ООО «Альтаир», ИНН 0010000088	Сбой внешнего источника	Экспресс-кредит, 2 000 000 ₽ на 12 мес.	При сбое БКИ заявка ждёт источник, после восстановления — одобрение
`romashka`	ООО «Ромашка-Строй», ИНН 0010000056	Стоп-фактор S01	Любая	Отказ: компания в процессе ликвидации
`meridian`	ООО «Меридиан», ИНН 0010000095	Стоп-фактор S03	Любая	Отказ: текущая просроченная задолженность
`analyst`, `analyst2`	Кредитные аналитики	Рабочее место банка	—	Очередь, досье, решения
`admin`	Администратор демо	Сброс данных и сбои	—	Страница «Конвейер и сбои»

Подготовка

- Запустите стенд по README и откройте http://localhost:8080.
- Чтобы начать с чистого листа, войдите как `admin`, откройте «Конвейер и сбои» и нажмите «Сбросить демо-данные». Заявки удалятся, все внешние системы вернутся в рабочее состояние, а для `vektor` и `grand` снова появятся заявки на ручном рассмотрении. То же из консоли: `make reset`.
- Сценарии независимы: каждый использует свою компанию.

Сценарии

С1. Ключевой сценарий: от потребности до выдачи — `veter`

1. Войдите как `veter`. На главной — данные компании из ЕГРЮЛ и кнопка «Подобрать кредит».
2. Нажмите «Подобрать кредит». В поле «Опишите задачу своими словами» введите: «Нужно 3 млн на закупку товара к сезону, верну за 2 года» и нажмите «Подобрать по описанию». Ожидание: распознаны цель «Пополнить оборотку», сумма 3 000 000 ₽ и срок 2 года; справа — подходящие продукты с платежом и переплатой.
3. В карточке «Кредит на развитие бизнеса» нажмите «Оформить заявку» → «Далее» → отметьте оба согласия → «Отправить заявку».
4. Ожидание: страница заявки показывает шаги проверки (ЕГРЮЛ, ФНС, отчётность, БКИ, обороты, оценка риска, решение) и обновляется сама. Через несколько секунд — «Кредит одобрен», условия и блок «Что повлияло на условия».
5. «Принять предложение». Ожидание: «Готовим договор», затем «Подпишите документы» и договор в PDF (кнопка «Открыть»).
6. «Подписать договор» → пароль `demo` → «Подписать». Ожидание: «Кредит выдан», сумма, ставка, ближайший платёж и график на 24 месяца с нулевым остатком в конце.
7. Колокольчик в шапке: уведомления о каждом этапе.

Автотесты: `test_auto_approval_and_issue` (API), «ключевой сценарий» (Playwright).

С2. Контрпредложение по овердрафту — `smirnova`

1. Подберите кредит с целью «Закрыть кассовый разрыв», сумма 1 000 000 ₽, выберите «Овердрафт», отправьте заявку.
2. Ожидание: «Банк предлагает другие условия» — 440 000 ₽ и объяснение: лимит ограничен поступлениями на счёт.

Автотест: `test_overdraft_counter_offer_limited_by_inflows`.

С3. Выбор из двух вариантов — `tehnomaster`

1. Кредит на развитие бизнеса, 6 000 000 ₽ на 24 мес., отправьте.
2. Ожидание: два варианта — 6 000 000 ₽ на 60 мес. с меньшим платежом или около 3,2 млн ₽ на 24 мес. Выберите вариант и примите — дальше как в С1.

Автотест: `test_counter_offer_with_two_options`.

C4. Банк запрашивает выписку — `orlov`

1. Экспресс-кредит, 1 500 000 ₽ на 24 мес., отправьте.
2. Ожидание: «Нужна информация» — запрос выписки по счёту за 12 месяцев и срок ответа. ИП не сдаёт отчётность в ГИР БО и не обслуживается в банке, поэтому выручку нечем подтвердить.
3. Скачайте тестовую выписку по ссылке «Скачать тестовую выписку» (формат 1С, файл `frontend/public/demo-files/vypiska_orlov_1c.txt`) и прикрепите её кнопкой «Прикрепить выписку». Ожидание: выписка разобрана — среднемесячные поступления около 700 000 ₽.
4. «Отправить ответ». Ожидание: проверка запускается заново и заканчивается одобрением.

Автотесты: `test_statement_request_then_automatic_decision`, «банку не хватает данных» (Playwright).

C5. Ручное рассмотрение: пояснения и решение выше лимита — `vektor` и `analyst`

1. Войдите как `analyst`. В очереди заявка ООО «Вектор Логистик» со статусом «Ручное рассмотрение», флагами F01 и F02 и сроком рассмотрения.
2. Откройте досье: вкладки «Сводка» (флаги, расчётный лимит и что его ограничивает, как принималось решение), «Источники», «Скоринг» (вклад каждого фактора), «Документы», «Чат», «История». Нажмите «Взять в работу».
3. «Запросить информацию» → «Пояснение по налоговой задолженности» → комментарий → отправьте. Ожидание: заявка переходит в «Ожидает клиента».
4. Войдите как `vektor`: на заявке запрос банка. Напишите ответ и отправьте. Ожидание: заявка возвращается аналитику.
5. Как `analyst`: «Изменить условия», сумма на 10 % выше расчётного лимита. Ожидание: без комментария отправить нельзя; с обоснованием — контрпредложение уходит клиенту. Превышение больше чем на 20 % запрещено полномочиями.
6. Как `vektor`: «Банк предлагает другие условия».

Автотест: `test_manual_review_info_request_and_counter_offer`.

C6. Кредит под залог — `grand` и `analyst`

1. Как `analyst` откройте заявку ООО «Гранд Девелопмент», «Взять в работу» → «Одобрить». Ожидание: расчёт условий без ограничений политики, кнопка «Одобрить» активна.
2. Подтвердите. Ожидание: статус «Предложение выдано».
3. Войдите как `grand`: «Предложение готово» → «Кредит одобрен».

Автотест: «аналитик рассматривает заявку вручную и одобряет её» (Playwright).

C7. Молодой бизнес — `start` и `analyst2`

1. Как `start` откройте подбор: первым предлагается «Кредит для начинающего бизнеса», кредит на развитие недоступен с пояснением «Доступен бизнесу от 1 года».
2. Отправьте заявку на кредит для начинающего бизнеса. Ожидание: ручное рассмотрение (класс риска C).
3. Как `analyst2`: «Отказать» с причиной и комментарием. Ожидание: клиент видит отказ и понятную причину.

Автотест: `test_young_business_goes_to_analyst_and_is_rejected`.

C8. Стоп-факторы — `romashka`, `meridian`

1. Отправьте любую заявку.
2. Ожидание: «Банк не может выдать кредит» с причиной простыми словами: компания в процессе ликвидации (`romashka`) или есть просроченный долг (`meridian`).

Автотесты: `test_liquidating_company_rejected_by_stop_factor`, `test_current_overdue_rejected`, «стоп-фактор» (Playwright).

C9. Сбой внешнего источника — `admin`, `altair`, `analyst`

1. Как `admin` на странице «Конвейер и сбои» переключите «Бюро кредитных историй» в режим «Ошибка».
2. Как `altair` отправьте экспресс-кредит на 2 000 000 ₽ на 12 мес. Ожидание: заявка в статусе «Проверяем заявку», шаг «Кредитная история» — «Ждём ответа бюро кредитных историй», с пояснением, что заявка не потеряется.
3. Как `analyst` на странице «Конвейер и сбои» заявка видна в списке ожидающих источник.
4. Как `admin` верните источник в рабочее состояние. Ожидание: конвейер продолжает с того же шага (или сразу — по кнопке «Повторить сейчас» у аналитика), заявка одобряется.
5. Если источник недоступен дольше лимита (15 минут), заявка уходит аналитику с флагом F05; после восстановления аналитик нажимает «Перезапустить автоматическую проверку».

Автотест: `test_source_outage_waits_and_recovers`.

C10. Разграничение доступа

1. Неверный пароль — сообщение «Не удалось войти».
2. Клиент, открывший `/bank`, попадает в свой кабинет; сотрудник банка, открывший `/client`, — на рабочее место.
3. Клиент не видит заявку другой компании: API отвечает 404.
4. Клиент не может вызвать методы рабочего места банка: 403.

Автотесты: `test_client_cannot_see_other_company_application`, «вход с неверным паролем и разграничение доступа», «сотрудник банка не может открыть кабинет клиента».

C11. Надёжность действий

1. Повторная отправка заявки или подписание с тем же `Idempotency-Key` не создают дубликатов (повтор в `apply` в API-тестах).
2. Недопустимое действие, например отказ от предложения у черновика, отклоняется с кодом 409 и понятным сообщением (`test_invalid_transition_is_rejected`).
3. Одновременные действия: откройте одну заявку у клиента и у аналитика; если клиент отозвал заявку, решение аналитика уже не пройдёт — он увидит, что действие недоступно, а данные на экране обновятся.

C12. Чат и отзыв заявки

1. Клиент пишет вопрос в блоке «Вопросы по заявке», аналитик отвечает во вкладке «Чат» досье. Сообщения появляются у второй стороны без перезагрузки.
2. Клиент может отозвать активную заявку до подписания («Отозвать заявку»): она закрывается, текущие задания конвейера снимаются.

С13. ИИ-помощник

1. Без модели (`AI_ENABLED=false`, по умолчанию) подбор по описанию работает на правилах — как в С1.
2. С моделью (см. ai.md) разбираются и фразы, где нужен смысл: «Планируем купить экскаватор за 7,5 млн, кредит на 60 месяцев». В досье аналитика появляется сводка «сильные стороны / риски».
3. Качество разбора: `make ai-eval` — отчёт по эталонному набору из 32 фраз.

Автоматические тесты

Набор	Где	Что проверяет	Запуск
Юнит-тесты домена	`backend/tests/unit`	Финансовые расчёты, скоринговая карта, кредитная политика, разбор потребности (правила и ответы модели без сети), качество на эталонном наборе	`make test`
Симулятор	`simulator/tests`	Ответы внешних систем и режимы сбоев	`make test`
Сценарии через API	`backend/tests/integration`	14 сценариев: из этого документа, а также удаление черновика, ошибки ввода и сброс демо-данных	`make test-api`
Сценарии в браузере	`e2e/tests`	6 сценариев Playwright: ключевой путь клиента, запрос выписки, стоп-фактор, решение аналитика, доступ по ролям	`make e2e`

Все наборы запускаются в CI на каждый коммит: линтеры и юнит-тесты, затем сборка образов, `docker compose up` и сценарии через API и в браузере (ci.yml).

Документ 10 из 14 · docs/ai.md

ИИ-помощник

ИИ в решении — необязательное дополнение. Основной сценарий (подбор, заявка, проверки, решение, подписание, выдача) работает без модели. Если модель выключена, не настроена или не ответила вовремя, подбор работает на правилах, и пользователь не видит ошибки.

Что делает модель

Где	Что делает	Чем подстрахован
Кабинет клиента, шаг «Подбор»	Разбирает описание задачи своими словами («нужно закупить товар к сезону на 3 млн, верну за полгода») и заполняет цель, сумму и срок	Разбор на правилах (ключевые слова и регулярные выражения). Клиент видит распознанные параметры и может их изменить до отправки
Рабочее место аналитика, досье	Пишет сводку: 3–5 пунктов «сильные стороны / риски» по данным досье	Блок помечен как подсказка ИИ. Если модель недоступна, блок не показывается

Модель **не принимает кредитных решений** и не влияет на скоринг. Решение принимает скоринговая карта и кредитная политика (credit-policy.md), в ручных случаях — аналитик.

Какие данные уходят в модель

- Разбор потребности — только текст, который ввёл клиент.
- Сводка досье — название компании, продукт, сумма, срок, финансовые показатели из проверок, факторы скоринга, флаги политики и статусы источников. Паспортных данных и контактов в запросе нет. Все данные в демо — синтетические.

Проверка ответа модели

- Ответ запрашивается в формате JSON (`response_format: json_object`). Блоки рассуждений `<think>…</think>` отбрасываются, к запросу добавляется `/no_think` — для моделей Qwen3 это отключает режим рассуждений и ускоряет ответ.
- Цель принимается только из списка допустимых значений, сумма — от 10 000 ₽ до 10 млрд ₽, срок — от 1 до 240 месяцев. Значение, не прошедшее проверку, заменяется результатом правил.
- Если ответ не разобран или модель не ответила за `LLM_TIMEOUT_SECONDS` секунд — используется результат правил.
- В системном промпте прямо запрещено придумывать суммы и сроки. Это проверяется на эталонном наборе (см. ниже).

Код: `backend/app/services/ai.py`, API: `POST /api/ai/parse-need`, `GET /api/bank/applications/{id}/ai-summary`, `GET /api/ai/status`.

Модели

Используются только модели с открытыми весами. Подключение — через API в формате OpenAI (`/v1/chat/completions`), поэтому подходит и модель организаторов, и локальная Ollama.

Вариант	Модель	Лицензия весов	Где работает
Модель организаторов хакатона	Qwen (имя модели — из письма организаторов)	открытые веса	сервер организаторов, доступ по ключу
Локально, по умолчанию	`qwen3:4b` в Ollama	Apache 2.0	компьютер, на котором запущено решение

Можно подключить и другую модель с открытыми весами (Llama, Mistral, Gemma, DeepSeek) — достаточно указать её имя в `LLM_MODEL`.

Как включить

Настройки задаются в файле `.env` в корне проекта (образец — `.env.example`). Файл `.env` не попадает в репозиторий, поэтому ключ остаётся только у вас.

Вариант 1. Модель организаторов

```env
AI_ENABLED=true
LLM_BASE_URL=<адрес API из письма организаторов, оканчивается на /v1>
LLM_API_KEY=<ключ из письма>
LLM_MODEL=<имя модели из письма>
```

```bash
docker compose up -d   # пересоздаст api с новыми настройками
```

Вариант 2. Ollama на своём компьютере (вне Docker)

Ollama работает на хосте, остальные компоненты — в Docker. Контейнер `api` обращается к хосту по имени `host.docker.internal` (в `docker-compose.yml` это имя прописано и для Linux).

1. Установите Ollama: https://ollama.com/download (Linux: `curl -fsSL https://ollama.com/install.sh | sh`).
2. Скачайте модель (около 2,5 ГБ):

   ```bash
   ollama pull qwen3:4b
   ```

3. Запустите Ollama так, чтобы она принимала подключения из контейнеров:
   - **Windows и macOS** (Docker Desktop): достаточно запущенного приложения Ollama.
   - **Linux**: Ollama должна слушать не только `127.0.0.1`:

     ```bash
     OLLAMA_HOST=0.0.0.0:11434 ollama serve
     ```

     Если Ollama установлена как служба: `sudo systemctl edit ollama`, добавьте `[Service]` / `Environment="OLLAMA_HOST=0.0.0.0:11434"`, затем `sudo systemctl restart ollama`. Порт 11434 не открывайте во внешнюю сеть.

4. В `.env`:

```env
AI_ENABLED=true
LLM_BASE_URL=http://host.docker.internal:11434/v1
LLM_API_KEY=
LLM_MODEL=qwen3:4b
```

5. `docker compose up -d`.

Без видеокарты `qwen3:4b` отвечает за 5–30 секунд в зависимости от процессора. Если ответ не приходит за `LLM_TIMEOUT_SECONDS` (по умолчанию 40), подбор продолжит работать на правилах.

Проверка

```bash
docker compose exec api python -m app.ai_eval --llm
```

Команда прогонит эталонный набор через модель и покажет точность. Если модель недоступна, в строке «Ответ на правилах» будет число фраз, разобранных без неё.

В интерфейсе: войдите как `veter` / `demo`, нажмите «Подобрать кредит», опишите задачу и нажмите «Подобрать по описанию» — параметры заполнятся автоматически.

Оценка качества

Эталонный набор — 32 фразы предпринимателей с разметкой цели, суммы и срока: `backend/app/ai_eval/golden_set.json`. В набор входят фразы с суммами в разных форматах («1 500 000 рублей», «400к», «2,5 млн», «полтора миллиона»), сроками («полгода», «на полтора года», «через 2 месяца»), отвлекающими числами («25 сотрудников», «три фургона») и фразы без суммы или срока.

Метрики:

- точность по каждому полю и доля фраз, где все три параметра распознаны верно;
- **придуманные значения** — сколько раз названа сумма или срок, которых в тексте нет. Должно быть 0: выдуманный параметр хуже пустого;
- число ответов на правилах из-за сбоя модели;
- время ответа (медиана и 95-й перцентиль).

Запуск:

```bash
cd backend
uv run python -m app.ai_eval                # только правила
uv run python -m app.ai_eval --llm --out ../docs/ai-eval-report.md   # правила и модель
```

Результат на правилах (проверяется в CI тестом `tests/unit/test_ai_golden.py`, порог — 90 % по каждому полю):

Показатель	Правила
Цель	97 % (31/32)
Сумма	100 % (32/32)
Срок	97 % (31/32)
Все три параметра верно	94 % (30/32)

Показатель	Правила
Придуманные сумма или срок	0

Правила ошибаются на фразах, где нужен смысл, а не ключевое слово: «купить экскаватор» (слова нет в словаре оборудования) и «лет на пять» (обратный порядок слов). Именно здесь модель полезнее всего.

Сводку досье автоматически оценить сложнее, поэтому её проверяют вручную по чек-листу: каждый пункт опирается на данные досье, в тексте нет чисел, которых нет в досье, сводка не содержит рекомендации «одобрить» или «отказать».

Ограничения

- Модель видит только текст клиента и не знает его финансового положения: она заполняет форму, а не подбирает условия. Условия и лимит считает кредитная политика.
- Качество разбора зависит от модели. Для маленьких локальных моделей правила остаются страховкой: значения, не прошедшие проверку, заменяются результатом правил.
- Сводка досье кешируется в памяти процесса `api` для пары «заявка — решение» и сбрасывается при перезапуске.

Документ 11 из 14 · docs/design.md

Интерфейс

Кабинет и рабочее место аналитика собраны на дизайн-системе Альфа-Банка и сверены с alfabank.ru: клиент банка должен чувствовать себя в знакомом продукте.

Дизайн-система

- Компоненты — `@alfalab/core-components` (MIT): кнопки, поля, слайдеры, шаги, вкладки, плашки, модальные окна. Иконки — `@alfalab/icons-glyph` (MIT).
- Темы: `corp` — для клиентов-юрлиц и ИП (главная кнопка красная), `intranet` — для сотрудников банка (главная кнопка чёрная). Тема подключается в макете раздела (Theme.tsx).
- Цвета — палитра `bluetint` дизайн-системы. Собственных цветов почти нет, всё на токенах.
- Логотипы и фирменная графика банка не используются: у прототипа свой знак.

Шрифт

Фирменные шрифты банка — Styrene UI и Alfa Interface Sans — распространяются по коммерческой лицензии, а ТЗ запрещает закрытые лицензии. Вместо них используется Onest (OFL 1.1): из свободных шрифтов с кириллицей он ближе всего к Styrene по рисунку — геометричные круглые «о», прямой хвост у «у», плоские «ножки» у «д», широкие пропорции. Сравнивали Inter, Onest, Manrope, Golos Text, Geologica, Wix Madefor и Montserrat на одной фразе с alfabank.ru при одинаковой ширине строки.

Шрифт подключён локально из пакета `@fontsource/onest` (начертания 400, 500, 700) и работает без интернета. Все шрифтовые токены дизайн-системы (`--font-family-system` , `--font-family-styrene` , `--font-family-alfasans`) указывают на Onest.

Компоновка

Значения сняты со скриншотов alfabank.ru.

Элемент	Как сделано	Значение
Шапка	Чёрная, белые пункты меню, активный пункт — тёмная плашка	`#000` , плашка `#1D1D1F`
Заголовок страницы	На чёрной полосе под шапкой, крупный жирный	Onest 700
Контент	Белый «лист» со скруглёнными верхними углами поверх чёрной полосы	радиус 40 px
Блоки	Серые панели без теней	`#F2F3F5` , радиус 32 px
Карточки внутри панелей	Белые	радиус 20–24 px
Поля ввода и второстепенные кнопки на сером	Полупрозрачные токены дают нужные оттенки	`#E6E8EC` , `#DBDDE2`
Главное действие	Красная кнопка, одна на экран	`#EF3124`
Выбранный вариант (чипы, плитки)	Чёрная заливка или чёрная рамка	`#212124`

Элемент	Как сделано	Значение
Основной текст	`rgba(3, 3, 6, 0.88)`	на белом ≈ `#212124`

Стили карточек описаны парами «основной / второстепенный фон». Внутри контента эти два токена меняются местами (global.css, правило `.page`), поэтому на любой глубине вложенности соседние уровни контрастны: белая страница → серая панель → белая карточка → серый блок.

Принципы

- Одно главное действие на экране, всё остальное — второстепенные кнопки.
- Статус заявки и следующий шаг всегда вверху страницы заявки.
- Суммы — с неразрывными пробелами и знаком ₽, в таблицах — моноширинные цифры.
- Тексты для клиента — без банковского жаргона: «Проверяем заявку», «Ждём ответа бюро кредитных историй».
- Ошибки — плашкой рядом с действием, с понятной причиной.

Экраны

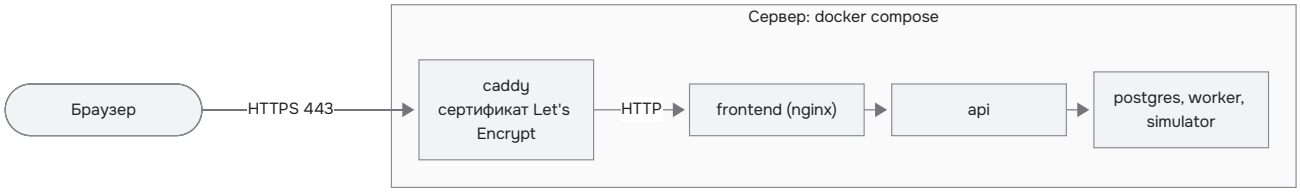
Раздел	Экран	Назначение
Вход	`/login`	Вход и быстрый вход под тестовыми учётными записями
Клиент	`/client`	Главная: приветствие и подбор кредита, заявки, требующие действия, список заявок, карточка компании
Клиент	`/client/new`	Мастер заявки: подбор продукта и расчёт, данные заявки, проверка и отправка
Клиент	`/client/applications/:id`	Заявка: текущий этап и действие, шаги проверки, предложение, подписание, кредит и график, чат, история, документы
Банк	`/bank`	Очередь заявок: счётчики, фильтры, поиск, срок рассмотрения
Банк	`/bank/applications/:id`	Досье и решение по заявке
Банк	`/bank/pipeline`	Конвейер и сбои: состояние внешних систем, управление сбоями, сброс демо-данных
Банк	`/bank/audit`	Журнал действий

Вёрстка рассчитана на экраны от 1280 px — организаторы уточнили, что адаптивность не обязательна. На ширине меньше 1024 px колонки перестраиваются в одну.

Документ 12 из 14 · docs/deploy.md

Публикация на внешнем сервере

Организаторы рекомендуют заранее поднять решение на бесплатном внешнем сервере, чтобы показать его, даже если у жюри не получится запуск в Docker. Стенд разворачивается тем же `docker compose`, перед ним ставится Caddy, который сам выпускает сертификат Let's Encrypt.



Что нужно

- Виртуальная машина с Linux (например, Ubuntu 22.04 или 24.04), 2 vCPU, 4 ГБ памяти, 15 ГБ диска. Подойдёт бесплатная ВМ облачного провайдера, например Always Free в Oracle Cloud, или любой VPS. Базовые образы мультиархитектурные, а в lock-файле фронтенда есть сборки инструментов и для ARM.
- Открытые входящие порты 22, 80 и 443.
- Доменное имя, которое указывает на IP сервера. Если своего домена нет, подойдёт имя вида `203-0-113-10.sslip.io`, где цифры — IP сервера через дефис: сервис sslip.io сам отвечает этим адресом.

Установка

1. Установите Docker с Compose v2 (официальный скрипт):

```bash
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER && newgrp docker
```

2. Склонируйте репозиторий на сдаваемой версии:

```bash
git clone <адрес репозитория> alfa-hakaton
cd alfa-hakaton
git checkout v1.0.0
```

3. Создайте `.env` из образца и задайте значения для сервера:

```bash
cp .env.example .env
```

```env
DOMAIN=203-0-113-10.sslip.io
APP_PORT=127.0.0.1:8080
JWT_SECRET=<результат openssl rand -hex 32>
POSTGRES_PASSWORD=<результат openssl rand -hex 16>
```

`APP_PORT=127.0.0.1:8080` закрывает прямой доступ к стенду снаружи — весь трафик идёт через HTTPS.

4. Запустите:

```
docker compose -f docker-compose.yml -f deploy/docker-compose.prod.yml up -d --build --wait
```
BASH

Первая сборка занимает 5–10 минут.

5. Проверьте: `https://<DOMAIN>/api/health` отвечает `{"status":"ok"}`, на `https://<DOMAIN>` открывается страница входа.

Перед показом

- Сбросьте демо-данные: `make reset` на сервере или кнопка «Сбросить демо-данные» на странице «Конвейер и сбои» под `admin`.
- Пройдите ключевой сценарий из test-scenarios.md (C1).
- Если включаете ИИ с моделью организаторов, добавьте в `.env` параметры из ai.md и повторите команду запуска.

Обновление и остановка

```
git pull
docker compose -f docker-compose.yml -f deploy/docker-compose.prod.yml up -d --build --wait
docker compose -f docker-compose.yml -f deploy/docker-compose.prod.yml down      # остановить, данные
  сохранятся
```
BASH

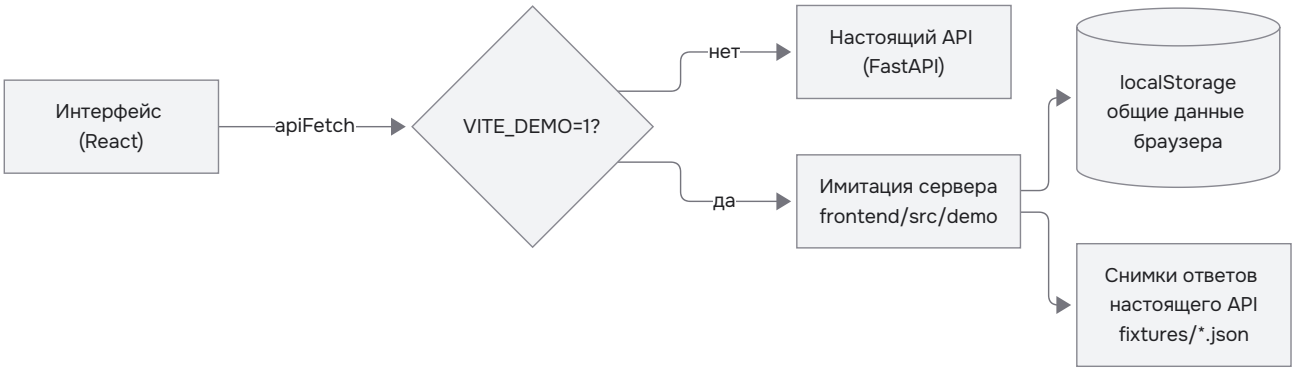
Безопасность стенда

- Данные на стенде синтетические, тестовые учётные записи с паролем `demo` известны всем — это демо, а не боевой контур.
- Обязательно замените `JWT_SECRET` и `POSTGRES_PASSWORD`. Ключ LLM храните только в `.env` на сервере.
- Наружу открыты только 80 и 443 (Caddy). База, API, обработчик и симулятор доступны лишь внутри сети Docker.
- Включите межсетевой экран: `sudo ufw allow 22,80,443/tcp && sudo ufw enable`.

Документ 13 из 14 · docs/demo-mode.md

Демо-версия без сервера (GitHub Pages)

Интерфейс можно показать без Docker и без бэкенда: в сборке с `VITE_DEMO=1` все запросы к API обслуживает имитация сервера прямо в браузере. Это тот же фронтенд — кабинет клиента и рабочее место банка с теми же экранами, тестовыми учётными записями и сценариями из test-scenarios.md, — но данные хранятся в браузере, а конвейер проверок работает во вкладке. Такую сборку публикует на GitHub Pages workflow pages.yml.



Что это и чем не является

- **Это витрина интерфейса и сценариев.** Подбор продукта, заявка, шаги проверки в реальном времени, предложение, подписание, выдача с графиком, запросы информации, чат, очередь и досье аналитика, решения в пределах полномочий, сбои внешних систем и сброс данных работают так же, как на стенде.
- **Расчёты и решения — те же, что на сервере.** В браузер перенесены каталог продуктов и формулы (`backend/app/domain`: аннуитет и график, признаки, скоринговая карта, кредитная политика, проверка полномочий аналитика), разбор описания задачи на правилах (`backend/app/services/ai.py`) и разбор выписки 1С/CSV. Ответы внешних источников по каждой тестовой компании сняты с настоящего симулятора. Поэтому у `veter` автоодобрение, у `smirnova` контрпредложение на 440 000 ₽, у `tehnomaster` два варианта, у `orlov` запрос выписки, `romashka` и `meridian` получают отказ с теми же текстами — проверка `npm run demo:check` сверяет это со снимками настоящего API.
- **Это не замена бэкенду.** Настоящая логика, хранение, безопасность и интеграции живут в `backend/` и `simulator/`. Имитация — упрощённая копия для показа: нет базы данных, настоящей авторизации, очереди заданий на сервере и проверок на гонки между пользователями.
- **Данные свои у каждого браузера.** Всё лежит в `localStorage` под одним ключом и общее для всех демо-пользователей этого браузера: аналитик одобрил заявку — выйдите и войдите клиентом в том же браузере, и предложение уже в кабинете. В другом браузере, на другом устройстве или в приватном окне — свои данные.

Над шапкой демо-версии всегда видна полоса «Демо-версия без сервера: данные хранятся в этом браузере».

Запуск на своём компьютере

Нужен только Node.js 22:

```bash
cd frontend
npm ci
npm run dev:demo          # http://localhost:5173, режим разработки
```

Сборка, как на GitHub Pages, и её просмотр:

```bash
npm run build:demo        # dist/ с имитацией сервера
npm run preview           # http://localhost:4173
```

Обычные команды `npm run dev` и `npm run build` собирают интерфейс для настоящего API — кода имитации в такой сборке нет.

Сбросить демо-данные: войдите как `admin` (пароль `demo`), откройте «Конвейер и сбои» → «Сбросить демо-данные». Стартовые заявки `vektor` и `grand` снова пройдут конвейер и встанут в очередь аналитика. Можно и просто очистить данные сайта в браузере.

Публикация на GitHub Pages

Workflow pages.yml запускается при изменениях в `frontend/` на ветке `main` или вручную (Actions → «GitHub Pages — демо-версия интерфейса» → Run workflow):

1. собирает `npm run build:demo` с `VITE_BASE=/<репозиторий>/` — сайт на Pages живёт в подкаталоге, адреса ресурсов и маршрутов строятся от него (`import.meta.env.BASE_URL`);
2. копирует `index.html` в `404.html`: Pages отдаёт его на неизвестные адреса, поэтому прямые ссылки на страницы приложения тоже открываются;
3. публикует `frontend/dist`.

Чтобы включить публикацию, один раз в настройках репозитория выберите **Settings → Pages → Build and deployment → Source: GitHub Actions**. Адрес демо-версии: `https://<владелец>.github.io/<репозиторий>/`.

Как устроена имитация

Все обращения интерфейса к серверу идут через `apiFetch` в client.ts: JSON-запросы, загрузка файлов, скачивание документов и поток событий stream.ts. В демо-сборке `apiFetch` подгружает отдельный чанк `frontend/src/demo/server.ts` и вызывает `demoFetch(path, init)`, который возвращает настоящие объекты `Response` — интерфейс не знает, что сервера нет.

Файл	Что делает
`server.ts`	Разбор запроса, маршрутизация, ответы и ошибки в формате FastAPI (`{"detail": ...}`, 401/403/404/405/409/422)
`routes.ts`	Обработчики всех методов API — порт `backend/app/api/*.py` с теми же проверками, порядком проверок и текстами ошибок
`views.ts`	Сборка ответов: карточка заявки, досье, очередь, сводка, уведомления, журнал — порт `services/views.py`

Файл	Что делает
`workflow.ts`	Машина состояний заявки, история, журнал действий, события и уведомления — порт `services/workflow.py`
`pipeline.ts`, `scheduler.ts`, `timing.ts`	Обработчик конвейера: шаги DATA_CHECK → EGRUL → FNS → GIRBO → BKI → BANK → SCORING → POLICY с задержками как у симулятора, формирование договора, выдача; сбои источников, повторы с растущей паузой, circuit breaker, ожидание до 15 минут и флаг F05; истечение предложений и запросов
`stream.ts`	`GET /api/stream`: поток Server-Sent Events через `ReadableStream`, `Last-Event-ID` при переподключении
`store.ts`, `seed.ts`	Данные в `localStorage` (ключ `credit-cabinet-demo-state-v1`), транзакции с откатом при ошибке, исходные данные и сброс
`data.ts`	Пользователи, профили компаний и ответы источников из снимков; даты сдвигаются к сегодняшнему дню
`engine/`	Порт домена: `catalog`, `finance`, `features`, `scoring`, `policy`, `text`, разбор потребности `ai` и выписки `statements`
`DemoNotice.tsx`	Полоса «демо-версия» над шапкой
`fixtures/`	Снимки ответов настоящего API (см. ниже)
`selfcheck.ts`	Сверка имитации со снимками, запускается `npm run demo:check`

Особенности, которые стоит знать:

- **Конвейер работает во вкладке.** Задания хранятся вместе с данными, поэтому после перезагрузки страницы проверка продолжается с того же шага; если вкладка была закрыта, конвейер «дорабатывает» пропущенное время, как сервер. Если открыто несколько вкладок, задания выполняет одна — видимая (Web Locks API), остальные получают изменения через `localStorage`.
- **Внешние источники** отвечают данными, снятыми с симулятора для каждой из десяти тестовых компаний. Даты в ответах отсчитаны от дня съёмки и сдвигаются к сегодняшнему дню, поэтому срок бизнеса, обороты по месяцам и история кредитов не «стареют»: «Новому старту» всегда 5 месяцев.
- **Договор** — один настоящий PDF со стенда (`frontend/public/demo/agreement-sample.pdf`, заявка `veter` на 3 000 000 ₽ на 24 мес.), он открывается для любой заявки.
- **Загруженные файлы** хранятся только в памяти вкладки. Разобранная выписка и запись о документе сохраняются, а открыть сам файл после перезагрузки страницы нельзя — вместо него откроется пояснение.

Снимки ответов API

Файлы в `frontend/src/demo/fixtures/` получены с запущенного стенда скриптом capture-demo-fixtures.mjs: он сбрасывает демо-данные, проходит сценарии всех тестовых компаний через настоящий API и сохраняет ответы, а в конце снова сбрасывает данные.

Файл	Используется	Что внутри
`meta.json`, `products.json`	имитацией	Справочники `/api/meta` и каталог `/api/products`

Файл	Используется	Что внутри
`users.json`, `companies.json`	имитацией	Пользователи (`/api/auth/login`), профили компаний (`/api/company`) и ответы ЕГРЮЛ, ФНС, ГИР БО, БКИ и банка по каждой компании из досье аналитика
`faults.json`, `agreement.json`, `ai-status.json`	имитацией	Внешние системы симулятора, размер и SHA-256 образца договора, режим ИИ
`scenarios.json`	проверкой	Заявки и досье в ключевых состояниях по каждому сценарию
`examples.json`	проверкой	Полные ответы методов — для сверки набора полей
`quotes.json`, `previews.json`, `statements.json`, `ai-golden.json`	проверкой	Подбор продуктов, расчёт условий аналитиком, разбор демо-выписок, разбор фраз эталонного набора `backend/app/ai_eval/golden_set.json` правилами сервера
`notifications.json`, `errors.json`	проверкой	Тексты уведомлений и ошибок

В сборку попадают только файлы из первых трёх строк (около 50 КБ), остальные нужны проверке.

Обновить снимки после изменений в бэкенде:

```bash
docker compose up -d --build --wait     # стенд на http://localhost:8080
cd frontend
npm run demo:capture                    # или: node scripts/capture-demo-fixtures.mjs --api <адрес
  стенда>
npm run demo:check                      # имитация совпадает с новыми снимками?
```

Если проверка нашла расхождения — значит, изменилась логика сервера, и её нужно перенести в имитацию (`engine/`, `routes.ts`, `views.ts`). Данные, сохранённые в браузерах со старыми снимками, при первом открытии новой версии сбрасываются к исходным.

Как расширить имитацию при новых методах API

1. Добавьте маршрут в таблицу `ROUTES` в `routes.ts` и обработчик рядом с остальными: проверка роли (`clientOnly`, `bankStaff`, `adminOnly`), проверка тела через `Validator` (ошибки 422 как у Pydantic), загрузка заявки через `load` или `loadForBank` (404 как на сервере), изменения — внутри `transact`, чтобы при ошибке данные откатились.
2. Смену статуса делайте только через `transition` из `workflow.ts` — она пишет историю, журнал, события и ставит задания конвейеру. Новые сущности добавьте в `DemoState` (`model.ts`), начальные значения — в `seed.ts`. Если меняется формат хранимых данных, увеличьте `STATE_VERSION` в `seed.ts`: старые данные в браузерах сбросятся к исходным.
3. Ответ соберите в `views.ts` с типами из `frontend/src/api/types.ts` — так TypeScript проверит, что имитация отдаёт то же, что ждёт интерфейс.
4. Если метод вызывается после изменения данных, разбудите обработчик (`kick()`), если нужно обновить интерфейс у других пользователей — создайте событие (`emit` или `refresh`).
5. Добавьте вызов метода в скрипт съёмки и сверку в `selfcheck.ts`, обновите снимки и запустите `npm run demo:check`.

Проверки

- `npm run demo:check` (в `frontend`) — проходит сценарии всех компаний через `demoFetch` в Node с ускоренным временем и сверяет статусы, предложения, суммы, платежи и переплаты, график, скоринг, решения и лимиты, расчёт условий, подбор, тексты ошибок и уведомлений, разбор выписок и 32 фраз эталонного набора со снимками настоящего API. Около 5 секунд.
- `npm run test:demo` (в `e2e`) — те же сценарии Playwright, что и для стенда, на демо-сборке: Playwright сам собирает её и поднимает `vite preview` на порту 4173. Состояние перед тестом сбрасывается очисткой хранилища, а сценарии с двумя участниками выполняются в одном браузере с выходом и входом. Локально можно указать установленный Chromium: `PW_CHROMIUM_PATH=/путь/к/chrome npm run test:demo`.
- В CI это задание «Сквозные сценарии на демо-версии без сервера» (ci.yml), ему не нужен Docker.

Ограничения

- Данные живут в одном браузере, их не видят другие люди и устройства. Ёмкость `localStorage` ограничена (около 5 МБ); старые события и записи журнала отбрасываются, лента уведомлений и журнал показывают последние.
- Нет настоящей авторизации: пароль у всех `demo`, токен не истекает, права проверяются так же, как на сервере, но только внутри браузера.
- ИИ работает только на правилах: модели нет, сводка досье для аналитика недоступна — как на стенде с выключенным ИИ.
- Список компаний фиксирован: заявку можно подать только от десяти тестовых компаний, их данные во внешних источниках не меняются.
- Конвейер выполняется, пока открыта хотя бы одна вкладка демо-версии. Действие во вкладке, закрытой в ту же долю секунды, может не сохраниться.
- Шаг, который выполняется прямо сейчас, показан как «выполняется» с текстом ожидания. На стенде результат шага появляется сразу после выполнения.
- Одновременных действий разных людей нет, поэтому ответа «Заявка изменилась, обновите страницу» (409) имитация не даёт.
- Ошибки проверки ввода (422) повторяют сообщения Pydantic для основных случаев; путь до поля (`loc`) может отличаться.
- Суммы считаются в числах с плавающей точкой с округлением как у `Decimal` на сервере; на всех снятых сценариях результаты совпадают до копейки, но в редких пограничных случаях возможна разница в копейку.
- Логика перенесена вручную. После изменений в `backend/app/domain`, `services` или `api` обновите снимки и запустите проверку — расхождения покажут, что поправить в имитации.

Документ 14 из 14 · docs/research/interview-guide.md

Сценарий интервью с предпринимателями

Цель: понять, какие задачи и трудности возникают у малого бизнеса при получении кредита, и на этой основе выбрать сценарии MVP. Результаты сводятся в таблицу «боль → сценарий → функция» — файл `findings.md` рядом с этим сценарием, его заводят по итогам первых интервью.

Кого ищем

3–5 человек, которые за последние два года получали или пытались получить кредит, овердрафт или кредитную карту для бизнеса:

- собственник или директор малого бизнеса (ООО или ИП);
- бухгалтер или финансист, который готовил документы для банка.

Подойдут знакомые, родственники, владельцы магазинов и кафе по соседству. Лучше, чтобы отрасли и роли различались.

Правила

- 20–30 минут, созвон или вживую.
- Спросите разрешения на запись. Имена, названия компаний и суммы, по которым можно узнать человека, в отчёт не попадают.
- Спрашивайте о том, что уже было («как было в последний раз»), а не о гипотетическом будущем («стали бы вы пользоваться»).
- Не подсказывайте ответы и не показывайте наше решение до последнего блока.
- Уточняйте: «Почему?», «Что было дальше?», «Можете привести пример?»

Вопросы

1. Контекст — 3 мин

- Чем занимается бизнес, сколько лет работает, сколько сотрудников?
- Какая у вас роль в финансах компании?

2. Последний раз, когда понадобились деньги — 10 мин

- Расскажите про последний раз, когда бизнесу понадобилось финансирование. Для чего? Какая сумма и на какой срок?
- С чего вы начали? Где искали варианты?
- Как выбирали банк и продукт? Что было непонятно?
- Какие данные и документы у вас попросили? Что было сложно собрать?
- Сколько всё заняло? Где пришлось ждать дольше всего?
- Как вы узнавали, что происходит с заявкой?
- Чем всё закончилось? Если отказали — объяснили почему? Предложили что-то взамен?

3. Трудности и обходные пути — 5 мин

- Что в этом процессе раздражало больше всего?
- Что приходилось делать несколько раз: вводить, отправлять, объяснять?
- Кто ещё в компании участвовал? Как вы делили работу?
- Если бы можно было изменить одну вещь, что бы вы изменили?

4. Потребность своими словами — 3 мин

- Опишите, на что вам были нужны деньги, так, как рассказали бы знакомому.

Запишите ответ дословно. Эти фразы войдут в тестовый набор для подбора продукта по описанию задачи.

5. Реакция на идеи — 5 мин, только в конце

Коротко опишите идеи или покажите набросок:

- подбор продукта по описанию задачи своими словами;
- данные компании подтягиваются по ИНН, ничего не нужно вводить повторно;
- статус заявки с этапами и понятным «что нужно от вас»;
- вместо отказа — контрпредложение: меньшая сумма или более длинный срок;
- подписание и выдача прямо в кабинете, без звонков и визитов.

Что из этого полезно, а что нет? Чего не хватает?

Завершение

- Кому ещё из знакомых предпринимателей стоит задать эти вопросы?

Как записать результат

Для каждого интервью создайте файл `interviews/NN.md` по шаблону:

MARKDOWN

```
# Интервью NN

| Поле | Ответ |
|---|---|
| Респондент | Роль, отрасль, ООО или ИП, примерный размер бизнеса (без имени) |
| Опыт кредитования | Что брали, когда, чем закончилось |
| Ключевые цитаты | Дословно, 3–5 штук |
| Боли | Что мешало и раздражало |
| Задачи (JTBD) | «Когда …, я хочу …, чтобы …» |
| Обходные пути | Как справлялись с трудностями |
| Реакция на идеи | Что понравилось, что нет, чего не хватает |
| Потребность своими словами | Дословно |
```

Что делаем с результатами

1. **Сводим боли и задачи.** Все боли и задачи (JTBD) собираем в `findings.md`. Для каждой указываем, сколько респондентов её назвали.

2. **Связываем с решением.** Для каждой боли записываем сценарий и функцию решения, которые её закрывают, и приоритет по MoSCoW.
3. **Фиксируем объём MVP.** По таблице определяем, что вошло в MVP и почему. На защите это показывает связь между исследованием и продуктом.